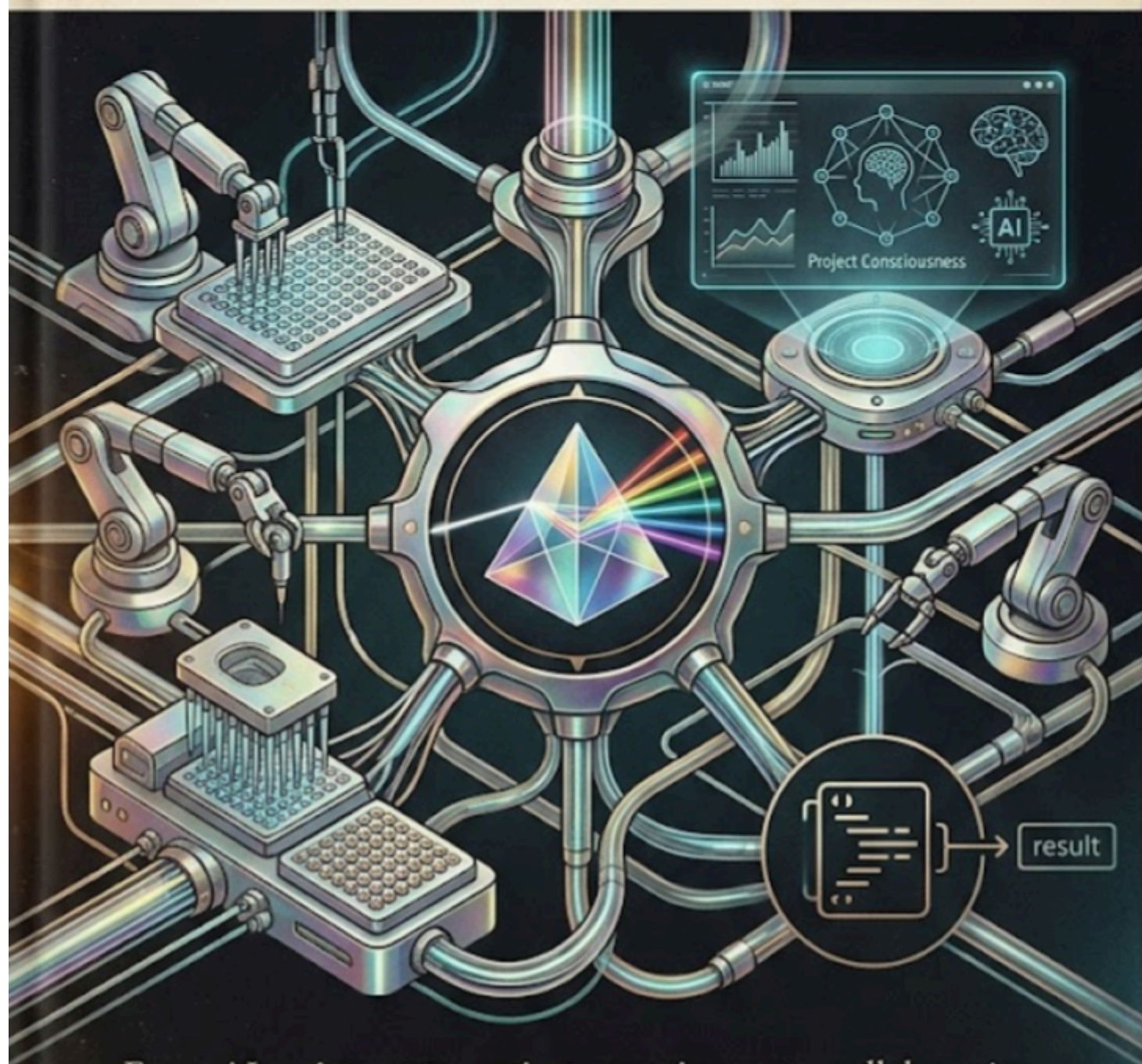


THE FUTURE OF RESEARCH INFRASTRUCTURE

A Vision for Science in the Age of AI



From AI assistants to project consciousness, collaborative research environments, and executable science

THE FUTURE OF RESEARCH INFRASTRUCTURE

A Vision for Science in the Age of AI

*From AI assistants to project consciousness, collaborative research environments, and
executable science*

Axiologic Research

Research vision — August 2026

Research Note

This book grows out of Axiologic Research's work in the Horizon Europe ACHILLES project, where a first generation of an integrated environment for researchers has been explored together with a copilot intended to support specification-driven work, collaboration, and early compliance assistance. That work is an important starting point, but it is not the subject of the book in the narrow sense. The central argument is that the research environment required for the AI era will have to become much broader than any tool we can build in 2026. Today's systems are useful fragments of a future infrastructure whose defining problem is not how to add one more intelligent assistant, but how to preserve scientific judgement, continuity, evidence, responsibility, and collective understanding when machines can produce research artefacts at unprecedented speed.

The book therefore treats existing Axiologic technologies as experimental evidence and as prerequisites for larger roles rather than as a catalogue of products. Work on persistent project workspaces, agent runtimes, explicit process representations, provenance, and neuro-symbolic execution suggests ways to build parts of the required infrastructure. None of these approaches is presented as the unique solution. Their value is that they expose recurring requirements: specialised agents, bounded authority, inspectable execution, replaceable models, explicit versions, provenance, customisation, and the ability to distinguish a proposal from a scientific result. The research programme developed here asks what classes of technology must exist if those requirements are to become routine properties of scientific work.

The argument is intentionally future-facing. It begins with how science is actually done today: through papers, meetings, notebooks, code, datasets, instruments, reviews, emails, repositories, tacit judgement, and institutional memory. It then asks what changes when AI systems become active participants rather than occasional tools. The answer is not a single Research IDE architecture. It is a family of research infrastructures in which conversational interfaces, collaborative rooms, agent populations, versioned project state, evidence systems, compliance services, and automated laboratories can be composed around the needs of a particular scientific community.

The early research strands developed here are presented as one integrated programme rather than as separate source documents. Their ideas are reorganised into a single argument about research infrastructure. External claims are grounded in the public literature and official technical sources listed at the end of the book.

Contents

1	Why Research Infrastructure Must Change
2	The Research Project as a Living System
3	From Scientific Intent to Executable Work
4	Memory, Versioning, and the History of Scientific Thought
5	Agents as Scientific Roles, Not Features
6	Evaluation as a Continuous Layer of Research
7	Choosing Direction When Results Become Cheap
8	Compliance as Continuous Scientific Context
9	Why Future Research Infrastructure Needs Neuro-Symbolic Foundations
10	When Research Infrastructure Reaches the Laboratory
11	A Research Programme for the Infrastructure of AI-Augmented Science
—	Conclusion: The Instrument We Do Not Yet Have

CHAPTER 1

Why Research Infrastructure Must Change

Science already has an infrastructure, but much of it is invisible because researchers have learned to live inside its fragmentation. A contemporary project may be distributed across literature databases, shared folders, notebooks, laboratory software, code repositories, spreadsheets, meeting transcripts, ethics documents, grant systems, email, and the memories of a few experienced people. The arrangement works because humans continuously reconstruct context. They remember why a file exists, which result was abandoned, who distrusted a dataset, what was agreed in a meeting, and which sentence in a paper should not be taken literally. AI changes the scale of this hidden coordination problem. As machines begin to search, write, code, analyse, critique and act in parallel, the rate at which artefacts can be produced may grow far faster than the human ability to reconstruct how those artefacts acquired meaning.

The immediate bottleneck is therefore not intelligence in the abstract. It is the infrastructure around intelligence. A research system needs to know what the project is trying to establish, what evidence has been accepted, which interpretations remain disputed, what an agent is allowed to change, which result supersedes another, what a human approved, and why a promising direction was abandoned six months earlier. Without such infrastructure, more capable models can accelerate local tasks while making the project as a whole harder to understand. The future of AI-assisted science depends as much on the quality of this surrounding environment as on the quality of the models placed inside it.

Axiologic Research arrived at this problem through work in the Horizon Europe ACHILLES project. ACHILLES is explicitly developing a specification-oriented integrated development environment for efficient, compliant and trustworthy AI. In the implementation line led by Axiologic, that environment is built on the AssistOS family of technologies and has become a first generation of a workspace in which people, agents, tools and artefacts can be brought into one project. The current environment is useful, but it should not be confused with the destination described in this book. What exists in 2026 is a small part of the infrastructure that will be required if AI becomes a genuine partner in scientific work rather than a peripheral assistant.

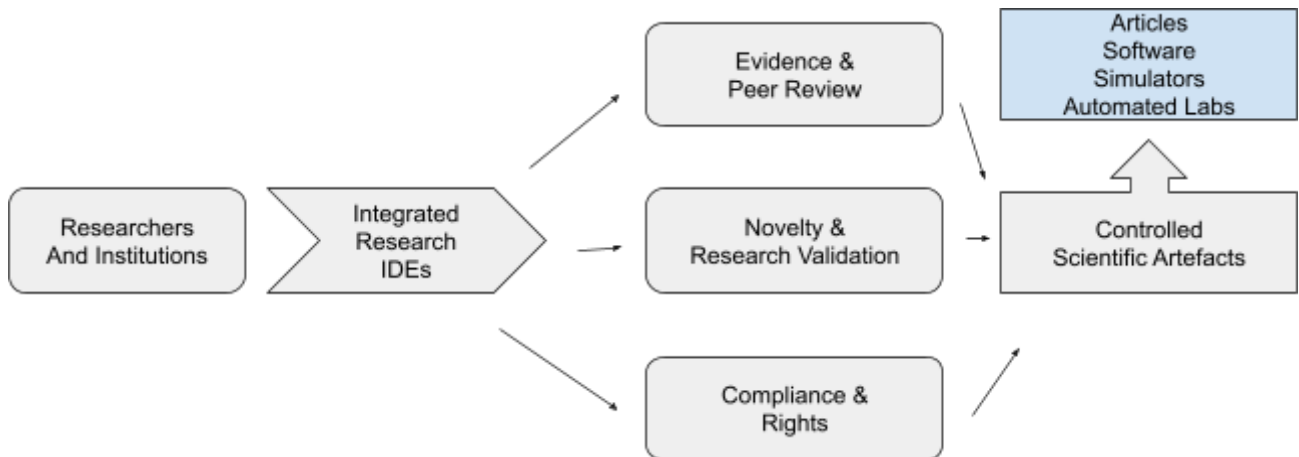
From faster outputs to slower understanding

The gap becomes easier to see if the unit of analysis is changed. Most current AI products are session-centred. A person opens a chat, supplies context, asks for an output and leaves with text, code or a file. Research is not session-centred. It is a long-lived process in which questions are narrowed, hypotheses are revised, evidence changes status, datasets are corrected, methods are replaced, negative results eliminate attractive paths, collaborators disagree, and old assumptions are rediscovered months later. The important object is not the conversation but the scientific project state. A Research IDE is an environment that treats this state as a first-class computational object.

The term IDE is deliberately borrowed from software engineering but must not be interpreted literally. A programming IDE integrates source code, compilers, tests, debuggers, version control and execution because software work became too complex to manage through disconnected tools. Scientific work is reaching a comparable point for a different reason. The researcher now needs documents, datasets, notebooks, repositories, simulations, literature services, models, agents, laboratories, meetings, policies, reviewers and institutional controls to act on a common project without collapsing their different meanings. A scientific IDE is therefore less a specialised editor than an operating environment for inquiry.

The book develops four early research directions inside this larger environment. One direction asks how scientific evaluation and peer review can become evidence-centred and partly executable rather than being reduced to a generated paragraph that imitates a reviewer. Another asks how a system can help determine whether a result is genuinely new, scientifically useful, or merely a reformulation of prior work, while preserving uncertainty and the limits of the searched corpus. A third asks how compliance, privacy, copyright, ethics and local institutional rules can be turned from static paperwork into runtime

constraints on the actions of research agents. The fourth is the Research IDE itself: the persistent state, orchestration, memory, provenance and human interface required to make the other capabilities useful together.



These directions have a common shape. Each begins where fluent language becomes insufficient. A review is not trustworthy because it sounds like a review. A novelty judgement is not reliable because an embedding returned a distant neighbour. A compliance decision is not correct because a language model produced a plausible legal explanation. A research project is not coherent because an agent can summarise yesterday's chat. In each case the system needs explicit objects, evidence boundaries, versioned state, specialised verification, and a visible distinction between proposal and authority. The research problem is to discover how far those properties can be made computational without pretending that scientific judgement can be completely formalised.

This is why the programme is naturally neuro-symbolic and naturally part of trustworthy AI. Neural models are exceptionally useful where the input is language, where concepts are incomplete, where analogies matter, or where a new representation must be proposed. Symbolic and deterministic components are stronger where the system must preserve identity, enforce access, execute a test, check a constraint, compare versions, reproduce a calculation, record provenance or decide whether a declared policy has been satisfied. The interesting architecture is not one in which the symbolic component replaces the neural component. It is one in which each component receives only the authority appropriate to what it can actually establish.

Axiologic's more mature technology lines provide a useful substrate for this programme. the current Axiologic workspace contributes a project-centred workspace and the idea that models should be replaceable while the project remains. the deployment and isolation infrastructure explores deployment of agents in isolated and controllable environments. the agent capability infrastructure explores reusable capabilities and explicit orchestration contracts. the experimental meta-rational execution approach investigates versioned operational harnesses in which a model is a proposal engine rather than the owner of authority. a human-inspectable executable representation and dynamic circuit experiments investigate whether language-mediated procedures can be compiled into inspectable, bounded executable structures. a provenance-oriented data-sharing approach contributes concepts and implementations for controlled data sharing, object history, identity, anchoring and provenance. These components are not the four research results studied in depth here; they are the comparatively mature ground on which the new questions can be attacked.

The maturity distinction matters. It would be easy to draw a large architecture diagram, connect every box with arrows and imply that the problem has been solved. That would be precisely the mistake the programme is designed to resist. In 2026 we can build agent runtimes, project workspaces, versioned artefacts, retrieval systems, workflow engines and provenance stores. We can also prototype structured review cases, novelty dossiers and machine-readable policy packs. What we do not yet know is whether these pieces can support scientific decisions with enough fidelity, calibration and scale to change research practice. The new components belong roughly to the early research range: their

categories are defensible, their mechanisms are partially prototyped, and their value must be established by experiments that can fail.

A useful Research IDE must be able to fail visibly. If the literature search was incomplete, novelty should remain uncertain. If a formal solver verified a constraint that was mistranslated from prose, the trace should expose the translation boundary rather than label the whole result verified. If two domain validators disagree, the disagreement should survive as state. If a compliance rule is inapplicable because the jurisdiction or role is unknown, the system should ask for the missing fact or escalate. If an agent exhausted its time budget before checking an important alternative, absence of evidence must not become evidence of absence. The design objective is not a machine that always has an answer. It is a machine that preserves the conditions under which an answer may deserve authority.

The same principle explains why specification-driven development became central to our current work. AI coding agents become much easier to control when the intended result, constraints, evidence expectations and acceptance conditions exist before implementation. GitHub's Spec Kit is one public sign that software engineering is moving in this direction. Research needs a stronger form of the same idea. A research specification cannot simply state what artifact to build. It may need to state the scientific question, admissible data, theory assumptions, alternative hypotheses, prohibited transformations, acceptable uncertainty, required evidence, privacy boundaries, human decision rights and stopping conditions. The specification becomes an executable contract around a research episode, not a bureaucratic document written after the work.

Table 1. The four early research programmes converge on explicit scientific objects rather than fluent agent output.

Term	Meaning in the research environment
Project state	The long-lived representation of questions, claims, evidence, experiments, decisions, permissions and unresolved uncertainty.
Review Case	A persistent object containing review targets, checks, issues, evidence and disagreement; prose is generated from it rather than used as the primary state.
Advance Case	A structured novelty-and-value assessment tied to a temporal corpus, prior art, mechanisms, evidence and uncertainty.
Compliance Case	A stateful governance object linking the proposed action to actors, data, rights, jurisdiction, policy, controls and audit evidence.
Executable science	Machine-actionable companions to research objects that expose formalizable procedures, dependencies, evidence and authority without pretending all science is formal.

From such specifications, pipelines can be generated without claiming that science itself has become deterministic. The generated pipeline can organise what is formalizable: retrieve a literature corpus, instantiate a method, run code, test an invariant, reproduce a figure, search for a counterexample, compare against a baseline, request a human decision, and record why an action was allowed. Uncertainty remains explicit. The specification does not predict the result. It defines how the system is permitted to look for one and what evidence is required before the result changes project state.

Research must remember alternatives, not only winners

Versioning is the second major inheritance from software engineering, but its scientific role is broader than a convenient history of files. The ACHILLES IDE implementation line already integrates with Git and GitHub because research projects increasingly contain code, documents and machine-generated artefacts that must evolve together. In a future Research IDE, branches and forks can become explicit scientific alternatives. A branch may represent a different preprocessing choice, a rival theory, a new protocol, a reproduction attempt, or a hypothesis that should not yet modify the accepted state. A merge is then not merely a text operation. It is a decision that some evidence, method or interpretation has earned a new status.

These histories are themselves valuable research material. Version graphs reveal which hypotheses survived, which tests changed conclusions, which instructions caused agents to fail, which reviewer objections led to better methods, and which specifications transferred across projects. With appropriate privacy, confidentiality, copyright and governance controls, such histories could become a new kind of training resource: not just a collection of final papers, but traces of scientific correction. The important caveat is that raw Git history is not automatically scientific provenance. Commit identity, data lineage, external tool versions, model calls, laboratory conditions, rights and institutional decisions must be connected to the version graph if it is to support serious audit or future learning.

This is where provenance and data-sharing technology becomes structural rather than decorative. W3C PROV provides a general model of entities, activities and agents involved in producing data or artefacts. RO-Crate provides a practical packaging approach for research objects and now includes mature ways of describing workflows and provenance. A provenance-oriented data-sharing approach explores a different but complementary problem: how bounded digital objects can retain identity, version commitments, rights and verifiable history across organisational boundaries. A Research IDE should not replace these standards or technologies. It should compose them so that a scientific claim can be traced from its current status back through data, method, computation, source, permissions and decisions.

The first four virtual services of the future environment

Peer review becomes the clearest test of whether this architecture provides more than convenient project management. Conventional automated review asks a model to read a paper and write something that resembles a review. An evidence-centred Research IDE starts elsewhere. It compiles the paper, data, code, protocols and citations into a Review Case. Specialised verifiers examine claim-evidence alignment, quantitative consistency, statistical assumptions, citations, reproducibility, artifacts, reporting standards, integrity signals, privacy and prior art. The system builds an issue graph with evidence, confidence and scope. A human reviewer or institution still decides what the issues mean for publication. Automation produces inspectable scientific work before it produces rhetoric.

Novelty assessment has a similar structure. The goal is not to build an oracle that predicts which idea will become important. The useful target is an observable change in scientific state. The system asks what was known before a declared time boundary, which part of the candidate contribution differs, whether the difference concerns wording or mechanism, whether it changes an empirical frontier, whether the supporting evidence is strong, and whether contradictions or negative results alter the interpretation. The output should be an Advance Case: a structured argument about novelty, evidence, uncertainty and possible value that researchers can challenge. Its strongest use may be steering rather than scoring. It can tell an agent or a research team which uncertainty is worth resolving next.

Compliance also changes shape once agents can act. Static review at the beginning of a project is not enough when an agent can decide during execution to upload a dataset to an external model, retrieve a licensed corpus, combine two datasets, install a tool, share a result with a partner or repurpose an earlier artefact. Compliance becomes a stateful runtime problem. A Compliance Case must know what the data is, who is acting, what role the organisation has, what jurisdiction and policy pack applies, what rights attach to the asset, what purpose is declared, what processing has already occurred and what action is proposed. The system can then produce controls, refusals, modifications or escalation requests. It supports governance; it does not become a legal authority.

Table 2. A layered maturity model: build new research services on infrastructure that is already useful independently.

Term	Meaning in the future infrastructure
Operational substrate	Project workspaces, versioning, deployment, identity and collaboration mechanisms that are already useful independently of advanced AI.

Trust substrate	Provenance, permissions, explicit process state, evaluation boundaries and replaceable model interfaces.
Research frontier	Project consciousness, reliable scientific agents, neuro-symbolic execution, continuous evaluation, direction intelligence and automated-laboratory control.
Physical extension	Instrument adapters, experiment specifications, safety gates, schedulers, digital twins and closed-loop laboratories.

Taken together, these components suggest a new technical category. Today's scientific software tends to divide into authoring tools, notebooks, data platforms, workflow engines, laboratory systems, literature databases and generic AI assistants. A Research IDE would cut across those categories by making scientific state and governed machine action the common substrate. It would allow an individual researcher to preserve rationale across months, an institution to enforce policies without reducing science to forms, an industrial R&D team to connect evidence to decisions, and a future research-native company to coordinate large populations of agents and automated laboratories without surrendering control to one opaque model.

The architecture is also relevant if fully autonomous science never arrives. Even modest agents create enough new work to justify better provenance, specification, review, memory and governance. Conversely, if agents become much more capable, the need increases. Stronger models can search more broadly, act faster and generate more alternatives, but they also concentrate greater operational and epistemic authority unless the surrounding system separates roles. The research environment is therefore a robustness strategy under uncertainty about AI progress. It is useful in the conservative future and necessary in the ambitious one.

The most important claim of this book is consequently infrastructural. AI-assisted science will not become trustworthy merely by improving the average intelligence of the model in the loop. It requires a surrounding environment that can remember scientific state, compile intent into bounded work, capture evidence automatically, compare alternatives, qualify novelty, enforce rights and policy at action time, preserve versions, reproduce decisions, and expose where human authority begins. This environment should be open to different models, formal systems, databases, laboratory platforms and organisational policies. No single representation will be adequate for every science, and no single agent should be the centre of the architecture.

The programme is intentionally falsifiable. A structured Review Case may fail to improve flaw detection relative to a simpler tool-using model. A novelty engine may turn out to be too dependent on incomplete literature coverage to guide expensive experiments. Machine-readable compliance may become so costly to maintain that human review remains preferable for most research actions. A symbolic circuit constructor may fail to scale beyond narrow domains. Version histories may be too noisy to provide useful training data. These are not objections to be rhetorically defeated; they are experiments to be run. A credible research map names the questions that can be answered and the evidence that would force the architecture to change.

The result is a programme for executable science in a practical rather than metaphysical sense. We do not propose that every scientific idea should become code, that empirical reality can be replaced by formal proofs, or that institutions should delegate judgement to software. We propose that the parts of research that can be made explicit should become executable companions to the human process. Claims should know where their evidence came from. Procedures should know which specification and version they implement. Agents should know their authority. Policies should know which action they constrain. Laboratories should know which experiment state they are modifying. When a machine cannot establish something, the boundary should be represented rather than hidden behind fluent prose.

The chapters that follow develop this thesis from the working environment outward. The research environment is first treated as persistent scientific state. Specification-driven research then turns intent into controlled execution. Versioning and provenance provide historical identity. Agent runtimes provide

bounded action. Peer review, novelty and compliance become specialised scientific services operating over the same state. Neuro-symbolic mechanisms provide one plausible route toward executable reasoning. Automated laboratories extend the control plane into the physical world. The final chapter returns to the research map and identifies the technology categories that still have to be invented before the vision can be treated as infrastructure rather than aspiration.

The practical consequence is that research infrastructure must become selective rather than merely productive. A system that makes generation ten times cheaper but doubles the effort required to decide what matters has not solved the institutional problem; it may have amplified it. The decisive service is therefore not another generator but a control surface for scarce attention. Researchers need to see which hypotheses are active, which claims lack decisive evidence, which branches are redundant, which results would change a decision, and where uncertainty is concentrated. Institutions need the same information aggregated across projects without losing the local context that gives it meaning. Research-native firms need it as an operating signal for capital, compute and laboratory time. The research environment is the place where these different scales can share a common scientific state without sharing one simplistic score.

This view also changes the role of the AI model. In ordinary conversational software the model is often treated as the centre of the application and everything else as tooling around it. In a research environment the model is one class of computational actor inside a longer-lived scientific process. Models may be replaced, routed, specialised or run locally; the project should survive. A model may propose a hypothesis today and a different model may challenge it next month. The stable object is the evidence-bearing project history. This inversion is strategically important because model capabilities, prices and vendors are moving quickly, while scientific institutions need continuity measured in years.

One foundational requirement is a scientific-state substrate. It must represent more than files and tasks. Questions, claims, hypotheses, methods, experiments, datasets, decisions, review issues, permissions and unresolved conflicts need identities that survive edits and model changes. The representation should be sparse and extensible rather than one universal ontology. Different disciplines will add their own objects, but a small shared core can preserve dependency, version, source, authority and temporal status. Without this layer every advanced service is forced to reconstruct the project from prose before it can act, and every reconstruction becomes another source of hidden error.

A second recurring requirement concerns executable specification and planning. Current specification-driven software development shows why this matters, but research imposes a harder contract. The specification can state what is being investigated, which assumptions are active, what evidence would count, which transformations are allowed, which data may be used, what must be reproducible, what should trigger escalation and which outcomes must never be silently promoted. An agent can then derive a plan while the specification remains the higher-order object against which execution is checked. The plan may change after new evidence; the reasons for that change become part of the project state.

Scientific evidence infrastructure forms another necessary layer. Today provenance, notebooks, workflow logs, datasets and citations are usually connected only loosely. A research environment needs claim-level lineage that can cross those layers. A numerical claim may depend on a dataset version, an exclusion rule, a software environment and a statistical procedure. A literature claim may depend on a search frontier and a source span. A laboratory claim may depend on sample identity, calibration and protocol state. The research challenge is not merely storing all of this information. It is making the dependency precise enough that a later correction can invalidate the affected claims without forcing the entire project to be rebuilt.

Machine-produced work also needs a qualification layer. As AI generates more code, analyses and interpretations, the system needs a lifecycle between candidate and accepted artefact. A newly generated review rule, data transformation, solver mapping or analysis circuit should remain a candidate until it is tested in the scope for which it is intended. Promotion should create a versioned capability with evidence about its behaviour and known limits. This is one of the deepest continuities between the agent capability infrastructure, the experimental meta-rational execution approach, a

human-inspectable executable representation experiments and the new research environment components: generative intelligence can propose; a governed environment decides what becomes reusable institutional competence.

Research memory must also be designed for correction rather than mere retrieval. A normal vector store answers a retrieval problem. Research memory must answer a historical epistemic problem: what did the project believe at time t , why, based on which sources and tests, and what later caused that state to change? Negative results and abandoned hypotheses are part of this memory because they prevent rediscovery of failed paths. Forks are part of it because legitimate scientific disagreement should not be overwritten by a single summary. Compression will be essential, but compression must preserve the events that matter for reconstruction, accountability and future learning.

Runtime governance becomes unavoidable once agents can act. In a fast agentic environment, privacy, copyright, security, ethics and local organisational rules cannot live only in documents read before the project begins. They must become constraints on concrete proposed actions. The infrastructure has to know enough context to distinguish reading a public paper from sending confidential partner data to an external model, or running a local analysis from redistributing a licensed corpus. Machine-readable rules can cover some cases; language models can help interpret facts and documents; human authorities remain necessary for ambiguity. The research problem lies in connecting these layers without converting policy into either brittle code or unreviewable model judgement.

Evaluation must become continuous rather than episodic. Peer review is one important instance, but the deeper requirement is that any consequential scientific state change can trigger appropriate checks. Some checks are mechanical, some statistical, some retrieval-based, some solver-backed and some interpretive. The environment should know which checks actually ran and which did not. It should preserve disagreement rather than averaging it away. The system can therefore turn evaluation from an episodic ceremony around publication into a continuous property of the research object, while still leaving final authority with researchers and institutions.

Scientific direction itself becomes a computationally assisted problem. Once generation becomes abundant, choosing the next question becomes more valuable. NOVA-like services are one attempt to formalise part of this problem by comparing a result with a temporal scientific memory, decomposing its contribution, identifying antecedents and contradictions, estimating evidence gaps and proposing discriminating next tests. The objective is not to rank all science with a universal score. It is to give the researcher a better map of where the project sits relative to what is known and which uncertainty is worth paying to resolve.

The environment also needs ways to combine neural and non-neural methods without erasing their different guarantees. the experimental meta-rational execution approach and a human-inspectable executable representation dynamic circuits are current Axiologic experiments in this direction, but the architectural requirement is broader than either implementation. A research system needs to be able to say that one step was a neural interpretation, another was a database lookup, another a statistical computation, another an SMT check and another a human decision. Their outputs may compose, but their authority should not be conflated. A future executable-science layer must make those boundaries operational rather than decorative.

Finally, the same principles must extend to physical research control. Automated laboratories make all previous abstractions concrete because an erroneous action can consume material, damage equipment or create unsafe conditions. Scientific intent, laboratory scheduling, safety interlocks, compliance rules, instrument control and evidence capture should therefore be separate control planes joined through typed contracts. This is the point at which the research environment becomes more than a software workspace. It becomes a possible supervisory environment for research organisations in which digital and physical experiments share the same project memory and governance.

These categories form the map for the rest of the book. Some are primarily engineering problems that can be built now. Some depend on research whose outcome is uncertain. Several will probably mature first in narrow domains rather than as universal systems. The important strategic choice is to build them around shared contracts so negative results remain useful. If dynamic symbolic reasoning

proves uneconomic, the project-state and provenance layers still matter. If novelty intelligence remains too uncertain for automated steering, it can still support human review. If compliance compilation works only for local policies, runtime action gating is still valuable. The programme is ambitious precisely because it is modular enough to survive partial success.

The architecture should also be judged by what it refuses to centralise. A research environment does not need to become the owner of every dataset, repository, instrument or model. It needs durable identities and contracts that let those systems participate in a common project state. This matters strategically because research infrastructures are long-lived and heterogeneous. A laboratory information system may survive several generations of AI models; a Git repository may outlive a cloud provider; a formal solver may remain authoritative for a narrow task while the surrounding natural-language interface changes completely. The research environment should therefore be a coordination and evidence layer whose components can be replaced without erasing scientific history.

The resulting vision is closer to an institutional nervous system than to an artificial scientist. It senses project events, routes work, preserves memory, applies constraints, notices conflicts and helps allocate attention. It does not possess one unified theory of scientific truth. Different evaluators can disagree; different disciplines can define different evidence; different institutions can retain different policy. The common infrastructure is valuable precisely because it can make those differences explicit and still support action. This pluralism is not a concession. It is a design requirement for software intended to serve real science rather than benchmark-sized worlds.

The transition can also be incremental. A laboratory or research team does not need to formalise its entire practice before gaining value. It can begin with versioned projects, reproducible agent runs and provenance. It can add specification-driven workflows around repetitive tasks, then specialised review checks, then compliance gates around sensitive actions. Novelty intelligence can remain advisory until prospective evidence justifies stronger use. Neuro-symbolic circuits can be introduced only where opaque model calls create recurring cost or risk. This order protects the programme from making adoption depend on the least mature component.

For the same reason, the book uses the phrase research programme more often than platform roadmap. The problem is not simply to implement a fixed feature list. We do not yet know the best representation of scientific state, how much semantic structure users will tolerate, how often explicit symbolic procedures will transfer, how to calibrate novelty judgements against incomplete corpora, or which compliance questions can be compiled safely. Those uncertainties are the work. The current infrastructure gives us a place to run the experiments under realistic conditions rather than answering them only in isolated prototypes.

A successful outcome would therefore be visible in research behaviour. Researchers should spend less time reconstructing context, repeating already failed work, checking routine mechanical issues and discovering late that an action violated policy. They should spend more of their limited attention on choosing questions, challenging assumptions, interpreting difficult evidence and deciding when a project deserves commitment. Institutions should gain stronger continuity without forcing every scientist into a rigid workflow. AI should increase the number of scientifically meaningful options available to people while the system makes the cost, evidence and authority of those options easier to inspect.

The four most immediate research services can now be stated as one operating loop. Evidence-centred review asks whether claims, methods, citations, computations and artefacts deserve the status they currently have. Novelty and value assessment asks whether a result changes the known scientific state enough to justify attention or another experiment. Runtime compliance asks whether the next research action is permitted under the relevant data, rights, ethics, organisational and regulatory constraints. The research environment binds these services to the same persistent project state so that review, steering and governance do not reconstruct different versions of reality from separate prompts.

These services become more useful when they are coupled but not conflated. A novel result can be methodologically weak. A correct result can be unimportant. A compliant action can still be scientifically foolish. A strong peer-review finding can invalidate the novelty case without changing the historical fact that the experiment occurred. The system therefore needs separate evidence objects and decision

rights, joined through provenance rather than collapsed into one confidence score. This is a recurring architectural principle throughout the book: integration should make distinctions easier to preserve, not easier to hide.

The programme consequently combines two horizons. The nearer horizon builds the operating substrate: specification-driven projects, version control, research memory, bounded agent execution, provenance, rights-aware data exchange and reusable capabilities. The farther horizon investigates new evaluative and neuro-symbolic technologies that can operate on that substrate: executable review protocols, scientific novelty intelligence, policy compilation, dynamic symbolic circuits and closed-loop control of automated laboratories. The mature layer makes the speculative layer testable under realistic conditions; the speculative layer gives the mature layer a research direction. Together they define the research environment as an instrument for governing the increasing speed of machine-assisted science rather than merely increasing that speed again.

CHAPTER 2

The Research Project as a Living System

A research environment should begin with the project, not with the model. That choice sounds minor, but it reverses the architecture of most contemporary AI applications. A model-centric system asks what context must be assembled for the next inference. A project-centric system asks which scientific objects already exist, how their status changed, what remains unresolved and which capabilities are permitted to act on them. Models become replaceable workers inside a longer-lived environment. The project becomes the continuity mechanism.

The first generation of the ACHILLES IDE, in Axiologic's implementation line, provides a concrete engineering baseline for this shift because it is built on AssistOS rather than on a single chat interface. The environment already treats collaboration, visualisation, meetings, files and agent access as parts of one workspace. The research question is what must be added for that workspace to understand the structure of science itself. The answer cannot be one universal ontology. It is more plausibly a small set of stable scientific contracts combined with domain-specific packages.

A general-purpose copilot is an excellent interface for asking questions, editing text, writing code and invoking tools. It is not, by itself, a model of a research project. Most assistants are organised around a session or a user request. Context is assembled, a task is completed and the result is returned. Long-running memory can improve continuity, but scientific continuity has stronger requirements than conversational memory. A scientist needs to know which hypothesis was tested, which version of a dataset was used, which experimental branch failed, which evidence contradicted a claim, which decision superseded another, and which result is invalid because a dependency changed. These are relations among scientific objects, not simply facts about what the user previously said.

Research software is also mostly artefact-centred. A document editor manages documents. A notebook manages cells and outputs. A code repository manages files and revisions. A data platform manages datasets. A workflow engine manages jobs. A reference manager manages papers. A meeting system manages communication. Each tool is useful, but the scientific meaning that connects them often exists only in human interpretation. An AI agent crossing tool boundaries therefore faces the same integration burden repeatedly. It must infer which artefact is authoritative, whether two names refer to the same experiment, whether a figure is current, and whether a failed run should affect the project conclusion. This inference is expensive and error-prone when every task begins from raw files and conversation history.

The first missing primitive is persistent scientific state. The environment needs explicit objects for questions, hypotheses, claims, evidence, experiments, methods, artefacts, decisions, constraints and evaluations. The objects do not need to impose a rigid scientific ontology on every domain. A minimal common core can be extended by domain profiles. What matters is that the state has identity, versioning and links. If an agent says that “claim C is weakened by experiment E”, the environment should be able to represent that relation directly. If a later agent replaces E with a corrected run, dependent objects can be re-evaluated. Without this layer, the system may produce polished explanations while the underlying project remains semantically fragmented.

The second missing primitive is controlled action. A research agent should not have an undifferentiated ability to “use tools”. It needs a capability scope. A literature agent may search public corpora but not read a confidential dataset. A reproduction agent may execute code in a sandbox but not publish results. A meeting agent may draft decisions but not commit them as accepted project knowledge. A privacy agent may block a data transfer but should not rewrite a scientific claim. These boundaries are easier to enforce when the environment exposes typed actions and machine-readable authority rather than only natural-language instructions.

The third missing primitive is a distinction between proposal and acceptance. Scientific work contains many provisional statements. Brainstorming should be cheap. The environment can allow agents and humans to create candidate hypotheses, interpretations and plans with low friction. Those candidates should not silently become accepted knowledge. A state change can be treated as a

research transaction: it starts from a known state version, proposes changes, records the actor and inputs, invokes any required checks, and commits only when the configured authority is satisfied. A transaction can still record a failed attempt. This gives the project replay, rollback, conflict detection and audit without forcing researchers to interact with database semantics.

Concurrency is the fourth missing capability. Agentic research is naturally parallel. One agent may search literature while another reproduces a result, a third designs an experiment and a fourth analyses a meeting transcript. They may all touch the same hypothesis. A conventional workflow engine often assumes a predefined graph, while a shared document model often assumes human conflict resolution after the fact. Research requires something between them: branchable state, explicit dependencies, optimistic concurrency, semantic conflicts and a way to compare competing research branches. The exact mechanism is a research hypothesis, but ignoring concurrency would make the architecture unrealistic.

The fifth gap is evaluation independence. A generic assistant can critique its own output, but self-critique is not equivalent to independent verification. Scientific projects benefit from different evaluators using different methods and evidence. A statistical check, a formal solver, a reproduction run and an LLM reviewer have different failure modes. The IDE should make evaluation composable and visible. A generated conclusion can carry multiple evaluations rather than one model confidence. The platform should also permit evaluators to disagree without forcing premature consensus. Disagreement is often a useful research state.

The sixth gap is temporal integrity. Research claims are made relative to a state of knowledge and a version of the project. Novelty assessment in 2026 is different from novelty assessment against a 2024 literature snapshot. A reproduction run may depend on an environment that later disappears. A model trained on future literature can contaminate a historical benchmark. A research environment therefore needs snapshots, versioned corpora where feasible, explicit retrieval times, environment capture and memory scopes. These are not advanced features for a future autonomous scientist; they are basic requirements for reproducible evaluation of current research agents.

The seventh gap is project-scale explainability. “Why did the model answer this?” is not the only relevant question. Researchers need to ask “Why do we still believe this?”, “What changed since last week?”, “Which conclusions depend on this dataset?”, “Why was this experiment prioritised?”, “Which agent or human rejected this branch?”, and “What would need to change for us to revisit it?”. These questions require explanations over state and provenance. They cannot be generated reliably from the latest chat alone. A project-level explanation should point to the objects, evidence and decisions that support it.

Project Consciousness: a memory that watches without becoming the scientist

A future research infrastructure may need something more continuous than a project database and more restrained than an autonomous project manager. We can call this virtual role the Project Consciousness. The term is deliberately metaphorical. It does not claim machine consciousness. It names a persistent AI function whose job is to observe the life of a research project, maintain a coherent account of what has happened, and help the group recover its own history. Most agents are active: they search, calculate, write, test, or propose. Project Consciousness is primarily passive. It listens to authorised events, notices state changes, and builds an evolving rationalisation of the project without pretending that the rationalisation is the project itself.

This role addresses a weakness that appears in almost every long research programme. The formal record contains the accepted results, while the real history also contains hesitation, discarded ideas, failed replications, temporary assumptions, arguments in meetings, unresolved doubts, and insights that were judged premature. Human teams remember some of this for a while. Then people leave, attention shifts, and the reasons behind earlier decisions decay. Conventional document search recovers text but often not the intellectual situation in which the text mattered. A project-aware AI could preserve that situation as a versioned interpretation linked to the underlying events.

The Project Consciousness should not merely summarize activity. A summary tends to compress away disagreement and dead ends. Its more valuable function is temporal reasoning over scientific intention. It can know that a hypothesis was abandoned because the available dataset could not distinguish two mechanisms, not because the hypothesis was refuted. It can know that a collaborator objected to a statistical assumption that was later reintroduced indirectly. It can remember that an experiment was postponed for budget reasons, that an idea was considered too speculative for a deliverable, or that a promising line was set aside when a key researcher became unavailable. Months later, when new data or new tools change those constraints, the system can surface the abandoned line as newly relevant.

Such a component would create a new kind of institutional memory. It would connect events that today belong to different channels: repository commits, notebook runs, meeting decisions, review comments, model outputs, literature changes, instrument logs, and human approvals. The goal is not to collapse them into one database schema. The goal is to maintain a navigable history of scientific meaning across them. A researcher should be able to ask not only 'what changed?' but 'why did we change our mind?', 'what alternatives were live at that moment?', 'what evidence would have changed the decision?', and 'which old questions have become answerable now?'.

The role is also important because future projects may contain dozens or hundreds of specialised agents. No individual human will inspect every intermediate action. An observing layer can detect that three agents repeatedly rediscovered the same failed path, that one line of evidence is being used by several conclusions, or that a new result silently invalidates assumptions elsewhere. It can provide context to active agents without giving them unrestricted access to the entire project. In this sense it becomes a bridge between memory and orchestration: it does not decide what the project should believe, but it helps other participants understand what the project has already lived through.

The Project Consciousness must itself be governed. A retrospective narrative generated by an AI can be wrong, overconfident, or seductively coherent. Its statements should therefore be linked back to events and artefacts, and competing interpretations of the history should be allowed. When it infers a reason that was never explicitly stated, that inference should remain marked as such. Some projects may permit the system to observe almost everything; others will require strict compartmentalisation because meeting conversations, personal notes, patient data, trade secrets, or partner information cannot be merged into one omniscient context. The architecture must support partial memory and explicit blind spots.

This virtual role also changes how we think about explainability. Explainability is usually discussed at the level of one model decision. Research needs explanation across time. Why did the project spend three months on a method? Why did a claim become accepted? Why was a contradictory paper discounted? Why did an automated laboratory stop exploring one region of parameter space? A coherent historical layer can make such questions answerable without requiring every scientist to read years of logs. The explanation is not a post-hoc story detached from evidence; it is a reconstruction built from versioned project events and therefore open to challenge.

If this idea proves useful, Project Consciousness may become one of the defining virtual components of future research infrastructure. It may materialise as a background agent, a versioned project narrative, a graph of decisions, a time-aware retrieval system, or a combination of these. Its interface may be almost invisible most of the time. What matters is the function: preserving continuity when the number of participants, agents, experiments and artefacts exceeds what unaided human memory can integrate.

The eighth gap is research-specific resource governance. Agentic systems can waste compute by repeatedly exploring low-value variations, retrying broken environments or performing expensive searches whose expected information gain is low. Existing schedulers optimise throughput or job completion, not scientific value. A research environment does not need a perfect "resource economist", but its orchestrator should expose budgets, stopping criteria, branch costs and results. Research hypothesis: resource allocation can be improved by combining scientific evaluation with operational metrics. The hypothesis must be compared with simpler quotas and human allocation; it should not be assumed.

Model replacement is another reason to separate scientific state from assistant memory. The useful lifetime of a research project can exceed the useful lifetime of a foundation model or an agent scaffold. A project started with one model may later migrate to another because of cost, availability, privacy or capability. If rationale and decisions exist only in model-specific memory, migration becomes a semantic reset. If they exist as explicit project objects with source links and provenance, a new agent can reconstruct context and its own understanding can be evaluated against the same state. This makes model competition compatible with scientific continuity.

Interoperability also matters at the level of institutions. A consortium should not need every partner to run the same vector database, workflow engine or LLM provider. A partner may expose a dataset through a controlled query service while another contributes a simulator and another contributes an evaluator. The research environment needs contracts that allow these capabilities to participate in one project without centralising everything. This suggests a service boundary around typed research objects and actions rather than around vendor-specific agent messages.

A final gap is the absence of a clear “scientific source of truth”. In software development, a repository and test suite provide imperfect but recognisable anchors. Research often has many competing sources of authority: the latest manuscript, a notebook, an internal spreadsheet, a meeting decision or the principal investigator’s memory. A research environment cannot solve scientific disagreement by decree, but it can make authority explicit. It can record that a dataset version is current, that a claim is provisional, that an experiment supersedes an earlier run, or that a decision was accepted by a named project role. This reduces accidental ambiguity without enforcing artificial consensus.

These gaps explain why the research environment should be an integration environment rather than another monolithic AI application. A new model can be plugged into a copilot. A new domain tool can be exposed through a tool adapter. A new evaluator can subscribe to state changes. A new privacy policy can constrain data movement. A new scientific domain can add object types and validators. The stable part is the contract around project state, execution, evidence and authority. This is the “puzzle” property: components can be replaced because their responsibilities are explicit, not because every feature has been turned into a microservice.

Interfaces should remain familiar while the substrate changes

The research environment should be familiar enough that a scientist can begin with existing habits. A project opens into documents, datasets, notebooks, visualisations, code, experiment runs, references, conversations and meetings. The general copilot is available everywhere but is not the only organising principle. The project itself is primary. A user can select a claim and ask for supporting evidence, select an experiment and ask for a reproduction, select a dataset and ask which conclusions depend on it, or select a meeting decision and inspect the work it triggered. This interaction pattern matters because it grounds language in project objects. The copilot becomes a conversational interface to the research environment rather than a separate conversational universe.

Documents should remain first-class because research is still communicated through proposals, protocols, reports and manuscripts. The difference is that the IDE can treat a document as both prose and a view over project state. An agent may extract candidate claims, citations, methods and unresolved questions. A reviewer may mark an assertion whose cited paper does not actually support it. A figure can link to the experiment run that generated it. A manuscript paragraph can become stale when the underlying claim changes. None of this requires replacing Word-like editing with a formal language. The formal structure lives underneath and is exposed only where it helps.

Notebooks and code environments become executable research surfaces rather than isolated technical tools. An agent can propose a new analysis, show the exact code and environment changes, execute them in a sandbox, compare outputs against a baseline and attach the result to an experiment object. A human can accept the result, request a reproduction with a different toolchain, or discard the branch. For domains that rely on simulators, the same interaction can apply to parameter studies and

models. For physical laboratories, instrument integration should be treated as a later and higher-risk extension because actions may be irreversible and safety constraints are stronger.

Chat remains important, but it should not be the project database. A conversation is valuable for exploration, explanation and negotiation. The system can extract proposals from it, but the transition into project state should be visible. When a researcher says “we should probably test the low-temperature regime”, the copilot can create a candidate experiment plan and ask whether it should be added. When a team later agrees that a hypothesis is no longer credible, the meeting agent can propose a status change linked to the discussion. This separation preserves the speed of natural interaction without treating every utterance as an authoritative scientific statement.

Real-time collaborative interfaces deserve a larger role in this vision. In the current AssistOS line, WebMeet is integrated with the copilot and explores a meeting in which a group of researchers can speak with one another while AI participants join the same room. These AI participants can be organised as a RiboTeam: several specialised agents working in parallel on the project rather than one universal assistant. During the discussion they may search evidence, update a document, create a diagram, compare architectures, prepare an analysis, or challenge a claim. A human facilitator may direct the work, but an AI facilitator could also coordinate the session under explicit rules. The important point is that the meeting becomes an interface into the research project, not a separate communication channel whose useful content must later be reconstructed.

From videoconference to shared scientific control surface

This pattern may become central to future research work because parallel agents create an interface problem. If ten agents can work at once, a scientist cannot control them through ten chat windows. The human needs a shared surface on which work becomes visible as it happens. A literature agent can place a source beside a claim. A statistical agent can expose a sensitivity analysis. A modelling agent can update a diagram. Another can propose a counterexample. Participants can interrupt, reject, redirect, or approve. The environment must show not only outputs but also roles, provenance, current authority, and what each agent is waiting for.

The shared object is therefore more important than the transcript. In ordinary videoconferencing, screen sharing makes one participant's application visible to the others. In a human-agent research room, the shared screen can become a manipulable project surface. Documents, figures, models, datasets, hypotheses, and experiment plans are common objects. An agent's contribution arrives as a visible proposal attached to an object. Human acceptance becomes part of the history. The distance between conversation and execution begins to disappear.

New forms of visualisation will be needed. Researchers may want to see which hypotheses are active, what evidence supports them, which agent is testing which branch, what remains uncertain, and where the current bottlenecks lie. The appropriate visual language will vary by discipline. A molecular biology team may need experiment and evidence views; a theoretical group may need argument and dependency structures; an industrial research group may need requirements, simulations, risk and manufacturing constraints. The infrastructure should therefore support composable views rather than impose one universal dashboard.

A meeting-centred interface also makes social authority visible. Science is not only computation. A principal investigator, statistician, laboratory technician, external reviewer, legal officer, and AI agent do not have interchangeable authority. Future interfaces should represent these differences without turning collaboration into bureaucracy. When a result is accepted, the environment can record who or what performed the check and who had authority to approve the transition. When an agent merely suggested a conclusion, the interface should not make that suggestion look like an institutional decision.

The larger hypothesis is that much research will move from sequential tool use toward sessions of coordinated attention. Humans will formulate goals, interpret surprising outcomes, negotiate assumptions, and decide what deserves commitment. Agents will work in the background and in parallel, bringing calculations and artefacts into the shared space. The most valuable interface may

therefore resemble neither a conventional IDE nor a conventional video call. It may be a collaborative scientific room in which conversation, execution, visualisation, memory and governance coexist.

The interface should provide epistemic views that conventional project management tools do not. A claim view can show direct evidence, counterevidence, reproductions, confidence and downstream dependencies. A contradiction view can show where sources or experiments disagree. A branch view can compare competing hypotheses and their resource cost. A provenance view can reconstruct how a figure or conclusion was produced. A novelty view can show the strongest known prior art and the aspects believed to differ. An attention view can aggregate events that need human judgement. These views are not decorative dashboards; they are projections of Scientific Project State for specific research questions.

The environment should also be able to explain difficult parts of the project on demand. A Documentation or Tutorial Agent can generate a primer on a method used by another team, a walkthrough of a simulation, a glossary for a new collaborator, or a visual explanation of why an evaluator rejected a result. Such artefacts should carry a freshness marker and a state reference. If the project changes, the explanation can be regenerated. This allows the system to produce abundant educational material without confusing generated prose with the authoritative evidence record.

A useful interface needs explicit modes of commitment but should keep them lightweight. Brainstorming is one mode: anything can be proposed. Drafting is another: the system produces an artefact but does not change accepted state. Execution authorises reversible sandbox work. Acceptance changes the project's scientific state. External action publishes, sends, purchases, submits or controls equipment and therefore deserves a higher authority threshold. The user should not need to think in access-control terminology; the interface can communicate these modes through familiar affordances. The underlying system, however, must know the difference.

A research environment can support proactive background work without becoming noisy. Literature scouts can monitor new papers against active hypotheses. Reproduction agents can rerun critical analyses when dependencies change. Privacy agents can check whether a planned data flow violates project rules. Evaluators can re-open a claim when contradictory evidence arrives. The user should see these actions as meaningful project events, not as a scrolling log of agent messages. Progressive disclosure is useful: a compact summary first, then evidence and trace, then low-level tool details if needed.

Multi-institution research adds another dimension. A project may contain shared claims backed by evidence that is accessible only to one partner. The IDE should support federated scientific objects: the shared project can know that an authorised evaluation exists without copying the protected dataset into a central environment. Depending on the use case, a computation can be sent to the data, an aggregate result can be returned, or a remote agent can expose only a bounded capability. The design space includes conventional access control, local inference, trusted execution, federated analysis and privacy-enhancing technologies. Which mechanism is appropriate is a deployment decision, not a universal architectural requirement.

Long-term speculation: ambient research assistance should be kept separate from near-term requirements. It is plausible that future researchers will use ambient assistants that continuously observe permitted project activity, prepare briefings and launch low-risk analyses automatically. It is also plausible that scientific projects will contain persistent agent teams that develop specialised procedural memory. Neither is necessary to justify the research environment now. The near-term system only needs enough persistence, modularity and governance that increasing autonomy can be added

What a day of research could feel like

A typical day in such an environment can remain simple. The researcher opens the project and receives a briefing generated from state changes rather than from message volume. It reports that two new papers affect an active hypothesis, an overnight reproduction succeeded, another failed because of a dependency mismatch, one agent-generated branch was stopped after exceeding its budget, and one decision requires human input because novelty evaluators disagree. The researcher inspects only

the disputed novelty comparison, requests a stronger prior-art search, edits the experiment plan and joins a WebMeet with collaborators. The system records the approved plan and schedules the run. The visible workflow is familiar; the difference is that background agency is connected to explicit scientific objects.

The interface should also make it easy to ask longitudinal questions. “What changed since the last project review?” can be answered by comparing state versions. “Which assumptions are currently untested?” can be answered by traversing dependencies. “Which conclusions rely on a single paper?” can be answered from evidence links. “What did we stop working on and why?” can be answered from rejected branches and decisions. These questions are difficult to answer reliably from a folder of files and chat transcripts, even for human team members. They are precisely the kind of questions a state-centred IDE should make routine.

The research environment should not force every researcher to inspect graphs. Most users will continue to prefer prose, tables and familiar visualisations. Scientific structure can remain latent until needed. A manuscript can render a claim graph only when a reviewer asks for dependency analysis. A project dashboard can show a compact confidence indicator while allowing an expert to drill into source-level evidence. The UI principle is progressive formalisation: natural work first, explicit structure where it improves action, reproducibility or understanding.

The product opportunity is therefore concrete. The visible interface can remain a web-based collaborative IDE with a strong copilot, while the research contribution sits underneath: project state, agent contracts, evaluators, provenance, trustworthy execution and scientific views. This creates a path from today’s assistant-centric interfaces to a research environment in which much routine work

It is useful to think of the future environment in terms of virtual components rather than fixed software modules. A virtual component is a role that the infrastructure must reliably perform even though different implementations may realise it in different ways. One laboratory may combine several roles in one service; another may split them across local software, cloud services, institutional systems, and specialised agents. The book therefore describes responsibilities before products. What matters is that the research process can invoke these responsibilities with known boundaries and can preserve the evidence of what they did.

Virtual components: roles that may take many technical forms

The first family of virtual components maintains the research workspace and the evolving state of the project. It gives persistent identities to questions, hypotheses, claims, datasets, experiments, decisions and artefacts, and it lets different user interfaces project that state in useful ways. This family includes the Project Consciousness described earlier, but the two should not be confused: project state records what exists; Project Consciousness builds a historically informed interpretation of how the project arrived there.

Another family performs scientific work through specialised agents. The important design choice is specialisation by responsibility rather than anthropomorphic role-play. An agent that verifies a citation, one that plans an experiment, one that checks a statistical assumption, and one that prepares a visual explanation need different permissions, context and evaluation. They may use the same underlying model or completely different technologies. The infrastructure should care about the contract of the role more than the brand of the model.

A third family evaluates and challenges. Peer review is one visible example, but the deeper need is for independent checks throughout the life of the project. Some checks are mechanical, some statistical, some formal, some literature-based, and some necessarily human. The infrastructure should be able to request the appropriate kind of challenge at consequential transitions without pretending that one universal scoring model can decide scientific quality.

A fourth family provides governance and contextual constraints. Privacy, copyright, institutional policy, ethics, data-use agreements, security boundaries, and regulation all influence what research agents may do. These constraints will rarely fit one fixed compliance engine. They need configurable

representations, source links, and enforcement points that can travel with the project and with the actions performed on its behalf.

A fifth family concerns evidence, provenance, and durable memory. It binds claims to sources and transformations, records versions, and allows corrections to propagate. The implementation may use repositories, provenance models, content-addressed objects, secure data-sharing mechanisms, or future systems that do not yet exist. The conceptual role is stable even when the technology changes: scientific work must remain reconstructable after people, models and tools have been replaced.

The final family connects digital reasoning to physical action. In an automated laboratory, the same project state and governance mechanisms must eventually control instruments, schedules, samples, simulations, and experiment execution. Here the distinction between proposal and authority becomes materially important. A software agent that suggests an experiment and a controller that moves a robotic arm should not inhabit the same permission boundary.

The first responsibility is the Research Workspace. It owns human-facing interaction and projections of project state. Its job is not to contain scientific truth; it presents documents, chat, meetings, data, code, experiments, evidence graphs, agent activity and decisions in forms researchers can use. It should be plugin-friendly because different domains require different visualisations and tools. A structural-biology project may need molecular viewers; a verification project may need proof traces; a social-science project may need survey and statistical views. The workspace should ask the platform for typed project objects rather than embed domain logic in a single frontend.

The second responsibility is Scientific Project State and Provenance. This is the architectural backbone. A minimal implementation needs stable object identifiers, versions, actor identity, timestamps, provenance links, access metadata and relations among objects. The state should support provisional and accepted objects, competing branches and supersession. It must also preserve raw artefacts, because structured summaries cannot replace source data, code or documents. The representation does not need to be a single graph database. A practical system can combine an event store for state changes, a relational or graph index for relations, an object store for large artefacts and specialised search indices. The research question is how much structure provides real benefit before authoring overhead becomes counterproductive.

A useful Scientific Project State contains only objects that help action or evaluation. Questions represent what the project is trying to resolve. Hypotheses are testable candidate explanations or predictions. Claims are assertions that may be supported, contradicted or scoped. Evidence points to observations, sources, outputs or validated analyses. Methods describe procedures. Experiments bind methods to inputs, environments and expected observations. Decisions record authoritative changes in direction. Constraints represent budgets, policies or domain restrictions. Evaluations attach assessments without overwriting the object being assessed. Tasks and agent runs describe operational

The third responsibility is Research Orchestration. Research is not a fixed pipeline. The orchestrator converts an intention into a plan, selects capabilities, manages branches, observes progress, enforces budgets and stopping conditions, requests evaluation, and escalates decisions. It should not be a scientific oracle. Its job is coordination. A simple task may use a fixed workflow; an exploratory task may use iterative planning or tree search. The architecture should permit several orchestration strategies under one contract so that their value can be compared experimentally. A complicated agent hierarchy should not be retained if a small library of explicit workflows performs as well.

The fourth responsibility is the Agent and Tool Runtime. This is where intentions become actions. It manages model calls, tool adapters, execution sandboxes, compute, data mounts, credentials, simulators and eventually laboratory interfaces. Every run should have an execution identity, inputs, environment metadata, resource consumption, outputs and logs. Tool calls should be typed and policy-checkable. The runtime also provides failure isolation. An agent that executes untrusted repository code should not inherit the same network and data permissions as the user's browser session. Reproducibility is easier when execution capture is a runtime feature rather than an optional habit of each agent.

The fifth responsibility is Evidence and Scientific Evaluation. It receives objects or state transitions that require checking and routes them to appropriate evaluators. A citation check, a statistical test, a formal solver, a reproduction run, a novelty assessment and a domain expert model are different evaluators but can share a common envelope: target object, evaluation method, inputs, result, uncertainty, provenance and evaluator identity. The evaluation layer should preserve multiple assessments and their disagreements. It should never silently convert “one evaluator passed” into “the claim is true”. Acceptance policy belongs to the project and may require a human decision.

The sixth responsibility is Knowledge and Memory. External knowledge includes literature, patents, standards, databases and domain corpora. Project memory includes accepted and rejected hypotheses, past experiments, decisions and operational experience. Procedural memory includes reusable methods, prompts, scripts, agent harnesses or skills learned during projects. These categories should not be collapsed. External knowledge has source and temporal boundaries. Project memory is part of scientific provenance. Procedural memory can influence future behaviour and therefore creates contamination and governance risks. A procedure should be promoted only after evaluation, versioned, scoped to domains where it has evidence, and removable when regressions appear.

The seventh responsibility is Trust and Governance. It cuts across all other blocks. Identity and capability determine what an actor may do. Data classification and policy determine which models, tools and locations may process an artefact. Secrets management constrains credentials. Audit records consequential actions. Security controls isolate untrusted content and code. Privacy controls manage personal, confidential and partner-restricted information. Human authority rules determine which state changes or external actions require confirmation. Incident handling traces affected scientific objects when a tool, dataset or agent is later found to be compromised. Trust is therefore implemented as concrete controls around action, not as a textual principle.

The common interaction among these responsibilities is the research transaction. Suppose the user asks the system to test whether a new regularisation method improves robustness. The orchestrator creates a candidate plan. The runtime checks out the relevant code, builds an environment and executes experiments. The evaluation layer validates outputs and may request independent repetitions. The knowledge layer retrieves related methods. The novelty evaluator compares the observed contribution with prior work. Trust controls restrict datasets and external access. The workspace shows the proposed conclusion. Only then is the Scientific Project State updated with the accepted evidence and claim status. Each stage can be implemented by different tools without changing the semantic outline.

A transaction also gives the system a unit of failure. If the code does not run, the transaction can record an operational failure without changing the scientific claim. If the experiment runs but the statistical evaluator finds no support, the negative result becomes evidence. If prior art shows that the method is known, the novelty status changes while correctness may remain unaffected. If the dataset was used outside its permitted scope, the system can invalidate or quarantine dependent outputs. This separation is more useful than a binary “agent succeeded/failed” status because scientific and operational outcomes are different.

State transitions need clear semantics. A hypothesis can move from candidate to active, rejected, suspended or superseded. A claim can remain disputed rather than being forced into true or false. An experiment can be planned, running, completed, failed operationally, invalidated methodologically or superseded by a corrected run. Evidence can be supporting, contradicting, inconclusive or unavailable. These states should be extensible and domain-specific, but their transitions should be auditable. An agent may propose a transition; policy determines whether it can commit it.

Schema evolution is unavoidable. A multi-year programme will discover that its initial object model is incomplete. New domains will introduce new evidence types, relations and constraints. The platform should version schemas and allow compatibility mappings rather than rewriting every historical object. Old project packages should remain readable. Agents should detect when they encounter a type they do not understand and either use a generic representation or request an adapter. This is a mundane

infrastructure problem with long-term scientific importance: a research record is useful only if future tools can interpret it.

Caching and indexing are implementation concerns but influence the architecture. A large project may contain millions of evidence links or external facts, and the IDE should not load them all into an agent context. Project state needs query interfaces that return task-relevant slices, with provenance and access rules preserved. Semantic retrieval can help, but exact identifiers, graph relations and structured filters are equally important. The state layer therefore acts as a context compiler: it

Infrastructure should describe responsibilities before implementations

The architecture should expose explicit contracts. An agent capability manifest can state accepted input object types, produced output types, required tools, data sensitivity limits, expected cost class and evaluators required before commit. A tool adapter can describe its parameters, side effects and sandbox needs. An evaluator can declare what kinds of objects it can assess and what its result means. A workspace plugin can declare which project views it renders. These contracts make dynamic composition possible without requiring agents to infer the entire platform from prose documentation.

The architecture should also prefer deterministic mechanisms where they are adequate. Provenance capture should be automatic. Access control should be policy-enforced. Checksums and environment fingerprints should not be generated by a language model. Statistical tests should use tested libraries. A language model can interpret an ambiguous request, propose a plan or explain an outcome, but the final access decision or computation should use deterministic infrastructure wherever possible. This rule improves reliability and makes agent behaviour easier to evaluate.

Research hypothesis: explicit Scientific Project State will improve continuity, reproducibility and replaceability over simpler approaches such as shared folders plus retrieval-augmented chat. This is the most important hypothesis in the architecture and should be tested aggressively. If a rich state model creates large authoring and maintenance costs without improving outcomes, it should be simplified. The programme should search for the smallest semantic core that produces measurable benefit.

Table 3. A minimal scientific project state needs semantic objects beyond files and chat memory.

Term	Meaning in the research environment
Question	What is the project trying to discover or decide, and how has the question changed?
Claim	What proposition is currently asserted, with what status and dependencies?
Evidence	Which observations, computations, sources or formal results support or contradict the claim?
Experiment	Which protocol, inputs, environment, outputs and evaluation state define the experimental episode?
Decision	Which human or machine authority changed project state, and on what basis?
Constraint	Which scientific, legal, ethical, resource or safety boundary limits legitimate action?

The research environment category is therefore justified even before the more speculative evaluators are mature. Persistent scientific state, explicit provenance, controlled execution and attention management solve immediate problems created by current agents. They also create the substrate on which stronger ideas can be tested without embedding them permanently in the user interface. If a novelty evaluator fails, the project state remains useful. If one representation of claims proves too rigid, it can be replaced without replacing the workspace. The architecture should make research hypotheses replaceable in the same way that it makes models replaceable.

A useful test of the category is simple: can a researcher return to a project after six months, replace the model, reproduce an important decision, see which evidence has changed, and understand why the current state differs from the old one without reconstructing the story from chat transcripts? If the

answer is no, the system is still an AI application around research rather than an integrated environment for research.

A persistent project state also changes the meaning of coordination. In a chat-centred system, coordination is conversational: one agent tells another what it believes has happened. In a state-centred system, coordination is transactional. A capability reads a versioned set of scientific objects, proposes a transformation, and either commits a new state under policy or leaves the previous state untouched. This makes conflicts observable. Two agents can attempt incompatible updates to the same hypothesis or dataset interpretation without one silently overwriting the other. The system can preserve both branches, request reconciliation, or apply a domain-specific merge rule. Research collaboration becomes safer because disagreement is represented before it is narrated away.

This transactional view deserves its own research programme. Scientific objects are not all amenable to the same consistency model. A checksum can be updated mechanically. A hypothesis can branch legitimately. An experiment may become invalid when one calibration record changes. A literature claim may need re-evaluation after a source correction. A compliance decision may depend on the policy version in force at the time of execution. The research environment therefore needs semantic invalidation rather than generic cache invalidation: it must know which downstream objects require recomputation, which require human reconsideration and which remain historically valid even though they would no longer be accepted today.

The same project state can become an attention model. As AI increases the number of active branches, researchers need to know where human judgement has the greatest expected value. Unresolved contradictions, high-impact unverified claims, expensive planned experiments, policy ambiguities and results whose novelty depends on weak retrieval are not merely items in a to-do list. They are different classes of epistemic debt. A research environment can make that debt visible and route it to people or specialised evaluators according to cost and consequence. The objective is not to optimise a researcher as if science were a queueing problem; it is to prevent machine-generated abundance from hiding the few decisions that determine the direction of a project.

Persistent state also provides a better foundation for collaboration between institutions. Teams should be able to exchange a bounded research object with identities, provenance, rights, declared evidence and version references without exporting the entire internal workspace. One institution may accept an external dataset but not the interpretation attached to it. Another may accept a verified protocol but require local re-execution. This suggests that interoperability should occur at the level of typed scientific objects and receipts rather than through wholesale synchronisation of internal memories. Such boundaries become increasingly important when research companies combine proprietary knowledge with public science and when automated laboratories operate across organisational borders.

The architectural principle is therefore continuity without forced consensus. The research environment should know enough about scientific state to preserve dependencies, alternatives and authority, but not so much that every discipline must adopt one ontology or one method of reasoning. A project can use domain-specific schemas and still share a common lifecycle for source, candidate, evidence, validation, decision and version. That small common grammar may prove to be more important than any particular user interface. It is what allows models, evaluators, repositories and laboratory systems to change while the scientific history remains interpretable.

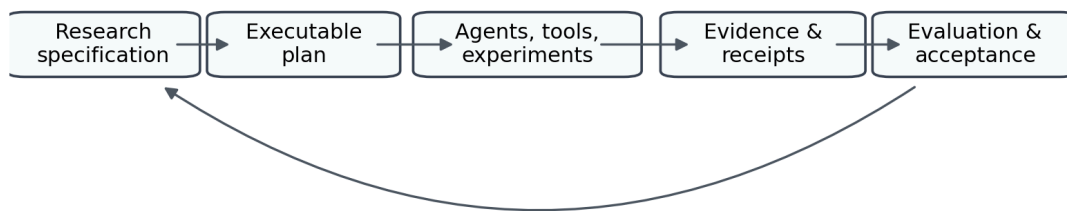
CHAPTER 3

From Scientific Intent to Executable Work

Specification-driven development became important to Axiologic because generative systems changed the economics of implementation before they changed the economics of intent. When code can be produced rapidly, poorly specified goals become the dominant source of waste. Research has the same problem in a more difficult form. An agent can execute a statistical pipeline or generate a literature review quickly, yet the scientific meaning of success may be underspecified. The system needs a representation of what is being attempted, what may be assumed, what must be preserved and what evidence would justify acceptance.

The word specification can sound hostile to exploratory science. The intended meaning is almost the opposite. A research specification is not a contract that predicts the result. It is a living statement of the current question and the boundaries of legitimate work. It can explicitly preserve alternatives, unknowns and exploratory branches. What it removes is accidental ambiguity about authority, data, method, resources and acceptance. The specification defines the frame within which exploration remains scientific rather than merely productive.

Figure 2. Specification-driven research as a versioned scientific loop.



Organizations repeatedly convert prose into executable obligations. Research protocols become validation rules. Regulations become compliance checks. Architectural decisions become invariants. Scientific theories become simulation requirements or admissible transformations. Project documentation becomes tests and monitoring conditions. The conversion is expensive because domain meaning and implementation meaning do not align automatically.

Large language models can draft specifications quickly, but the central risk is not poor grammar. It is semantic drift. A generated specification may omit an exception, strengthen a recommendation into an obligation, merge requirements with different scopes, invent a missing threshold, or silently choose one interpretation of an ambiguous term. The output can look more precise than its source while being less faithful.

A trustworthy process keeps the original theory or document set canonical. Candidate requirements receive identities linked to source spans and versions. Their normative strength, actors, preconditions, obligations, prohibitions, exceptions, temporal scope, measurable outcomes, and unresolved terms are represented explicitly. When the source is vague, the specification should preserve the gap as a clarification obligation rather than fill it with plausible detail.

The resulting object is more than a polished requirements document. It can generate downstream tests, proof obligations, static-analysis rules, monitoring queries, or implementation tasks. These executable artefacts remain derived from the accepted requirement and cannot redefine it. If implementation experience reveals inconsistency or infeasibility, the evidence returns to a new requirement version.

This environment provides a direct measure of symbolic learning. A circuit or template learned from one set of documents is valuable only if it transfers to unseen documents without silently changing

meaning. Evaluation can measure source coverage, unsupported additions, missed exceptions, contradiction rates, executable-test yield, correction effort, transfer across paraphrases, and localization of regressions after source edits.

The same approach applies to scientific theories. A theory can be partially compiled into concepts, relations, constraints, and procedures. New papers can then be checked for consistency with the formalized portion, and unresolved consequences can generate candidate experiments. Partial formalization is expected. The research question is whether the executable portion remains faithful and useful, not whether every theoretical insight can be reduced to code.

Specification generation is thus the bridge between document interpretation and persistent symbolic competence. Successful conversions can be promoted into reusable artefacts. Over time, recurring document structures and domain procedures become part of a domain-specific symbolic language model rather than being rediscovered through prompts for every task.

Contemporary agents often use a large language model as planner, reasoner, tool selector, memory interpreter, error judge, and final writer. This concentration of roles is convenient because one interface can handle many tasks. It is also difficult to audit because the same component decides what the request means, what action to take, whether the action succeeded, and how to explain the outcome.

The research programme proposes an incremental migration rather than a wholesale replacement. Existing agents can be instrumented to classify model calls by function. Recurrent calls used for retrieval, exact transformation, policy checking, solver orchestration, or validation can be compared with explicit symbolic alternatives. Once a circuit is validated for a functional role, the surrounding agent can invoke it through a stable contract.

Replacement is therefore functional, not token-level. A symbolic component need not generate the same words as a neural model. It replaces a role when the agent can submit the same class of tasks and receive semantically useful outputs with stronger guarantees or lower recurring cost. Exact policy checking may become fully symbolic. Open-ended interpretation or rhetorical realization may remain neural.

This framing avoids an unproductive opposition between symbolic and neural AI. Neural models are valuable for language interpretation, candidate synthesis, analogy, and communication. The question is where they should have authority. A model may propose a task interpretation or a new command implementation, while deterministic validators decide whether the proposal satisfies a reviewed interface and test suite.

The migration can be measured. A domain-specific system can record what fraction of successful tasks depend on runtime LLM calls, what roles those calls serve, which calls produce unverified assertions, and how performance changes as reviewed circuits accumulate. A mature symbolic language model may still contain neural components, but its stable domain competence is increasingly represented in inspectable artefacts.

Different domains can share the same runtime and constructor while loading different knowledge, theories, capabilities, and policies. A scientific-review model, a regulatory model, and a literary-analysis model need not pretend to possess identical notions of evidence. Generality can reside in common execution mechanisms and in transferable subcircuits rather than in one universal knowledge base.

The use case is successful when symbolic execution improves the surrounding agent's controllability without destroying its linguistic usefulness. The relevant comparison is not an immature prototype against a frontier model asked to do everything. It is a controlled replacement study in which one recurring role is migrated at a time and evaluated for accuracy, provenance, latency, cost, robustness, and human correction effort.

The aspiration to make natural-language-like text executable has a long history. It emerges from a genuine tension. Ordinary language is accessible but ambiguous. Formal languages are precise but often inaccessible to domain experts. Controlled natural languages attempt to occupy the space between them by restricting vocabulary, syntax, or semantics while retaining enough natural form to

remain readable. The classical survey literature emphasizes that CNLs are engineered languages, not naturally evolved sublanguages, and that they vary widely in purpose and degree of formality.

Attempto Controlled English is a foundational example. ACE constrains English so that sentences can be translated into discourse representation structures and Prolog. The resulting knowledge base can be queried and executed for simulation, prototyping, and validation. The importance of ACE is not that it solved unrestricted English. It demonstrated that a human-readable textual view can have a precise mapping to formal execution.

Common Logic Controlled English and related languages pursue English-like access to first-order or Common Logic representations. The Object Management Group's SBVR standard takes a business-oriented route by separating vocabularies, facts, and rules and allowing structured natural-language notations to express machine-processable semantics. These approaches show that formalization becomes more usable when domain concepts and rules are exposed in terms recognizable to stakeholders.

Grammatical Framework addresses a complementary problem. GF is a programming language and formalism for natural-language grammars with typed abstract syntax and reversible mappings to concrete languages. A shared abstract representation can be parsed from and linearized into multiple languages. This is particularly relevant when the executable representation should not be permanently tied to English surface forms.

Recent work connects controlled language to modern declarative solvers. CNL2ASP translates a controlled English specification into Answer Set Programming, including disjunction, choice rules, aggregates, and optimization constructs. NL2ASP adds a neural stage that translates freer natural language into the controlled form before deterministic conversion to ASP. These systems provide a concrete precedent for a hybrid pipeline in which a neural model proposes a normalized representation and a formal compiler owns the executable semantics.

Other traditions use natural-language-like text as a programming or testing interface. Inform 7 is a natural-language-based programming language for interactive fiction. Gherkin gives plain-text scenarios enough structure for Cucumber to treat them as executable specifications connected to step implementations. Their success shows that human-readable executable text can be practical when the domain, grammar, and operational binding are carefully constrained.

The approaches differ in where they place semantic authority. ACE aims for a controlled language with a formal interpretation. GF separates abstract and concrete syntax. Gherkin structures examples whose operational meaning is supplied by step definitions. Inform 7 is a full domain-specific programming language with an English-like surface. SBVR represents domain vocabularies and declarative business rules. CNL2ASP lowers a controlled specification to a particular reasoning formalism.

These distinctions matter for a human-inspectable executable representation. It would be misleading to describe a human-inspectable executable representation simply as another controlled English. Its declarations are readable and commands may adopt natural-language-like argument grammars, but the stable language does not attempt to define the full semantics of every phrase. It is closer to an extensible executable intermediate representation whose commands own domain-specific microgrammars. A natural-language interface may compile into a human-inspectable executable representation, and a controlled language may be implemented on top of it, without requiring the SOP kernel itself to become a universal grammar of English.

Imagine a research team asking an AI system to compare two theories, locate the decisive empirical disagreements, and propose an experiment. The answer is excellent. It isolates the right variables, notices a hidden assumption, and suggests an affordable design. Everyone is impressed. A week later, a similar request receives a shallow comparison. No one can determine whether the first result depended on a better source set, a fortunate model sample, an unrecorded prompt variation, or a temporary tool response. The team possesses a good answer, but it has not acquired a dependable research capability.

This gap is easy to miss because answers and capabilities can have the same surface form. Both may appear as polished text. Yet they belong to different ontological and institutional categories. An answer is situated: one request, one context, one set of available materials, one moment. A capability is counterfactual and reusable: it claims that under stated conditions a system can perform a recognizable responsibility again. That claim must survive variation.

The current the experimental meta-rational execution approach philosophy makes this distinction foundational. A raw response has no inherent owner, no stable scope, no reliable boundary between evidence and memory, and no explanation of why the next request should be treated in the same way. A capability, by contrast, needs an interface, resources, tests, provenance, observable failure modes, and a place in a larger composition (the experimental meta-rational execution approach Documentation, 2026b). The difference is not bureaucratic. It is the difference between an event and a practice.

The word “capability” often sounds purely technical, as though it named a function available in a software library. the experimental meta-rational execution approach gives it a richer meaning. A capability is a social contract among those who rely on a harness, those who maintain it, those who provide knowledge, and those who authorise its effects. It states what the system is entitled to attempt, what evidence it may use, which outcomes count as acceptable, and how it should behave when the problem exceeds its scope.

This contract has an inward and an outward face. Inwardly, it disciplines implementation. The capability should have one coherent responsibility. Its instructions, rubrics, knowledge, code, and tests should be inspectable. Its resource needs and permissions should be declared. Its output should have a recognizable form. Outwardly, the contract makes the system legible to domain experts and users. A person should not need to reverse-engineer a prompt in order to understand what role the system claims to occupy.

Consider a contract-review harness. “Review contracts” is not a capability in the useful sense. It is a domain label. A coherent capability might be “identify clauses that change the governing-law assumption in the approved template family.” That capability can name its inputs, jurisdictional sources, exclusion conditions, output artefact, confidence language, escalation rule, and tests. Another capability may assess indemnity structure. A third may compose an executive summary. The harness can then make a bounded claim without pretending that one procedure covers the whole legal world.

Capability as contract also clarifies why negative cases matter. Testing only successful activation teaches us whether a procedure can work. Testing negative transfer teaches us whether the system knows when the procedure should not apply. In adaptive systems, the latter is often the more important discipline. A new capability that improves five target cases but activates across fifty unrelated requests has not become useful; it has become contagious.

A successful interaction can reveal several kinds of reusable insight. It may expose a missing knowledge source, a recurrent transformation, a better decomposition, a stronger validation question, a cheaper route to the same result, or a boundary that should have been explicit. The temptation is to capture the interaction itself: copy the response into memory, add the user’s phrase to a prompt, or create a special rule for the incident. This often produces the appearance of learning while narrowing the system around accidental details.

the experimental meta-rational execution approach proposes a different conversion. The interaction becomes evidence for a capability gap. That gap is restated as a reusable responsibility. A candidate procedure is written or revised. It is tested on varied cases and on cases where it must not activate. Its relationship to existing capabilities is examined. It is evaluated as part of the whole harness. Only then may it enter an active generation.

This sequence is demanding because it refuses to identify repetition with generalisation. Repetition says, “we saw this before.” Generalization says, “we have isolated the invariant that makes these cases relevantly similar.” The first can be achieved by memory. The second requires conceptual work.

The distinction has a close analogue in science. A single observation can motivate a hypothesis, but it does not establish a law. An experimental anomaly can justify attention, but it does not determine the

theory that should replace the current one. In software, a bug report can justify investigation, but a patch that hard-codes the exact input is not a robust fix. the experimental meta-rational execution approach imports this methodological modesty into agent learning: experience proposes; evaluation disposes.

The capability test A response becomes evidence for capability only when the system can state the reusable responsibility, the conditions under which it applies, the resources and authority it needs, the observable artefact it produces, the ways it can fail, and the evaluation by which a future revision will be compared.

First, request output should not silently become persistent knowledge. Generated text may be useful for the present task while remaining unsuitable as a future source. It may summarise uncertain evidence, combine assumptions, or contain stylistic material that should not be retrieved as fact. Memory governance therefore requires explicit retention decisions and layered scopes.

Third, improvement must be versioned. A team should be able to compare the active capability with a candidate, identify the evidence for change, observe regressions, and restore the previous configuration. Otherwise adaptation is only drift with a positive story attached.

Fourth, the system must preserve failure as information. A rejected output, an unresolved validation, or a planner frontier can reveal where the current capability contract is incomplete. Forcing every request into a polished completion destroys precisely the evidence needed for responsible learning.

Finally, a harness should be judged by more than immediate fluency. Relevant measures include unseen task success, selection accuracy, use of exact tools, quality of qualification, preserved work after local failure, reviewer effort, latency, cost, negative transfer, and reversibility. These measures describe an institution of capability, not merely a model benchmark.

The larger ambition is not to eliminate improvisation. Improvisation is one of the strengths of language models. The ambition is to create a disciplined path by which useful improvisation can become maintained practice—and by which failed improvisation can remain visible without poisoning future work. The capability gap is therefore not a complaint about generative AI. It is the opening through which generative intelligence can be connected to engineering responsibility.

The practical opportunity is to move repeatability out of individual prompting skill and into a maintained system. Once scope, procedure, evidence, qualification, and ownership become explicit, a stronger LLM improves execution without forcing the organisation to redefine what it trusts every time the model changes.

A typical agent system has an appealing simplicity. Give a model a goal, a set of tools, and a loop. The model reasons, acts, observes, and continues until it declares completion. This pattern can be effective. It is also easy to build. Its weakness appears when the system's intermediate decisions become consequential.

Which part of the request was solved? Which source was considered authoritative? Why was one tool chosen rather than another? What assumptions were made before a solver was called? Which result was checked, and under what criterion? If a local premise changes, which downstream work is invalid? When may a generated statement enter memory? A transcript may contain clues, but it rarely provides a stable operational answer to these questions.

A prompt can contain a procedure, but it is a poor long-term home for a practice when several conditions hold: the procedure has distinct responsibilities, different knowledge sources, alternative methods, independent failure modes, or a need for local revision. In a large prompt, these elements coexist as prose. Their boundaries are understood, if at all, by the model at runtime. A change to one instruction may alter the interpretation of another. Intermediate results may not be named. Validation may be performed by the same model under nearly identical context. The system can produce a coherent answer while leaving the operational structure implicit.

This does not mean prompts are unserious. Natural-language instructions are an important procedural medium. The problem is not their linguistic form. The problem is the absence of an execution model around them. Source code becomes software not because text ceases to be ambiguous in

principle, but because languages, compilers, runtimes, tests, version control, and operating systems create a disciplined environment for interpretation and effect. the experimental meta-rational execution approach asks what analogous environment is needed when part of the procedure remains natural-language and model-mediated.

The same criticism applies to transcripts. A reasoning-action history records a temporal sequence, but sequence alone does not expose the dependency structure of the work. A later step may depend on a particular extraction rather than on the entire conversation. If that extraction is challenged, replaying the whole transcript can reproduce irrelevant work and entangle the repair with previous phrasing. A graph of named responsibilities permits a more local question: what must be recomputed because this value changed?

In formal programming languages, operational semantics describe how expressions change machine state. Natural-language agent systems usually lack a comparable public account. Their practical semantics emerge from the interaction of prompt, retrieved context, model, tools, memory, and control loop. The meaning of “review this evidence,” for example, is partly determined by which evidence was selected, which rubric was active, what tool was authorised, and what counted as completion.

Executable pragmatics begins from this runtime fact. Natural language does not need to become perfectly formal before it can participate in controlled execution. It needs an architecture that exposes the policies by which text becomes operational. The key questions are pragmatic: which reading is active, which context is admissible, what

At the front of the harness layer is a task representation that preserves human intent without forcing the whole request to be repeated at every step. The representation becomes a visible plan whose nodes carry meaningful responsibilities and dependencies, while a capability catalog gives each reusable unit an explicit scope, resource envelope, test set, and version.

The same layer maintains runtime state that distinguishes provisional from committed artefacts, binds work to language models, code, retrieval systems, solvers, or human review, and applies qualification procedures before results may propagate. It also keeps a trace of selection, effects, revision, and stopping conditions, together with retention and publication controls that separate immediate output, evidence, and reusable knowledge.

The first translation is from intention to work. A request is normalized, bounded by policy and budget, and decomposed into responsibilities. The second is from work to regime. Each responsibility is related to an appropriate mode of interpretation or computation: semantic judgement, deterministic transform, retrieval, symbolic reasoning, simulation, or human decision. The third is from output to artefact. A provisional result is qualified before it becomes an accepted value. The fourth is from experience to evidence. A trace and authorised feedback are packaged without assuming that they already describe a reusable improvement. The fifth is from evidence to capability. Candidate work proposes a general procedure and evaluation determines whether it earns adoption.

These translations are where many agent failures hide. A model may answer the wrong interpretation of the request. A planner may omit a necessary qualification. A retriever may select topically similar but legally irrelevant material. A symbolic solver may prove a property of a mistranslated specification. A critic may validate rhetorical completeness instead of evidentiary adequacy. A memory system may retain a tentative claim as fact. When the translations are implicit, the final error appears mysterious. When they are represented, the system can ask where the meaning changed.

A central design choice is to keep the authority-bearing runtime comparatively thin. The kernel should own the mechanics that must not be reinterpreted freely during active execution: parsing the intermediate programme, maintaining state, scheduling dependencies, enforcing budgets and permissions, isolating untrusted code, recording trace events, pinning versions, and controlling publication. Domain procedures, knowledge, filtering, planning strategies, qualification rubrics, and composition guidance belong in versioned capability units.

The division is direct: the runtime carries stable authority and lifecycle rules, while the capability library carries versioned domain procedures (the experimental meta-rational execution approach Documentation, 2026b). The runtime does not prescribe every professional action. It defines roles, limits, permitted effects, records, and legitimate transitions. Domain practice remains adaptable within those controls.

The separation also prevents a subtle circularity. If a system can rewrite the authority that decides whether its rewrite is valid, then evaluation loses a stable reference point. the experimental meta-rational execution approach does not claim that the runtime can never change. It claims that runtime change is a different engineering act from ordinary capability learning and should pass through a separate process. Active requests require a finite bootstrap, even if future deployments revise that bootstrap.

The harness layer is not merely middleware. It encodes a view of meaning. It refuses two extremes. The first is semantic monism: the belief that every task should ultimately be expressed in one privileged representation or solved by one universal reasoning regime. The second is unstructured contextualism: the belief that because meaning depends on context, explicit structure is futile.

The harness layer is where control becomes constructive. It does not suppress model interpretation; it turns interpretation into one visible operation among selection, planning, exact computation, qualification, human review, and retention. This is the difference between using an LLM and engineering a trustworthy capability around it.

Table 4. A research specification is an executable contract around exploration, not a prediction of the answer.

Term	Meaning in the research environment
Intent	The scientific question, target capability or decision that motivates the work.
Boundaries	Permitted data, effects, resources, assumptions, prohibited outcomes and human decision rights.
Evidence	What observations, tests, sources or formal checks are required before a result changes status.
Plan	A derived and revisable executable structure; it is not authoritative merely because it was generated.
Acceptance	The explicit conditions under which the system may publish, merge, escalate or abstain.

The strongest form of specification-driven research would make a research plan executable without making the science mechanical. It would let the system derive work from intent, but require every derived step to preserve the evidence and authority boundaries of the source specification. The implementation may use a conventional workflow engine, an agentic planner, the experimental meta-rational execution approach, a human-inspectable executable representation circuits, notebooks, laboratory software or a mixture. The scientific contract should survive the change of execution substrate.

This chapter also points to a new research object: the specification history itself. When a project changes direction, the important event is often not a new file but a change in what the team believes the problem to be. Capturing that transition gives future evaluators and models information that final papers rarely contain. Specification-driven science is therefore both an execution discipline and a way of making the evolution of scientific intent observable.

A useful way to test specification-driven research is to ask whether the specification remains valuable after the first execution fails. If the answer is no, the specification was only a prompt with better formatting. A scientific specification should support diagnosis: which assumption was violated, which evidence requirement was not met, which branch exceeded its budget, which tool behaved outside contract and which uncertainty was never resolved. The project can then revise the specification or the

implementation deliberately. This is the research analogue of debugging at the level of intent rather than patching a generated output until it appears plausible.

Specifications also create a surface for comparison between human and machine research strategies. Two agents can receive the same scientific contract and construct different plans. A human team can provide a third plan. Because the inputs, constraints, required evidence and stopping conditions are shared, evaluation can compare the strategies without confusing differences in task interpretation with differences in execution quality. This makes the specification useful as an experimental instrument for agent research. It enables controlled studies of planning, tool selection, model routing and escalation while keeping the scientific objective fixed.

The most important open question is how much meaning belongs in the specification. A specification that attempts to encode an entire theory, ontology and laboratory protocol before work begins becomes expensive and brittle. A specification that contains only a goal and a deadline does not constrain enough. The likely solution is progressive formalisation. High-level intent and non-negotiable boundaries are explicit from the start; more detailed objects are introduced when repeated execution, risk or verification justifies them. The environment should preserve the relation between each refinement and the source decision that created it.

This suggests several levels of executability rather than one binary state. At one level, a specification merely names required artefacts and reviewers. At a stronger level, it generates a deterministic workflow. At another, it generates a dynamic agent plan under explicit capabilities and budgets. At another, selected claims or procedures are lowered into formal constraints, symbolic circuits or proof obligations. A research environment should be able to combine these levels in one project. Demanding the strongest formalism everywhere would make the environment unusable; pretending all levels provide the same guarantee would make it untrustworthy.

Specification libraries could eventually become institutional assets. A validated literature-review contract, a reproducibility protocol, a privacy-preserving data-analysis template or an experiment-qualification pattern can be reused across projects with explicit scope. Their histories reveal which requirements were repeatedly misunderstood and which checks prevented failures. Coding agents can propose improvements, but promotion should remain evidence-based. This transforms specifications from static documents into governed, testable scientific capabilities while preserving the human ability to inspect and revise the intent they encode.

The deeper scientific hypothesis is therefore not that specifications eliminate ambiguity. They make ambiguity locatable. When an agent cannot determine whether a sentence is a hard requirement, a preference or an unresolved research question, the system can preserve that status instead of choosing silently. When two interpretations produce different plans, both can be represented and tested. The specification layer is valuable when it converts hidden disagreements about intent into explicit objects that can be resolved by evidence or authority.

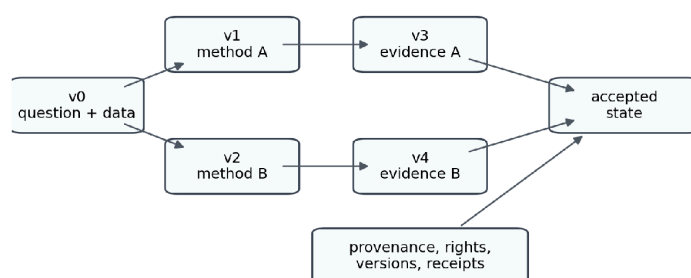
CHAPTER 4

Memory, Versioning, and the History of Scientific Thought

Version control solved a practical problem for software: many people and tools need to change an artefact without erasing the history required to understand, compare and recover it. Science has the same requirement but a larger object space. A research version can include a hypothesis, corpus boundary, data snapshot, protocol, model, code revision, environment, analysis, interpretation, review state and rights context. If only the code is versioned, the history is incomplete. If only the paper is archived, the causal chain that produced its claims is lost.

The existing ACHILLES IDE line already integrates Git and GitHub because they are indispensable for software and document history. The next step is conceptual rather than merely technical: to use branching, forking, merging and immutable identities as scientific semantics. A fork can represent a rival analysis without contaminating the accepted project state. A branch can preserve a high-risk hypothesis. A merge can require evidence rather than only conflict resolution. This richer version graph can become a core index of scientific change.

Figure 3. Branches and forks are scientific alternatives; provenance keeps their meaning recoverable.



Digital systems were designed under an assumption that is becoming progressively less reliable: content moves faster than verification, but human review can eventually restore context. A document can be emailed without its complete history because a person may telephone the sender. A database extract can lose provenance because an auditor can request logs. A decision can be recorded as a final value because the responsible team remains available to explain how it was reached.

Autonomous AI systems weaken this assumption. An agent can receive information, choose a tool, invoke an external service, update a record, communicate with another agent, and trigger a financial or operational consequence before a human understands that the workflow has started. The principal security problem is therefore no longer confined to whether a message is authentic. The system must determine whether the proposed action belongs to the accepted history, whether the authority remains current, and whether the evidence is sufficient for the consequence. A message may be correctly signed by a legitimate service and still be unsafe because the agent used a superseded patient record, a model version withdrawn after a safety incident, or an authorization that had been revoked seconds earlier. A receiving pharmacy system that verifies only the signature will accept a correctly signed but contextually invalid act.

The gap is not specific to machine learning. It exists in supply chains, digital identity, clinical research, financial messaging, software distribution, scientific publication, and public administration. AI increases its severity by reducing the time available for correction and by making plausible content inexpensive to produce. A malicious agent may not need to forge a durable record. It may need

The same property makes conventional logs insufficient. Logs are usually controlled by the service whose behavior they are intended to explain. They may be mutable, incomplete, inaccessible to another organization, or expressed in a format that is meaningful only to the original software. Even when they are retained honestly, they often record technical events rather than the semantic act: a file read rather

than the declared purpose of an analysis, an API call rather than the authority under which a decision became operational.

The a provenance-oriented data-sharing approach response is to move essential evidence closer to the information object and the act. A consequential change should create a signed event connected to the preceding accepted state. The event should commit to the resulting version, the controlling authority, the operational actor when distinct, the applicable policy, and any detailed provenance object required by the domain. An external witness may then register the commitment at the assurance level justified by risk.

This approach is sometimes described as notarization by default. The phrase must be interpreted precisely. It does not require publication of confidential data. It does not mean that every read operation deserves a permanent global record. It means that a material transition should normally produce evidence as part of the transaction rather than as an optional compliance process added later.

Materiality is domain dependent. A local note may need only a local signed history. A cross-institution clinical result may require a consortium receipt and a regulated provenance profile. A public safety directive may require several independent witnesses and threshold approval. The framework supplies a common form for producing and escalating evidence; policy determines the required level.

The receiving system should evaluate the evidence before consequence. This shifts the order of operations. Conventional systems often act and then audit. A verification-oriented system first resolves the object, checks the anchor or nano-ledger, establishes the capability and authority epoch, evaluates required provenance, and determines whether the witness state satisfies policy. Only then does it reconstruct protected content or permit an external side effect.

Human oversight remains necessary, but it becomes more specific. Instead of asking a reviewer to infer trust from a narrative, the system can present a verification report that separates what was established from what remains uncertain. A valid signature, a current branch, an accepted provenance profile, and an independent receipt can be shown independently. The reviewer can then focus on domain judgment, fairness, and exceptional circumstances.

The verification gap cannot be eliminated entirely. Sensors can be wrong, actors can collude, institutions can be corrupt, and validators can contain defects. The architectural objective is narrower and measurable: prevent valid-looking content from becoming operational merely because the surrounding history, authority, and provenance are difficult to obtain in time.

The basic unit of the web is often treated as a resource identified by a URL. The basic unit of a database is a row, document, object, or graph node. The basic unit of a filesystem is a file. These units are useful, but they are usually too thin for accountable exchange.

A file can be copied without its history. A database row can be exported without its policy. A URL can resolve to different content over time. A message can be forwarded without the context in which it was authorized. The content survives; the responsibility dissolves.

This boundary produces two complementary views. From the inside, a DSU resembles a small filesystem or private data environment. It can contain folders, records, media, configuration, credentials, and code. The user or agent reconstructs this environment only after resolving the correct capability and verifying the relevant history.

From the outside, a versioned DSU is associated with an anchor or nano-ledger. Observers may see only an identifier and a chain of cryptographic commitments. They may be able to verify that versions exist, that the sequence has not been silently rewritten, and that authorized keys approved the transitions, without reading the content itself.

Near-chain is not a synonym for every off-chain database. It describes a deliberate evidentiary relationship: the external content can be matched to commitments, history, authority, and policy. Far-chain systems can consume or derive data, but their outputs need explicit provenance before they re-enter the accountable history.

Versionless local unit A lightweight object exposes only its current state and deliberately omits version history. This is suitable for simple local or edge state, but it is not evidentially equivalent to a versioned object backed by a nano-ledger or to a programmable micro-ledger.

The VersionLessDSU direction is important because it prevents the architecture from turning history into dogma. A unit that does not claim durable lineage can remain simple. The boundary becomes dangerous only when a versionless object is presented as though it offered the same continuity, dispute resistance, or reconstructability as a versioned one.

The word sharing can be misleading if it suggests that every DSU is widely distributed. A DSU can remain entirely local. It becomes a sharing unit because its access and evidence are structured so that it can cross a boundary without losing its identity. The possibility of controlled exchange is built into the object.

Consider a research dataset. In a conventional workflow, the data may live in object storage, access rights in an identity system, transformation logs in a pipeline database, code in a repository, model parameters in another service, and approvals in email. The dataset exists only because an organization keeps these fragments coordinated.

Per-object or per-process units allow governance to become more granular. A clinical consent can be attached to the relevant data unit. A supply-chain event can be verified without exposing unrelated commercial data. A model run can carry its own inputs, parameters, and outputs. A contract negotiation can preserve the sequence of offers without publishing the confidential text to the world.

In the AI age, this precision becomes indispensable. An agent should not receive a broad account credential when it needs one temporary capability to read one dataset for one declared purpose. It should not write to an unbounded enterprise data store when it needs to append one signed result to one accountable history. The DSU boundary makes least authority technically plausible.

a provenance-oriented data-sharing approach uses the term anchor for the object-scale history associated with a DSU. In this book, anchor and nano-ledger name the same conceptual structure. The term nano-ledger emphasizes that the anchor is not merely a single hash or a pointer stored somewhere else. It is a compact ledger identified by an AnchorID and extended through cryptographically authorized append operations.

The simplest nano-ledger has one recurring operation: append a commitment to a new DSU version. Each entry binds the stable AnchorID, the preceding entry, a timestamp or sequence claim, and the hash link of the BrickMap representing the new version. The controller signs the operation. The resulting sequence is independently verifiable even when the confidential BrickMaps and data bricks remain outside the anchoring ledger.

The distinction matters because an anchor can exist before a distributed ledger is introduced. A local implementation may store the signed sequence in an append-only file. A company may place it in an organizational transparency service. A consortium may register it in a permissioned ledger. A public infrastructure may receive selected checkpoints. These choices change the strength and independence of the witness, but they do not change the identity of the nano-ledger.

A control transition illustrates why the anchor is a ledger rather than a static pointer. The object identity should remain stable while the authority to append future entries changes. The current controller signs an entry containing the new public control key. Subsequent version commitments are accepted only under the new key. The prior controller remains visible in history, but cannot legitimately extend the object after the transition.

This mechanism supports a precise interpretation of ownership for unique digital objects. Ownership is not inferred from possession of bytes, because encrypted content can be copied. It is represented by the current authority to admit successors to the stable nano-ledger, together with whatever legal or domain rules give that authority meaning. A transfer therefore changes control of future history rather than rewriting the object identity.

The comparison with non-fungible tokens also clarifies where the model is not appropriate. A DSU and its nano-ledger are natural for unique or bounded objects whose content, metadata, provenance, and control history matter. Fungible assets that require atomic global balance changes may still require a consensus system with shared execution. Object-scale ledgers do not remove the need for global consensus where the domain genuinely contains global invariants.

The external ledger has a narrower responsibility than is often assumed. The nano-ledger entries already use signatures and chaining to make unauthorized modification detectable. A shared anchoring ledger primarily strengthens freshness, single-branch visibility, and availability. It helps the verifier determine that the history presented is the latest accepted history rather than a valid but truncated prefix or a branch shown only to one participant.

Freshness should not be reduced to one timestamp. A verifier needs to know the last sequence known to the required witnesses, whether a pending append exists, whether an authority epoch has changed, and what delay the domain permits between local commit and external receipt. An old entry can remain perfectly authentic while being unsafe for present use.

The nano-ledger is deliberately compact. Detailed provenance, validation traces, prompts, model outputs, business documents, and personal data should normally remain encrypted and be referenced by commitment. This protects confidentiality and keeps verification costs proportional. A compact history can still become large in high-frequency systems, so aggregation, Merkle checkpoints, retention rules, and evidence summaries may be required. nano-ledgers can be related without being merged. A transfer between two DSUs can produce an aggregate event that commits to the accepted state of each nano-ledger. A ledger can checkpoint a batch of nano-ledger roots. A micro-ledger can record a complex domain process and place its accepted state commitment into the DSU anchor. These compositions preserve local object identity while allowing higher-level coordination.

The anchor-as-nano-ledger definition corrects a recurring conceptual error. The nano-ledger is not a future refinement smaller than the anchor. It is the anchor understood as the minimal cryptographic ledger of one object. The term is useful because it makes clear that a provenance-oriented data-sharing approach distributes ledger properties to the scale at which continuity and authority are actually needed.

A nano-ledger answers a narrow question: which commitments and control transitions form the accepted continuity of one bounded object? Many applications require a richer question: did each domain-specific operation produce a valid successor state according to the rules that governed the object or process? a provenance-oriented data-sharing approach uses the concept of a micro-ledger for this richer scoped history.

A micro-ledger is a ledger at the scale of one data structure, process, relationship, or smart-contract-like object. It can support many operation types, validation rules, actors, and state transitions without requiring a global consensus network to execute every rule. The history remains bounded, and external witnesses can be used only for the ordering and independence that the threat model requires.

Self-Validating Data is a possible implementation of a micro-ledger. An SVD combines a history representation, provenance metadata, primitive cryptographic operations, state operations, and replay or validation logic. A receiving environment loads the history, associates the applicable validation artifacts, replays the accepted commands, and obtains a state that it can read or extend only if validation succeeds.

This relation should be stated carefully. Not every micro-ledger must be an SVD. A micro-ledger may be implemented through a conventional event-sourced service, a domain-specific ledger, a replicated state machine, a DAG, or a CRDT with appropriate validation semantics. Not every DSU must contain a micro-ledger. A DSU may be an immutable package, a versioned content object, a wallet, or a versionless local unit. SVD is the pattern used when the history and the rules for interpreting that history must remain bound to the data.

The phrase self-validating does not imply that data executes itself or guarantees truth. A compatible execution environment performs the validation. The term describes architectural locality: the object

carries or resolves to the history, rules, credentials, provenance, and execution references needed for the receiving environment to evaluate the transition without relying exclusively on a remote service controlled by the originator.

An SVD separates primitive operations from state operations. Primitive operations provide stable facilities such as hashing, signature verification, key resolution, time or witness access, and interaction with the execution environment. State operations define how domain commands read or transform the current state. This separation allows different language implementations to agree on semantics while using different runtimes.

The most straightforward SVD implementation records a command for each change and derives the current state by replay. This model provides complete interpretability but may become expensive as the history grows. Checkpoints, snapshots, proofs of replay, compact state commitments, and selective validation can improve performance, provided that the historical rules and the path from checkpoint to current state remain verifiable.

The representation of history is a design choice. A linear chain provides simple ordering and is appropriate when one accepted successor must be selected. A DAG preserves parallel contributions and causal relations, but requires conflict and merge rules. A CRDT supports convergence among replicas, but the domain still needs to define which operations are authorized and which converged states are acceptable. The essential requirement is that the representation and validation semantics are explicit.

A micro-ledger SVD can keep its full command history off-chain in a DSU while the DSU nano-ledger stores commitments to accepted micro-ledger states. This division is important. The micro-ledger contains the rich programmable history; the nano-ledger provides compact continuity and can receive external witness receipts. The shared ledger does not need to execute confidential business rules or store the underlying data.

This architecture supports the earlier a provenance-oriented data-sharing approach concept of secret smart contracts. The phrase describes placement, not a claim that hidden code is automatically trustworthy. Domain logic executes near confidential data inside an authorized Security Context. The witness records commitments and order. Participants can independently replay or validate the micro-ledger when they possess the required capability and artifacts.

Executable choreographies are a natural application. Instead of one central service orchestrating every participant, each organization or agent validates the input state it is authorized to see, performs its permitted transition, and returns signed evidence. An aggregate event can commit to the participating micro-ledger

AI workflows make these requirements more concrete. A task package can identify the input DSU versions, permitted tools, model or service references, policy, expected output commitments, and required provenance profiles. The receiving agent executes inside a constrained Security Context. The result becomes a command in the relevant micro-ledger only when the required evidence is present. The agent's natural-language explanation may accompany the event, but it does not replace the committed inputs, authority, and validation report.

The SVD idea also applies to collaborative knowledge. A scientific claim, review, correction, and competing interpretation can be represented as attributable operations rather than overwritten text. A multi-perspective publication need not force all participants into one consensus view; it can preserve claims, dependencies, reviews, and audience-specific projections while maintaining verifiable authorship and history. The challenge is not only cryptographic. Relevance, moderation, incentives, privacy, and lawful removal require governance.

SVD therefore occupies a precise place in the framework. The DSU provides the protected object boundary and reconstructable storage. The anchor or nano-ledger provides compact continuity and current control. The micro-ledger provides domain-specific state history. Self-Validating Data is one method for implementing that micro-ledger so that rules and evidence remain available where the data is interpreted.

Most digital identifiers answer one question: what object or account are we talking about? Access tokens answer another: what may the bearer do? Cryptographic keys answer a third: who can sign or decrypt? In conventional systems these functions are often separated across URLs, usernames, passwords, access-control databases, certificates, and key-management services.

The early a provenance-oriented data-sharing approach families illustrated the idea through a hierarchy. A high-power key could control and update a DSU. From it, a read capability could be derived. From the read capability, a zero-access identifier could be derived that revealed only enough to verify the anchor and version history. LightDSU preserves the same functional structure with three compact roles: full control, read and reconstruction, and verification without content access.

A zero-access witness can preserve history without reading the object. A zero-knowledge system can prove a specific property of hidden data. Either may complement the other, but neither should be advertised as proof that the underlying statement is factually true.

Collapsing these instruments creates dangerous ambiguity. A credential that says an agent is a purchasing agent does not itself grant access to every purchase-order DSU. A DID identifies the actor but does not establish current authorization. A capability authorizes an operation but does not prove legal personhood. A profile or persona organizes context but should never become an invisible source of ambient authority.

This hierarchy embodies least authority. A person who needs to verify that a record exists should not receive the power to decrypt it. A service that needs to read a dataset should not receive the power to publish a new version. An AI agent that needs one temporary input should not inherit the full authority of the human or organization that launched it.

Derivation creates a one-way delegation path. The holder of a stronger capability can produce a weaker one, but the weaker capability cannot recover the stronger secret. This is conceptually different from asking a central access-control server to remember an entry for every delegation. The cryptographic relationship carries part of the proof.

On the ordinary web, a URL identifies a location and TLS authenticates a server, but the right to read the resource is usually mediated through a session, cookie, account, or bearer token. The user must trust the server to apply the right policy and serve the right version. In a KeySSI-oriented system, the identifier can carry enough information for a resolver to locate the relevant trust domain, verify the anchor, and derive the decryption context locally.

Capability systems also create operational risks. A secret capability can be copied; a human-readable identifier derived from weak input can be guessed; a long-lived master capability can become a single point of catastrophic loss; and an AI agent can leak authority into prompts, logs, tool calls, or memory. The architecture therefore needs explicit capability-lifecycle disciplines.

Capability semantics A well-designed capability says something precise: "The bearer may perform this operation on this unit under this policy." A badly designed credential says only: "The bearer is inside." The constrained form is preferable in systems that require least authority and independent verification.

The rise of autonomous agents makes capability design a foundational security problem. An agent may be competent enough to perform a task and still be unsafe if its authority is too broad. It may be hijacked, confused by an adversarial input, or induced to use a legitimate tool for an illegitimate purpose. The correct response is not to trust the agent's reasoning more. It is to give the agent authority that is narrow enough to survive imperfect reasoning.

An agent should be able to present a cryptographically verifiable capability tied to a data unit, purpose, policy, and evidence trail. The receiving system should be able to verify the capability without opening an unrestricted session. The resulting action should be appended to the unit's history under both the object's control key and the actor's identity.

This definition matters because keys without an execution environment are unusable. A person cannot meaningfully control a capability if every application can silently extract it. An enterprise cannot

claim sovereignty if private keys travel through generic microservices and application logs. A secure system needs a place in which sensitive operations occur without exposing the secrets themselves.

This prevents a common antipattern: treating a key-management system as a decryption web service that hands plaintext to any application with network permission. The key is protected, but the authority is still ambient. A better enclave evaluates both the caller and the intended operation.

An edge wallet maximizes personal or local sovereignty. Keys and reconstruction happen on a device controlled by the user or organization. The advantages are privacy, independence, and reduced server visibility. The disadvantages are device loss, limited availability, constrained computation, and difficult recovery.

A cloud wallet maximizes availability, integration, and organizational management. It can run continuously, connect to enterprise systems, and use professional backup and hardware security. The disadvantages are concentrated trust, metadata exposure, and the risk that the service provider becomes an unchallengeable intermediary.

The correct architecture is often mixed. A high-value signing key can remain in a hardware enclave while encrypted bricks are replicated in the cloud. A personal device can approve sensitive operations while a cloud agent handles notifications and availability. An enterprise can maintain a shared enclave for organizational data while preserving separate personal contexts for employees or patients.

The Self-Sovereign Application, or SSApp, extends sovereignty to code. Data and signed application logic can be packaged in DSUs and executed in compatible wallets or sandboxed environments. The application becomes portable with the data rather than permanently attached to one server.

The W3C provenance model offers a useful foundation by distinguishing entities, activities, and agents. An entity is the data or artifact. An activity is the process that used or generated it. An agent is the person, organization, device, or software that bears responsibility.

This triad is more powerful than a creation timestamp because it captures transformation. Most important data does not have one origin. It passes through acquisition, normalization, analysis, aggregation, review, publication, and reuse. Each step changes what the data can mean.

One operation may create several events. An AI analysis of genomic data might produce an access event for reading the dataset, a provenance event for the model run, a data-use record showing that the purpose was permitted, and a policy event if the output enters a regulated retention class.

A dataset can state: "Any derived model must include the dataset commitment, code commitment, model version, parameters, evaluator, and output hashes." A regulated record can state: "Any modification must include a reason and an attributable actor." A consent-bound dataset can state: "Any use must record the declared purpose and the applicable policy version."

AI reproducibility creates unusual provenance requirements because AI systems may depend on artifacts and services that are difficult to preserve. A result may depend on a model whose weights are unavailable, a remote API that changes silently, a non-deterministic sampling process, hidden safety filters, retrieval sources that disappear, or prompts assembled dynamically from user data and tool outputs.

The relevant unit is not necessarily one file or prompt. A harness operates through a resolved configuration: capability versions, instruction resources, knowledge, model bindings, tool policies, budgets, isolation settings, retention rules, and interface policy. The experimental meta-rational execution approach calls the capability component a catalog generation and associates it with a settings snapshot. A request pins the resulting operational configuration before selection begins (the experimental meta-rational execution approach Documentation, 2026b).

Rollback is often described as restoring a previous file. In adaptive systems, that may be insufficient. A capability revision can depend on changed settings, neighbouring units, knowledge versions, or interpreter bindings. Reverting only one fragment may create a configuration that never existed or passed evaluation.

This property supports research as well as operations. Experiments can be reproduced against archived generations. Baseline and candidate comparisons have stable identities. A regression can be localised to a transition. Historical analysis can ask whether improved quality came from a procedure, a model route, a knowledge source, or a validator change.

Settings are often treated as a miscellaneous control panel. the experimental meta-rational execution approach treats them as explicit deployment policy. They determine which semantic model roles map to which providers or local models, how much time and compute a request may use, how many isolated operations may run, which connectors are available, what may be retained, which model providers are permitted, and what authentication or streaming policy applies.

The distinction between semantic role and provider identity is particularly useful. A capability can request “fast interpretation,” “coding,” “deliberate reasoning,” or “planning” without hard-coding a vendor. Deployment settings resolve those roles according to cost, privacy, availability, locality, or policy. The request snapshots the binding, so model choice remains traceable.

The same person may occupy several roles in a small team, but the system should still distinguish the actions. A developer editing a candidate is performing a different act from the same developer approving publication. The trace and interface should make the transition visible.

Authority is exercised when a request is admitted and bound to a session, when sources or connectors are accessed, and when tools are invoked or side effects are attempted. It is exercised again when provisional artefacts are committed and when outputs or feedback are retained.

This principle is especially important for privacy. A session may contain sensitive context that is necessary for current work but prohibited from learning. A capability may be allowed to query an approved internal database without being allowed to export retrieved

A capability’s identity should reflect its contents or immutable version, not merely a mutable name. Content-addressed identity supports reproducibility: the trace can point to the exact instruction, code, and knowledge used. A catalog generation resolves human-readable responsibilities to immutable capability versions.

Provenance extends beyond code. Knowledge should record source, authority, scope, transformation, review, and intended harness use. Instruction changes should identify the evidence and rationale. Evaluation cases should distinguish origin and learning-use permissions. Publication should record who approved the transition and which evidence was considered.

Governance is sometimes portrayed as friction that prevents systems from learning quickly. The opposite can be true. People are more willing to teach a system when they know how their feedback will be used, who can inspect it, whether it can become permanent, and how a harmful change can be reversed.

The deeper human-centred definition of adaptation is not increased autonomy for its own sake. It is reduced distance between human intention, operational knowledge, and reliable action. People teach through examples, constraints, sources, and critique. Coding agents translate that teaching into candidate practice. The runtime keeps the boundary between contribution and activation unmistakable.

Current implementation snapshot — publication discipline Current documentation describes candidate-only workspaces, complete generation staging, affected replay, full regression, sealed holdout, explicit publication, request-level generation pinning, and generation rollback. It also states that the current A/B/C learning experiment is planned rather than yet reported as completed evidence. This distinction between implemented mechanism and unproven outcome is part of the research discipline.

Versioning and governance give improvement a grammar. A team can state: the prior generation behaved this way; these traces revealed a capability gap; this candidate encoded a proposed invariant; this evaluation compared it with the baseline; these side effects were observed; this authority approved publication; this generation activated the change; and this earlier generation remains a rollback target.

Coordinating specialist interpreters is how the experimental meta-rational execution approach avoids a false competition between LLMs and exact methods. The LLM handles incomplete semantics and

translation; deterministic and formal components handle stable relations; human judgment carries institutional responsibility. The harness makes those boundaries explicit and testable.

The broad competence of language models creates a powerful temptation: route every local task through the model and ask it to approximate whatever method would have been appropriate. This can work surprisingly often. It also discards the strongest property of mature computational methods—their specialised semantics.

A parser can deterministically recognise a formal grammar. A database can retrieve records under explicit query semantics. A symbolic solver can establish satisfiability or return a counterexample. A simulator can generate consequences under a model. A statistical procedure can quantify uncertainty under stated assumptions. A human expert can exercise institutional judgement and accept responsibility. A language model can interpret messy material, suggest structure, translate interfaces, synthesize prose, and operate where semantics remain incomplete.

the experimental meta-rational execution approach coordinates these as a set of specialist interpreters. The runtime’s intelligence lies partly in deciding which component is adequate for which bounded subproblem, what evidence that component must return, and how its result should be reintegrated into the plan.

Table 5. Version control, provenance and research memory solve different layers of the same continuity problem.

Term	Meaning in the research environment
Git history	Excellent for immutable file and code evolution, branching, diffs and collaboration; insufficient by itself for scientific semantics.
W3C PROV	Interoperable representation of entities, activities and agents involved in producing artefacts and data.
RO-Crate	A practical research-object packaging layer for data, software, workflows, metadata and provenance.
OpenDSU	Axiologic technology for bounded objects, controlled sharing, identity, version commitments, rights and verifiable history across organisations.
Scientific memory	The semantic layer that records what a version means for hypotheses, claims, experiments and decisions.

The important boundary is between proof of history and proof of truth. Git can establish that a file changed. A cryptographic anchor can strengthen claims about continuity or authorship. A provenance graph can show which activity produced an artefact. None of these mechanisms establishes that the scientific interpretation is correct. The research environment needs them because scientific judgement is impossible to audit when history is missing, not because history substitutes for judgement.

Once this distinction is explicit, versioning becomes a research field in its own right. We need schemas for semantic diffs between scientific states, methods for training on version graphs without leaking confidential work, ways of compacting enormous histories while preserving decision-relevant events, and protocols for cross-institution provenance. a provenance-oriented data-sharing approach, W3C PROV, RO-Crate and Git supply useful pieces. The research environment must discover how to compose them around the life cycle of scientific claims and experiments.

CHAPTER 5

Agents as Scientific Roles, Not Features

An AI agent becomes scientifically interesting at the moment it can act rather than merely answer. Action changes the threat model and the epistemic model at the same time. A wrong sentence is a defect; a wrong tool call can overwrite a dataset, spend a compute budget, expose confidential material or launch an experiment. The surrounding harness therefore matters as much as the model. Research agents need an operating environment that defines what they may see, what they may change, which tools they may invoke, how much work they may consume, and what evidence must be returned.

Axiologic has explored this layer through related but distinct projects. the deployment and isolation infrastructure focuses on deployable isolation and agent packaging. the agent capability infrastructure explores capabilities, orchestration and reusable skills. the experimental meta-rational execution approach treats the harness as a versioned operational system in which procedures, interpreters, knowledge and policies can be qualified and published. the current Axiologic workspace provides the workspace in which those runtimes become useful to people. The research environment should build on these components without freezing their current interfaces as the final scientific architecture.

An agent harness is the software scaffold that turns a foundation model into an operational actor. It manages instructions, state, context, tool exposure, permissions, scheduling, retries, handoffs, human approvals, traces, and stopping conditions. The term is useful because it separates model capability from system capability. Two products can use the same model and achieve different results because one provides a better interface, smaller relevant context, stronger tools, executable feedback, and more disciplined recovery. Conversely, a strong model can appear weak inside a harness that hides state, overwhelms it with tools, or evaluates the wrong outcome.

The harness is not identical to a framework. A framework supplies programming abstractions and reusable components. A harness is the concrete control arrangement around a model in a particular system. The same framework can host several harnesses with different authority and evaluation. Nor is the agent harness identical to an evaluation harness. The evaluation harness supplies tasks, environments, scorers, concurrency, recording, and aggregation so that agent systems can be compared. In mature development, the two are connected: every production trace can become evaluation material, and every evaluation should exercise the same controls that matter in deployment.

The harness can be understood as a control plane around a probabilistic data plane. The model handles semantic interpretation and generation. The control plane determines which context is visible, which operations are legal, how calls are routed, where state is stored, what must be checked, and how the process is observed. This division resembles operating systems, database engines, and

By 2026, agent infrastructure is no longer a peripheral engineering concern. Mature systems increasingly separate the model from the harness that manages tools, memory, execution environments, tracing, evaluation, and human intervention. This industrial convergence matters more than the details of any one framework. It suggests that serious scientific agents will require a surrounding control layer that can define roles, restrict capabilities, persist state, observe actions, and recover from failure. The research question is how to extend these engineering patterns with scientific semantics: evidence status, provenance, experimental authority, and project-level memory.

The first responsibility is state management. A serious runtime distinguishes user input, environment state, working variables, session history, durable memory, and artifact storage. It supports checkpoints so a long task can resume after failure or approval. It controls isolation so one user's context does not leak into another's. It records versions of prompts, models, tools, and knowledge so a trajectory can be reconstructed. State is the difference between a demo that appears coherent for one run and a system that can be operated over time.

Context engineering is the harness responsibility most likely to be underestimated. The runtime decides which instructions, tool schemas, memories, artefacts, prior observations, and policies become visible for the next inference. More content can increase coverage and simultaneously reduce attention,

increase latency, expose secrets, preserve stale state, and magnify prompt injection. Less Context, Better Agents reported an enterprise benchmark in which pruning and summarisation improved task completion while reducing tokens and elapsed time relative to full history.

ContextBudget formulates compression under a budget as a sequential decision problem. These are early results, but they support a strong systems conclusion: context is an actively managed operating substrate, not a transcript dump (Lodha et al., 2026; Wu et al., 2026).

The third responsibility is action mediation. Tool calls must be parsed, validated, authorised, executed, and returned in a form the model can use. The runtime can enforce schemas, rate limits, cost limits, data boundaries, and approval requirements. It can wrap side-effecting operations in transactions or dry runs. It can expose safe synthetic tools rather than raw credentials. It can translate low-level errors into structured recovery signals. The model should not directly own secrets, network access, or irreversible authority merely because it can produce the right JSON shape.

The fourth responsibility is scheduling. A scheduler decides when to continue a loop, when to branch, which work can run in parallel, when to use a smaller model, when to call a specialist, and when to stop. Scheduling is increasingly cost-aware. It can allocate a reasoning budget according to task value, use cascades, or select a deterministic implementation when a stable skill exists. The scheduler also handles retries and backoff. Retries should be classified, because repeating a transient network call differs from repeating a reasoning failure with identical context.

The fifth responsibility is verification and governance. Guardrails can inspect input, output, tool arguments, and environment state. A verifier can test code, check a schema, compare a claim with retrieved sources, or enforce a business rule. Human approval can be inserted before a high-consequence action or after evidence is assembled. Policies should be local to the action boundary: reading a public page and issuing a refund do not require the same assurance. The harness converts abstract principles such as least privilege and human oversight into executable gates. OpenTelemetry semantic conventions reflect the movement toward vendor-neutral instrumentation. Observability makes debugging possible, but its deeper function is epistemic: it records what evidence and authority were present when the system acted. A trace is not proof that the action was correct. It is the minimum material from which correctness, cost, and accountability can later be assessed.

Current industrial agent platforms converge on a similar lesson: a useful agent is not only a prompt attached to a model. It operates inside a runtime that manages tools, state, permissions, traces, retries, and interaction with humans. For research, this runtime must go further. It should know whether an action is exploratory or authoritative, whether data may leave an institution, whether an output has been independently checked, and which project object the action is allowed to modify. These are responsibilities of the environment rather than properties of the language model itself.

The same convergence appears across model-agnostic and enterprise-oriented agent systems. Sequential, parallel, and iterative execution are becoming ordinary engineering primitives; so are telemetry, configuration, human approval, and deployment portability. The relevant implication for scientific work is that multi-agent research should be treated as a managed service environment. The open questions concern scientific coordination: how to divide responsibilities, how to prevent duplicated or contradictory work, how to expose partial results, and how to preserve meaningful traces when agents operate concurrently.

Enterprise deployments add another lesson: agent identities and actions must be connected to organisational permissions, data boundaries, and accountability. Research intensifies this requirement because projects often cross institutions and contain sensitive or unpublished material. Every additional connector and delegation boundary increases the need for least privilege, provenance, clear ownership, and cancellation. The scientific infrastructure should therefore treat identity and authority as part of the research process rather than as an administrative layer added after deployment.

A second family of industrial systems focuses on durable workflow state and on agents that operate directly on complex computer environments. These patterns show why checkpoints, interrupts, sandboxes, tests, and event streams matter. They also expose the limit of generic orchestration: knowing that an agent executed a command is not the same as knowing what that command means

scientifically. Future research infrastructure must connect operational traces to scientific objects and decisions.

Interoperability protocols increasingly allow agents to discover and invoke external tools and resources through common interfaces. This reduces integration cost, but it does not establish trust. A tool description does not tell us whether the provider is reliable, whether the current user is authorised, whether returned content should be treated as data or instruction, or whether an action satisfies the project's scientific and legal constraints. Research infrastructure therefore needs a governance layer above interoperability: authentication, capability restriction, provenance, context filtering, audit, and explicit data-handling policy.

Communication among agent services introduces a related but distinct problem. Future systems may need agents from different vendors, institutions, or laboratories to exchange tasks, artefacts, and progress without sharing the same internal implementation. Common protocols can make such interaction technically possible, but accountability still belongs to the surrounding research organisation. Identity, mandate, confidentiality, cancellation, provenance, and responsibility for a composed result must remain visible at the project level.

Protocol standardisation does not remove semantic ambiguity. Two tools can share a schema and disagree about side effects, freshness, or authority. Two agents can exchange a task and hold different assumptions about completion. Interoperability therefore needs capability descriptions, provenance, versioning, trust domains, idempotency, cancellation, and error taxonomies. The next phase is likely to resemble distributed systems more than chatbot integration.

Research on harnesses is becoming more explicit. Architectural Design Decisions in AI Agent Harnesses studies design variation across dozens of open systems, while AI Harness Engineering formalises the model-harness-environment system and proposes auditable episode packages. MemoHarness, published as a July 2026 preprint, explores harnesses that retrieve and adapt control experience from prior executions. These works differ in maturity, but together they mark a conceptual transition: prompts and workflows are no longer the only optimisation target; context policy, tool exposure, memory, verification, and intervention logic are themselves learnable or designable artefacts (Huang et al., 2026; Wei, 2026; Zhong and Zhu, 2026).

This expansion raises a methodological problem. A more elaborate harness can improve a benchmark by adding hidden tools, privileged information, more inference, or evaluator-specific rules. AI Agents That Matter argues that practical evaluation must include cost, reproducibility, held-out tasks, and complete system configuration. Harness research therefore needs ablation: hold the model and environment fixed, add one control responsibility at a time, and inspect trace-level effects. When the application is local, the benchmark should also be local. A system designed for one recurring enterprise process should be judged on that process and its failure consequences, not marketed as generally superior because it has more components (Kapoor et al., 2024).

Evaluation should operate at three levels. Component tests verify parsers, schemas, permissions, and deterministic validators. Trajectory tests inspect routing, context, tool selection, recovery, and stopping. End-to-end tests evaluate the resulting environmental state under repeated trials, realistic budgets, and perturbations. The episode package should include model and prompt versions, tool and data versions, selected context, actions, observations, evaluator

The frontier is a harness that can improve without silently rewriting its own authority. Production traces and user feedback can reveal missing context, brittle instructions, redundant calls, and candidates for deterministic implementation. The system may propose changes to prompts, skills, routing, evaluators, or code, but those changes should be evaluated offline, compared against the previous version, reviewed according to risk, and promoted with rollback. This controlled learning loop anticipates the sleep-phase design explored in Axiologic's the experimental meta-rational execution approach research.

the agent capability infrastructure is a project-specific agent runtime developed around the ACHILLES and AssistOS research direction. Its purpose is pragmatic: when a class of requests recurs, the system should be able to transform repeated language-model improvisation into a capability with

clearer context, execution semantics, review, and operational evidence. The architecture belongs to the broader 2026 movement from prompt engineering toward harness engineering, but it does not assume that one harness pattern should dominate every domain.

The central position is deliberately modest. the agent capability infrastructure does not attempt to beat the ReAct pattern in general. ReAct remains an excellent exploratory mechanism when the problem is unfamiliar, observations redefine the next step, or no stable procedure exists. The Achilles question is narrower and more engineering-oriented: where can recurring work be made more trustworthy and more efficient by moving part of the process into explicit orchestration and executable components? When such an opportunity exists, the aim is to implement and validate it quickly rather than to preserve generality as an abstract virtue.

The relevant evidence is local: accepted result quality, latency, total inference and tool cost, run-to-run variance, amount of human review, context size, failure severity, and recoverability. A specialised agent can be a meaningful success if it improves these measures on the problem for which it was designed, even when a general ReAct agent remains better elsewhere.

The essence of the architecture is progressive stabilisation. Broad model interpretation is used to discover intent and handle novelty. Recurring coordination is moved into orchestration skills. Stable execution is assigned to code or database-aware components. Generated code is isolated and evaluated before it receives authority. Context is reduced to the smallest useful frame, with explicit escalation when the local frame is insufficient. The design goal is not maximal autonomy, but a practical increase in bounded competence.

The central entry point is currently known as RecursiveSkilledAgent and will be named MainAgent in newer versions. The newer name is clearer because the component is not itself a promise that every task should recurse through a hierarchy. It is the principal coordination boundary. MainAgent receives a request, binds it to a session, determines whether a named capability was requested, discovers an appropriate orchestration path when necessary, prepares the relevant context, and applies the review and model-access policies associated with the operation.

MainAgent provides one public surface while preserving distinct semantics below it. A caller may invoke an explicit orchestration skill, which behaves more like a stable service contract, or submit an open request that requires discovery. The explicit mode reduces routing uncertainty and is preferable for integrated applications. The open mode retains assistant-like flexibility and provides a fallback when no known capability matches. Generality therefore appears as controlled routing between known and exploratory regimes, not as the assumption that every invocation should begin with free planning.

The core reusable object at this layer is an orchestration skill. It describes how a bounded operation should be interpreted and coordinated, which executable families may be used, what context is required, what outputs are expected, and where review or escalation belongs. An orchestration skill may compose other orchestration skills, but recursion is a means rather than an identity. The important property is that the runtime can separate coordination from execution and can test them independently.

This separation is a trust boundary. The language model may propose a route or fill a semantic gap, while the harness decides which execution family is authorised, which credentials are exposed, which artefacts are returned to context, and which evidence closes the operation. A single JSON tool abstraction would conceal important differences between stable code, generated code, and structured-database work. MainAgent preserves a common interface

CSkills, or Code Skills, package stable executable behaviour. They are appropriate when repeated use has revealed a procedure that can be implemented, tested, versioned, and executed more reliably than a fresh language-model interpretation. A CSkill may still receive parameters extracted by a model, but its core operation is

CGSkills, or Code Generation Skills, are used when the operation requires code that cannot be fully fixed in advance. The model generates or adapts a program for the current task, and the harness treats that program as an untrusted artefact. It must run inside a constrained environment with explicit inputs, resource limits, dependency policy, tests, and result capture. CGSkills occupy an important middle

ground. They preserve more local generality than CSkills while avoiding the token cost and copying errors that arise when large structured transformations are performed entirely through conversational tool calls.

DBTable Skills specialise the runtime for relational and tabular data. They begin from an abstraction of the database schema: tables, fields, types, keys, relationships, permitted operations, and domain annotations. From this representation, the system can create the capability to formulate queries, connect related tables, aggregate data, produce reports, and explain how a result follows from the schema and selected records. The abstraction is intended to reduce repeated schema discovery and to constrain generated operations. A database engine performs the exact relational work; the model interprets the user's intent and the meaning of the results.

The three families are intentionally distinct because they make different promises. CSkills promise stable implementation. CGSkills promise adaptive implementation under isolation and verification. DBTable Skills promise schema-aware access to structured data under explicit permissions and query semantics. An orchestration skill can combine them, for example using a DBTable Skill to retrieve evidence, a CSkill to compute a regulated metric, and a CGSkill to generate a one-off visualisation. The harness can then apply family-specific logging, context reduction, review, and security rather than one undifferentiated tool policy.

Context is the recurring research problem in the agent capability infrastructure. The architecture attempts to give an orchestration skill the local information it needs instead of replaying a global conversation and every available capability. This can reduce cost, improve tool selection, and protect unrelated data. It can also remove the nuance that explains why a request is exceptional. The important design is therefore not aggressive compression alone, but a layered context policy: a small default frame, named references to durable artefacts, provenance for retrieved knowledge, and an explicit path for requesting broader context when local confidence is insufficient.

Sessions provide the operational container for this policy. A session can hold request history, selected context, intermediate artefacts, skill decisions, model usage, errors, approvals, and review outcomes. It should not be treated as unlimited conversational memory. Different elements require different retention, access, and validity rules. A database result may become stale; a user preference may require consent; generated code may need quarantine; a review decision may need long-term audit. The session is valuable when it makes these distinctions visible and prevents state from leaking between unrelated agents or users.

Reusable agent skills are useful when they package procedures, instructions, code, and resources behind a stable responsibility. The important distinction for research is whether such a skill merely gives a general agent advice or whether it behaves as a governed capability with known inputs, outputs, effects, and evidence. Both forms will probably coexist. Simple procedural guidance is often enough for low-risk tasks; consequential scientific transitions benefit from stronger contracts and independent evaluation.

Soul Proxy, also described as Soul Gateway, is conceived as an Agent the deployment and isolation infrastructure placed between agent runtimes and model providers. Its role is to turn model access into a governed service rather than a collection of direct API calls. It can monitor requests and responses, enforce model and data policies, apply guardrails, route among providers or local models, constrain budgets, support caching, record usage, and mediate what information may leave a trust domain. Centralising this boundary also makes it possible to change model infrastructure without rewriting every orchestration skill.

A governed agent service should expose familiar integration surfaces so that existing applications can invoke it without adopting the entire research environment. Compatibility, however, must not erase policy. Identity, tenant boundaries, quotas, trace correlation, permissions, and data-handling rules should remain explicit even when the external interface looks conventional. The infrastructure earns trust by preserving these constraints underneath changing application protocols.

The gateway is also a natural point for context protection and observability. It can redact or tokenise sensitive fields, apply compression policies, reject requests that exceed an authority boundary, attach

provenance, and connect model calls to session-level traces. It cannot determine the full meaning of every request and should not become a single opaque policy oracle. Local

the agent capability infrastructure should be evaluated through task-specific benchmark packages derived from the applications it intends to support. Each package should include representative requests, difficult edge cases, hidden perturbations, unacceptable outcomes, expected evidence, cost and latency budgets, and a ReAct-style baseline using comparable models and tools. The objective is not to prove that the architecture produces a generally superior agent. It is to determine whether a particular orchestration and skill configuration earns its additional structure.

The most informative experiments are ablations. Compare global context with local context plus escalation. Compare a broad tool catalogue with skill-scoped exposure. Replace a CSkill with model-mediated execution to measure the value of stabilisation. Compare a CGSkill with repeated tool calls to examine token traffic and copying error. Compare a DBTable Skill with open SQL generation to measure schema errors, policy violations, reporting quality, and explanation. These experiments can reveal whether improvements come from the model, the context policy, the executable family, or the reviewer.

The expected development path is progressive. A new task may begin as an instrumented ReAct prototype. Repeated traces reveal the stable coordination pattern and common failures. That pattern becomes an orchestration skill. Stable operations become CSkills; variable transformations may temporarily become CGSkills; structured-data work becomes a DBTable Skill. The change is promoted only when local evaluation shows a useful improvement and when the context boundary does not erase critical nuance. In this sense the harness grows by converting experience into explicit structure.

the agent capability infrastructure therefore represents a disciplined engineering thesis: use language models where interpretation is genuinely valuable, but do not require them to rediscover every recurring procedure. Preserve ReAct as an open path rather than as the universal control law. Build narrow competence where evidence justifies it. Make context selection, execution semantics, model access, and review visible enough to improve. the experimental meta-rational execution approach extends this thesis by asking whether the executable knowledge itself can become the main substrate of a multi-agent environment and can be refactored through controlled periods of self-research.

A taxonomy of agents is useful only if it helps composition, permissions and evaluation. The platform should avoid defining one microservice for every human-sounding role. Several roles may be implemented by the same generic agent with different skills, tools and capability scopes. Conversely, one critical role may require several independent implementations to reduce correlated failure. The taxonomy below therefore describes responsibilities. It is closer to an interface catalogue than to an organisational chart.

The first family is interaction and coordination. The General Research Copilot is the primary conversational entry point. It explains project state, helps formulate tasks, edits artefacts and delegates to specialised capabilities. A Meeting and Collaboration Agent extracts candidate decisions, unresolved questions and actions from permitted WebMeet sessions. A Research Planning Agent decomposes goals into tasks and candidate branches. A Project Briefing Agent compresses background activity into a scientifically meaningful summary. These agents interact frequently with humans and therefore require strong grounding in current project state and clear distinction between suggestion and commitment.

The second family is exploration and knowledge. A Literature Scout retrieves and monitors relevant publications, patents or databases. A Claim and Knowledge Curator maps retrieved sources into candidate claims, concepts and evidence relationships while preserving source spans. A Hypothesis Explorer proposes testable explanations, variations and connections. A Diversity or Challenge Agent deliberately searches for alternative framings and counterexamples. These roles are generative and should have low authority: they expand the search space but do not decide what the project accepts.

The third family is experiment and engineering. An Experiment Designer converts a hypothesis into a protocol with variables, controls, measurements and acceptance criteria. An Execution Agent or Research Engineer modifies code, prepares environments and operates approved tools. A Data

Analysis Agent performs statistical or computational analysis. A Simulation Agent controls domain simulators. A Reproduction Agent attempts to recreate an existing result in a clean environment. In many domains these agents will be the most immediately useful because their work can be evaluated against concrete outputs. Their permissions must nevertheless be bounded because they may execute code, consume resources or access sensitive data.

The fourth family is scientific evaluation. An Evidence Reviewer checks source support and scope. A Reproducibility Evaluator determines whether an artefact can be recreated and whether the result matches the claimed output. A Methodology or Statistics Reviewer checks assumptions, experimental design and analysis. A Contradiction Agent searches for counterevidence and incompatible claims. A Formal or Constraint Verifier invokes theorem provers, solvers, type systems or domain rules when available. A Novelty Assessment Agent compares a candidate contribution with prior art. A Significance or Impact Reviewer can estimate whether a result materially changes a decision or opens a useful research branch, but such judgements are highly domain-dependent and should remain advisory.

The fifth family is trust and governance. A Privacy Steward classifies proposed data flows and identifies policy-relevant risks. A Security and Tool-Risk Agent analyses untrusted repositories, prompts, external services and potentially dangerous actions, while deterministic sandboxing and access controls enforce the actual boundary. A Provenance Auditor checks whether important outputs have traceable inputs and methods. A Policy Explanation Agent translates machine-readable decisions into researcher-facing language. A Human-Authority Broker routes actions to the appropriate approver when policy cannot resolve them automatically. The word “agent” should not obscure the fact that final enforcement belongs to conventional security mechanisms.

The sixth family is infrastructure and operations. An Execution Supervisor observes jobs, failures, retries and environment health. A Resource Scheduler allocates compute and tool budgets. A Recovery Agent reconstructs affected work after a failure or incident. An Artifact Curator checks that project packages contain required metadata, code, data references and output links. A State Consistency Agent detects conflicting or stale project objects. Some of these roles may be implemented entirely without LLMs. They appear in the taxonomy because they control the environment and because their interventions influence research progress.

The seventh family is memory and learning. A Project Memory Curator decides which operational events deserve durable representation. A Procedure Distillation Agent proposes reusable workflows or skills from successful runs. A Regression Agent tests learned procedures against held-out tasks and new model versions before they are promoted. A Benchmark Contamination Auditor checks whether a memory or tool has exposed information that should be unavailable in an evaluation. This family is important because an adaptive research environment can easily become less trustworthy if “learning” means retaining everything that seemed useful once.

Agents should be selected by capability rather than by fixed name. A task such as “reproduce this result and assess whether the difference from prior work is substantive” may require literature retrieval, environment construction, execution, statistics, novelty analysis and a final synthesis. The orchestrator can assemble a temporary harness from available capabilities. The harness itself becomes a provenance object: which agent versions were used, which models, tools and prompts, what communication topology, what budgets and what evaluators. This makes it possible to study whether performance comes from the base model, the scaffold, the toolchain or the division of labour.

The capability manifest is therefore a key interface. It should describe what the agent can consume and produce, what permissions it requests, which tools it needs, whether it has side effects, what data classes it can process, what cost limits apply, and how its outputs should be evaluated. A natural-language description can help discovery, but the operational fields should be machine-readable. An agent that asks for broader permissions than its role requires should not be deployable simply because its prompt says that it is trustworthy.

The taxonomy also helps define privilege tiers. Exploration agents normally read and propose. Evaluation agents read broadly and attach assessments but should not silently rewrite the target object. Execution agents may have side effects inside bounded environments. Control-plane agents can

allocate resources, block actions or change operational state and therefore require particularly strong evaluation. External-action agents, if introduced, can submit jobs to laboratories, publish artefacts or contact third parties and should sit behind explicit approval policies. The privilege model should follow capability and reversibility, not the perceived intelligence of the agent.

Agents can also be instantiated temporarily. The platform does not need twenty permanently running personalities. A generic model-backed worker can be configured at runtime with a role contract, tools, state slice and evaluation policy. For high-stakes tasks, multiple workers can be created with deliberately different models or strategies. This reduces architectural clutter while preserving specialisation. Persistent named agents are justified only when they need durable responsibility or domain memory.

Human escalation should be a normal output type. An agent may return not only a result but also “cannot decide because required corpus is inaccessible”, “two policies conflict”, “reproduction failure is ambiguous”, or “the expected experiment cost exceeds my authority”. Escalations should include a compact explanation and the minimal decision required from the user. This converts uncertainty into structured workflow rather than forcing an agent to invent a confident answer.

Agent trust should be earned through a lifecycle. A new agent first runs on benchmark and project-specific fixtures. It can then operate in shadow mode, observing real tasks and producing recommendations without committing changes. If performance is adequate, it receives bounded production capabilities. Regression tests continue after model, prompt, tool or dependency changes. Authority can be reduced automatically when evaluations fail. This approach is more defensible than assigning a permanent label such as “trusted agent”. Trust is evidence about a versioned capability in a particular scope.

The system should evaluate controlling agents for false intervention as well as missed intervention. A privacy mechanism that blocks most legitimate work is not successful. A resource supervisor that terminates productive long-running experiments is not efficient. A provenance auditor that produces constant low-value warnings creates alert fatigue. Metrics therefore need both safety and utility: precision, recall, override rate, intervention cost, recovery quality and effect on scientific outcomes. Governance agents are part of the research instrument; systematic bias in their decisions can bias what research gets performed.

A further design principle is separation of proposer and evaluator where the consequence matters. The agent that designed an experiment should not be the only agent deciding that the experiment succeeded. The model that generated a novelty claim should not be the only model assessing prior art. Independence can be strengthened by different prompts, tools, models or deterministic checks. Perfect

Table 6. A research agent should be defined by a governed capability rather than by a personality or prompt.

Term	Meaning in the research environment
Capability	A bounded scientific or operational responsibility with inputs, outputs, permissions, evidence expectations and tests.
Harness	The runtime layer that controls context, tools, state, budgets, permissions, tracing and publication around a model.
Sandbox	An execution boundary that limits the filesystem, network, process and secret access of untrusted or adaptive components.
Qualification	Evidence that a capability works within a declared scope and can be promoted into a trusted generation.
Authority	The right to change scientific or operational state; confidence and fluency do not create authority.

The long-term objective is not to construct the largest possible society of agents. It is to make delegation cheap enough to use and explicit enough to govern. A researcher should be able to grant a capability for one task without granting the model permanent authority over the project. A laboratory

should be able to run several specialised agents without confusing their independent outputs with independent evidence. An institution should be able to replace an agent implementation while retaining the specification, tests, permissions and history that define the role.

This architecture also creates a path for gradual automation. Recurrent tasks can move from open-ended model calls toward reviewed skills, deterministic tools or symbolic circuits as the organisation learns what the task actually requires. Creativity remains available at the edge. Authority becomes progressively encoded in the stable part of the system. That is a more realistic path to trustworthy automation than assuming that one future model will simultaneously understand the science, plan the work, execute it, validate itself and govern its own permissions.

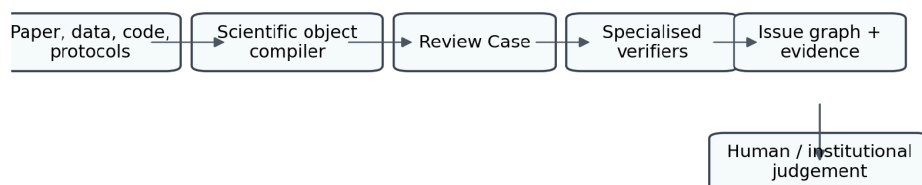
CHAPTER 6

Evaluation as a Continuous Layer of Research

Peer review is an unusually good proving ground for a research environment because it forces the architecture to distinguish what can be checked from what must still be judged. A manuscript contains claims, methods, evidence, comparisons, interpretations and rhetorical choices. Some questions are highly executable: whether reported numbers are internally consistent, whether a cited source contains the attributed result, whether code reproduces a figure, whether a statistical procedure matches the description, whether an artifact exists. Other questions concern novelty, explanatory adequacy, importance or disciplinary taste. A useful machine reviewer must make this boundary clearer rather than erase it.

The central design choice is to stop treating review text as the primary computational object. The primary object should be a Review Case that persists claims, artefacts, checks, issues, evidence, uncertainty and revision history. Prose becomes one view over this object. That single inversion changes evaluation: the system can now be tested on whether it found a known defect, whether the defect is linked to the correct evidence, whether the check was reproducible and whether the reviewer understood the scope of the machine conclusion.

Figure 4. Automated review should produce evidence before it produces judgement.



The objective is not to argue that human reviewers should disappear. Current publisher policies appropriately retain human accountability and impose strict confidentiality requirements on AI use in peer review. Nature Portfolio, for example, allows assistive use under risk-based conditions but explicitly requires reviewer accountability and prohibits uploading confidential manuscripts to unsecured or public AI systems. A credible research programme must therefore distinguish assistance, delegated verification, independent machine review and editorial decision automation rather than treating them as one deployment mode.

The thesis developed here is narrower and, for that reason, more defensible: peer review can be decomposed into a set of evidence-producing evaluation procedures, many of which can be automated or machine-assisted; integrating those procedures around a persistent symbolic review state may produce a more reliable system than a monolithic LLM reviewer. Whether that thesis holds across domains is not assumed. It is the central question of the proposed research programme.

Peer review performs at least three functions that should be separated technically. The first is scientific criticism: does the work support what it claims? The second is scientific positioning: what, if anything, is new, useful, general or consequential relative to the existing state of knowledge? The third is editorial selection: should this particular venue publish or accept this work under its current scope, capacity, standards and portfolio priorities? Human review reports routinely mix all three. An automated architecture should not.

Scientific criticism is closest to an engineering verification problem. It can ask whether the experimental design matches the stated hypothesis, whether the statistical test is applicable, whether a theorem's assumptions match its use, whether a reported number can be reproduced from the supplied data, whether the claimed causal interpretation exceeds what the design supports, or whether a

benchmark comparison omits a necessary baseline. These checks can often be attached to concrete evidence and, in favourable cases, to executable tests.

Scientific positioning is harder. Novelty is relational rather than intrinsic: a contribution is novel only relative to a defined corpus, time boundary, concept representation and level of granularity. Relevance is conditional on a research problem and community. Significance is partly prospective. A result that looks minor may later unlock a field, while a seemingly dramatic result may not replicate. Automated systems can search prior art, identify structural differences, estimate consequence paths and detect changed constraints, but they should represent uncertainty rather than transform these operations into an oracle for future impact.

This decomposition leads to an immediate design principle: a review system should be evaluated on the correctness of its scientific operations before it is evaluated on whether it predicts the venue's final decision. A system that agrees with a historically noisy decision while missing a fatal flaw has learned the wrong target.

The common review dimensions - soundness, novelty, significance and clarity - are useful headings, but they are too coarse for automation. Each hides several distinct questions with different evidence requirements. "Soundness" can refer to internal logical consistency, appropriateness of methods, implementation correctness, data validity, statistical inference or robustness. "Novelty" can mean a new object, a new relation, a new mechanism, a new proof, a new empirical regime, a new scale, a new combination, or a meaningful improvement on a frontier. "Clarity" can mean merely readable prose, but it can also mean whether the paper reports enough information to evaluate or reproduce the work.

No automated system will be equally strong on all these dimensions. PRISM's 2026 evaluation supports precisely this point: different automated reviewers show distinct specialization profiles, and no single tested system consistently reproduces the balanced human baseline across all assessed dimensions (Loc et al., 2026). This suggests that architecture matters. A collection of explicit evaluators with different evidence sources may be more defensible than a single model asked to optimise an aggregate notion of "review quality".

Human reviews are not a clean ground truth. Reviewers disagree because expertise differs, because papers are ambiguous, because some criteria are value-laden, and because review itself is noisy. An evaluation regime that treats every human criticism as true will reward imitation of human errors. Conversely, a machine can identify a real issue absent from the historical reviews.

The useful unit of evaluation is therefore not "does the machine say what the reviewer said?" but, where possible, does the machine identify a verifiable issue, attach it to the right manuscript evidence, and calibrate the severity and uncertainty appropriately? Human reviews remain indispensable as data, but they should be augmented with expert adjudication, reproducibility outcomes, counterfactual defects, executable checks and time-bounded prior-art evidence.

This distinction also changes the role of LLM-as-judge evaluation. Model-based evaluators are useful for scaling exploratory work, especially on style, coverage and constructiveness. But if the same class of models generates and judges the reviews, correlated blind spots can create an illusion of progress. The strongest 2026 benchmark work is moving toward multidimensional, grounded and counterfactual evaluation rather than a single unconstrained LLM score. That trajectory should be treated as a design requirement rather than an incidental benchmarking preference.

Debates about "AI peer review" are often unproductive because they collapse very different deployment modes. A spelling assistant, a statistical checker, an agent that reproduces experiments and a system that issues an accept/reject recommendation do not create the same scientific or governance risk. A research programme should make the delegation boundary explicit.

The proposed programme concentrates on delegated verification, independent machine Review Cases and meta-review assistance. Autonomous editorial decision-making is intentionally not the core objective. This is not a philosophical prohibition. It is a prioritisation based on where current evidence is strongest and where scientific evaluation can be made most falsifiable.

The history of peer-review automation already contains mature examples of a useful pattern: automate a well-defined check, surface evidence, and leave interpretation to a responsible reviewer or editor. Crossref Similarity Check identifies textual similarity but explicitly warns against converting a similarity score into an automatic plagiarism verdict. The STM Integrity Hub combines multiple screening tools for suspected paper-mill and integrity signals. *statcheck* can identify inconsistencies in the reporting of null-hypothesis significance tests and has been studied as an intervention in editorial workflows. Reviewer-assignment systems already operationalise expertise matching, conflicts and bids at scale.

These systems do not solve peer review, but they reveal a robust architectural principle. Their value comes partly from refusing to solve more than they can validate. A general AI reviewer should learn from that restraint. Instead of producing one opaque judgement, it should orchestrate narrow verifiers and preserve their evidence boundaries.

Some review tasks are candidates for assistance but poor candidates for hard automated verdicts. Long-term scientific significance is one. A system can estimate whether a result removes a known bottleneck, changes a Pareto frontier, unifies separate explanations or opens a previously inaccessible regime. These are useful indicators of possible relevance. They are not reliable forecasts of future citations, adoption or societal impact.

Ethics is another. Automated systems can flag missing consent statements, privacy risks, known dual-use categories or inconsistent disclosures. They cannot infer every context-dependent ethical harm from the manuscript, and a false sense of completeness could be dangerous. Similarly, theory evaluation can sometimes use proof assistants, SMT solvers or symbolic algebra, but only when the formal object accurately represents the theorem being claimed. The translation from natural-language mathematics to a formal obligation is itself an error-prone step.

A review is normally visible as prose, so it is natural to train AI systems to generate prose. That optimisation target is incomplete. Review text combines evidence, interpretation, rhetorical compression and recommendation into a single surface form. Two reviews can use different words for the same issue; one polished paragraph can contain three unsupported accusations; a correct technical concern can be buried under stylistic commentary; and a model can produce detailed criticism without having performed the calculations that would justify it.

PRAIB makes this distinction concrete by asking whether LLMs engage with papers in the same way as human reviewers rather than merely generating review-looking text. Its large-scale study reports systematic differences in calibration and in the detection of atomic weaknesses (Zurawicki et al., 2026). The counterfactual faulty-reasoning study is even more direct: when defects were deliberately inserted into research logic, automatic reviewers did not reliably change their outputs (Dycke and Gurevych, 2026). A system can therefore look reviewer-like while being insensitive to the causal variable that should have changed the review.

The architecture proposed later in this book takes the opposite approach. It stores the evaluation as a graph of objects and verification events, then renders prose from that state. If a major issue cannot be linked to a manuscript span, a method component, external evidence or an explicit verification run, it should be marked as an unsupported hypothesis rather than silently promoted to a reviewer claim. The Review Case also stores unresolved questions, confidence and provenance. It is not necessarily shown directly to authors. It is the internal scientific state from which different views can be produced: a reviewer report, a reproducibility report, a novelty comparison, a meta-review dashboard, a rebuttal checklist or an editor-facing risk summary.

This structure does not guarantee correctness. A graph can encode a wrong interpretation just as prose can. Its advantage is epistemic inspectability. The system exposes where the uncertainty enters: extraction, retrieval, rule applicability, tool execution, evidence interpretation or synthesis. That makes ablation and correction possible.

The strongest argument for a neuro-symbolic design is not that symbolic AI is intrinsically superior to neural models. It is that peer review contains persistent state and explicit constraints that are awkward to maintain reliably in unconstrained generation. Claims have provenance. Statistics have

preconditions. An issue can invalidate several downstream claims. Two criticisms may be duplicates. A rebuttal can resolve one premise without resolving another. A later manuscript version can change the evidence but not the research question. A verifier may apply only to randomised experiments or only when sample-level data are available. These relations are naturally represented as typed symbolic objects.

LLMs are nevertheless difficult to replace as the semantic interface. Scientific manuscripts are heterogeneous and often underspecified. The same methodological idea can be described with different terminology across fields. A purely symbolic parser would require prohibitive manual engineering. The design inference is therefore to use language models to propose structured

The rest of the document therefore does not present an accomplished technology. It presents an architecture whose value should be judged by difficult tests: does it find more real critical flaws at the same false-accusation rate; can it show why; does it remain stable under irrelevant perturbations; can it reproduce or falsify parts of the work; can specialists inspect the evidence trail; and does it reduce human workload without making human judgement more complacent?

The central invariant of the proposed system is simple: every major machine criticism should be representable as an evidence-bearing issue object before it becomes prose. The issue can be wrong, but it must be inspectably wrong. It should identify what it concerns, why the system believes it matters, which sources or verification runs support it, how confident the system is, and what would resolve it.

This changes the role of generation. A language model can still write the final reviewer report because natural language is the right interface for authors and reviewers. But the report is generated from a structured Review Case and can be regenerated after evidence changes. If a sentence in the report cannot be traced to the case, it should be considered an unsupported generation defect.

This requirement is stricter than ordinary retrieval-grounded generation. A citation attached to a paragraph does not prove that the cited source supports the specific criticism. The Review Case should encode proposition-level relations where practical: source S contradicts claim C under condition K; experiment E fails to distinguish explanations A and B; reported statistic R is inconsistent with values V under test T; artifact execution X produced output O rather than reported output Q.

The most important architectural choice is that the language model does not own persistent scientific state. It proposes interpretations and actions into typed interfaces. The state can therefore survive model replacement, be audited after a policy change, and support multiple models performing independent checks.

A claim representation should distinguish at least: proposition, scope, modality, conditions, population or domain, comparator, claimed direction or effect, and whether the claim is descriptive, causal, theoretical, predictive or normative. A method representation should include design class, input data, procedure, assumptions, controls, baselines, parameters and outputs. An evidence representation should link reported observations to methods and claims.

A useful research tactic is dual extraction: one model produces the candidate structure, while another or a symbolic validator checks invariants such as missing source spans, inconsistent units, impossible cross-references, unbound variables or claim/evidence relations that lack a path. For high-severity findings, the interface can require confirmation against the exact manuscript excerpt before issue synthesis.

Scientific venues already publish reviewer guidelines, reporting checklists and domain standards. The proposed system converts these into versioned Review Protocols. A protocol is not a numerical scoring rubric alone. It describes what questions are applicable, what evidence is needed, which verifier can answer them, what constitutes a warning versus a blocking issue, and when the system should abstain.

A protocol can inherit from other protocols. A universal empirical protocol may require claim-method alignment and limitation reporting. A machine-learning benchmark protocol adds leakage, baseline, evaluation and compute checks. A clinical-trial profile can incorporate CONSORT-style obligations. A

software-artifact profile can require environment, dependency and execution checks. A specific venue can then add scope and scoring rules without redefining scientific validity.

Rigid checklists produce false positives when applied outside their domain. A protocol should therefore support conditions such as: if the paper makes a causal claim from observational data, inspect identification assumptions; if multiple hypotheses are tested, inspect multiplicity handling; if software artifacts are available, attempt execution; if the claimed contribution is a new benchmark result, inspect baseline comparability and leakage; if no individual-level data are available, skip tests that require them and record the limitation.

Applicability can itself be uncertain. The system may detect that a paper appears to make a causal claim but uses ambiguous language. Instead of silently deciding, it can create a reviewer question: “The conclusion uses causal terminology, while the described design appears observational. Is the intended claim causal or associational?” This is often more useful than forcing a brittle automated verdict.

It would be tempting to hand-author a large library of protocols and declare the problem solved. That would not scale. A more realistic programme combines three methods. First, encode stable rules derived from explicit standards and high-consensus methodological knowledge. Second, mine recurrent reviewer concerns and map them into candidate protocol rules, followed by expert validation. Third, allow domain specialists and coding agents to author and test new protocol modules against regression suites.

The contribution verifier reconstructs the paper’s claimed contribution, retrieves prior work, compares the nearest scientific relations and produces a contribution map. It should explicitly separate retrieval confidence from novelty confidence. If retrieval coverage is weak, the correct output is not “novel” but “no close prior result was found under this search process”.

A stronger implementation combines lexical search, dense semantic retrieval, citation graph traversal and structured graph queries. Vector-symbolic or hyperdimensional representations could be explored as an optional structural-retrieval sidecar for compositional fingerprints, but this is speculative and should survive only if it improves recall-cost trade-offs over dense and graph-native baselines. No particular representation technology is architecturally required.

A useful technique is counterfactual questioning. For each important claim, ask what alternative explanation could also produce the reported evidence and which experiment would distinguish them. This does not automatically invalidate the claim. It creates a test of discriminative adequacy. The strongest output is not “there may be confounding” but a specific alternative mechanism plus the missing observation that would separate it from the authors’ explanation.

For theoretical work, parts of this module may call theorem provers, symbolic algebra or SMT solvers. The system must preserve the translation boundary: a machine-checked formalisation verifies the formal object, not automatically the authors’ prose. A human-readable mapping between theorem statement, formal assumptions and checked proof obligation remains necessary.

This module should use deterministic or specialised statistical code whenever feasible rather than asking an LLM to perform arithmetic in natural language. It can check reported p-values and test statistics, confidence-interval consistency, unit conversions, sample counts, impossible percentages, multiple-comparison handling, effect-size interpretation, baseline comparability, uncertainty reporting and power assumptions.

The artifact verifier constructs an execution plan from documentation, resolves dependencies inside an isolated environment, executes declared workflows and compares produced artifacts with reported outputs. It should record environment hashes, tool versions, commands, runtime, deviations and failure causes. When exact reproduction is impossible, it can still evaluate whether the instructions are complete enough to reconstruct the workflow.

A future version can go beyond reproduction to targeted stress tests. For example, after reproducing a machine-learning result, the system may rerun a small set of seeds, perturb a preprocessing choice, swap a baseline configuration or remove a suspicious feature. These experiments can reveal

brittleness. They are more expensive and should be chosen by expected information gain rather than executed indiscriminately.

Citation verification has at least three levels. The first checks existence and bibliographic correctness. The second checks whether the cited source actually supports the proposition. The third checks whether important contradictory or predecessor work appears to be missing. LLMs can help align claims to source passages, but the system should retain the source text and relation, not just a generated summary.

A criticism such as “the cited paper does not support this claim” is serious and can itself be wrong. The issue should therefore include the cited proposition, source passage, machine interpretation and confidence. For paywalled or unavailable sources, the system should not pretend to have verified content it could not access.

Reporting checks are well suited to automation because many requirements are explicit. The system can inspect whether datasets, model versions, random seeds, inclusion criteria, units, ethical approvals, preregistration identifiers, software versions, limitations or resource identifiers are reported when required by a protocol.

Integrity modules consume outputs from similarity systems, image forensics, duplicate-submission checks, citation-anomaly detectors, data-consistency tests and paper-mill signal detectors. The symbolic layer records these as signals with provenance, not allegations. Escalation policy can require independent corroboration or human research-integrity review.

Ethics automation should focus on explicit conditions and omissions: missing consent where human-subject research appears to require it; identifiable data inconsistent with stated privacy handling; absent approval identifiers where venue policy requires them; dual-use categories; undeclared conflicts; safety evaluations omitted for high-risk systems. The module should route uncertain or high-impact cases to human specialists.

Each issue has a type, affected objects, severity, confidence, evidence, counterevidence, status and remediation. Edges can represent causes, duplicates, depends-on, supersedes, resolves and contradicts. Meta-review becomes easier because disagreements are attached to the same issue object rather than scattered across paragraphs.

A proposed severity scale can be semantic rather than numerical: critical claim-invalidating issue; major issue requiring new evidence or substantial claim revision; moderate limitation affecting scope; minor reporting/clarity issue. Protocols can map specific verified conditions into these categories, while uncertain model-generated concerns remain hypotheses until corroborated.

Confidence should not be computed solely from the language model's self-reported probability. It can combine extraction confidence, verifier reliability on calibration benchmarks, evidence independence, retrieval coverage and agreement between genuinely different methods. Repeating the same prompt across several correlated models should not be counted as independent evidence.

Scientific review frequently contains irreducible disagreement. The system should preserve it. If the statistical verifier finds a likely problem but the domain-methods agent argues that a design feature makes the test inapplicable, the Review Case should show both interpretations and identify the condition that would decide between them.

Even a highly accurate review assistant can harm the process if it anchors human reviewers too early. If reviewers see a confident machine assessment before forming an independent view, they may converge on the machine's blind spots. Therefore human-AI evaluation must study workflow order, not only model accuracy.

A strong experimental design randomises when machine findings are shown. One group reviews independently before seeing the Review Case; another receives it from the start; a third sees only tool-verified findings initially and speculative model findings later. Outcomes can measure critical-issue recall, false accusations, time, confidence, diversity of concerns and changes in recommendations.

One of the strongest roles for the system may be question generation. Instead of guessing missing experimental details, it can produce a small set of high-information questions. An author response can then resolve uncertainty before reviewers spend time arguing over an ambiguity.

This suggests an information-theoretic scheduler: prioritise checks or questions expected to reduce uncertainty on high-severity issues. A cheap citation check that resolves a novelty concern may be preferable to an expensive reproduction run if both address the same decision-critical uncertainty.

This is compatible with current publisher policy and also scientifically useful. It creates data about which machine findings experts trust, where experts disagree, and which checks produce genuine added value. That data can improve the system without silently training it to mimic acceptance outcomes.

A general peer-review architecture cannot mean identical checks for every field. Generality should mean that the same execution model can host different scientific protocols. The universal kernel manages claims, evidence, provenance, issues, uncertainty, tool execution and review-state updates. Domain packs provide terminology, study-design ontologies, standards, specialised retrieval sources and verifiers.

The research programme should test at least three meaningfully different domains. A useful set would include: machine learning or software-intensive research, where artifacts and benchmark checks are accessible; biomedical or life-science research, where reporting, statistics, ethics and experimental controls are strong; and a mathematically or physically grounded field, where formal or constraint-based verification can be tested. The point is not to claim full coverage but to expose whether the architecture's reusable core survives methodological diversity.

A central risk is constructing a benchmark that rewards the same shortcuts the architecture is meant to avoid. BLEU, ROUGE and generic semantic similarity to human reviews are insufficient. Unconstrained LLM-as-judge scores are useful secondary diagnostics but should not determine the main ranking. The benchmark should contain several complementary evidence types.

Existing resources provide starting points rather than a complete benchmark. FLAWS and the faulty-reasoning counterfactual framework support controlled error detection. ICLR, ICML and NeurIPS review corpora provide open peer-review material. ReScience C and software-artifact datasets support reproducibility. BadScientist provides adversarially unsound papers. Transparent journal peer-review corpora can broaden domain coverage. The programme should contribute a curated cross-domain benchmark with explicit licensing and confidentiality constraints.

Novelty and literature positioning require historical snapshots. For each target manuscript, the retriever should be restricted to literature available before a chosen cutoff, ideally the original submission date. Later citations, surveys and retrospective labels must be excluded from the evaluation input even if they are used for outcome analysis.

This is more difficult than setting a date filter. Search indexes can contain updated abstracts, later versions and citation links that leak the future. The benchmark should document the residual risk and use frozen corpora where feasible. A smaller, clean time-machine benchmark is preferable to a larger dataset with invisible hindsight contamination.

A complex system earns its complexity only against strong baselines. At minimum the programme should compare: a frontier LLM with a high-quality review prompt; a retrieval-augmented reviewer; a multi-stage tool-using agent; a specialised/fine-tuned reviewer where available; representative 2025-2026 research systems that can be reproduced; and human reviewers or expert panels on selected tasks.

Ablations should remove one architectural element at a time: structured CMEG; review protocols; live prior-art search; executable tools; issue graph; independent verification; domain pack; adversarial sandbox; and different synthesis policies. For multi-agent variants, the programme should compare one model with multiple roles against genuinely heterogeneous models/tools to quantify correlated error.

Negative controls are important because reviewer systems are rewarded for finding problems and may learn to overcriticise. Clean or deliberately repaired paper pairs should test whether issue generation decreases when the flaw is removed. Style-only perturbations should not change scientific findings. Randomly shuffled author metadata should not alter validity scores. Harmless code comments should not influence scientific conclusions. An irrelevant retrieved paper should not become evidence merely because it shares vocabulary.

Table 7. Peer review is a portfolio of epistemic functions, not one text-generation task.

Term	Meaning in the research environment
Prior art	Search and compare contribution claims against a documented literature frontier without treating missing retrieval as proof of novelty.
Evidence	Check that cited sources, data, figures and computations actually support the target claim.
Method	Apply domain- and method-specific checks rather than relying on generic review prose.
Reproducibility	Re-execute feasible artefacts and preserve the difference between reproduction and independent replication.
Integrity	Use multiple bounded signals for citation, manipulation, adversarial input, ethics, privacy and artifact integrity.
Judgement	Keep significance, editorial policy and contested interpretation visible as human or institutionally governed decisions.

Continuous peer review follows naturally once the Review Case exists. A revision need not restart the entire evaluation. A changed dataset can invalidate dependent claims. A retraction can reopen citation support. A new baseline can trigger a targeted comparison. A post-publication reproduction can add a new issue without rewriting history. The review becomes a maintained scientific object rather than a one-time document.

This is a larger claim than automating referee labour, and it is more defensible. The research environment can make parts of scientific criticism continuously executable while preserving human and institutional judgement. If that architecture improves flaw detection, reproducibility and contestability, it has value even if no journal ever allows an AI system to issue an autonomous accept or reject decision.

The most informative benchmark for this architecture is not a corpus of final reviewer scores. It is a corpus in which the location and nature of defects are controlled. A paper can be mutated so a citation no longer supports a claim, a sample count changes in one table, an exclusion criterion becomes inconsistent, a baseline disappears, a causal statement exceeds the design, or an artefact cannot reproduce the reported figure. The system can then be evaluated separately on detection, evidence localisation, severity, explanation and appropriate abstention. This is harder to game than similarity to historical review prose.

Review should also be longitudinal. A revision changes some parts of the scientific object and leaves others untouched. The Review Case should know which checks depend on the changed spans, datasets, code or assumptions and rerun only those checks. A rebuttal can attach evidence to an issue rather than create another unstructured document. If an editor accepts a justified exception, the decision becomes part of the case. The result is a maintained evaluation state rather than a pile of reviewer versions. This could make peer review faster while increasing rather than reducing accountability.

Post-publication correction is a natural extension. Retractions, errata, corrected datasets and later replication failures can invalidate support relationships after publication. A persistent Review Case can reopen affected issues and propagate warnings to dependent claims without rewriting history. The original publication state remains recoverable; the current state reflects the correction. This is technically similar to dependency invalidation in software systems but epistemically more delicate because a changed source may weaken rather than simply break a scientific claim.

Human studies are essential because automation can distort reviewer behaviour even when the machine checks are accurate. Reviewers may over-trust confident issue summaries, spend less effort on dimensions the system does not cover, or converge prematurely on machine-generated framings. Interfaces should therefore expose evidence and the scope of each verifier, not merely recommendations. Experiments should measure correction of machine errors, diversity of independent judgement, review time and the fate of disagreements. A review assistant that raises apparent agreement while reducing critical independence would be a failure.

The strongest long-term use may be before submission. Researchers can run the same evidence-centred review protocols continuously while the project is still changing. Mechanical and reproducibility defects can be caught close to the moment they are introduced. Claims can accumulate evidence links before manuscript writing. Novelty and citation checks can inform positioning without becoming acceptance predictors. In this form automated peer review is not primarily automation of gatekeeping. It becomes quality infrastructure inside the research environment, with journals and external reviewers later inspecting a richer scientific object rather than starting from a static PDF.

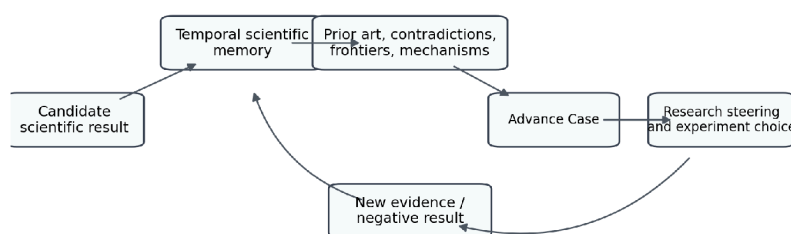
CHAPTER 7

Choosing Direction When Results Become Cheap

When candidate generation becomes cheap, the question “is this new?” changes from an occasional literature problem into a continuous control problem. A research agent can produce many plausible hypotheses, methods and combinations. Most are not worth expensive validation. Some are rediscoveries. Some differ only in terminology. Some move an empirical frontier without introducing a new mechanism. Some are genuinely surprising but wrong. A useful novelty system must therefore reason about scientific state change rather than reward semantic distance.

We use the name NOVA for this research direction, but the name matters less than the contract. The system should receive a result capsule that describes the candidate contribution and its evidence. It should compare that object against a temporal scientific memory, preserve the corpus and time boundary, analyse prior art, contradictions, mechanisms and frontiers, and return an Advance Case rather than a scalar novelty score. The purpose is to support scientific steering, not to manufacture authority.

Figure 5. Novelty intelligence closes the loop only when it changes what the research system does next.



Much of scientific organisation assumes that producing a serious candidate result is expensive. A researcher must acquire knowledge, frame a question, implement or perform an experiment, analyse the outcome, compare it with prior work and persuade other experts that the result matters. Scarcity enters at almost every step. Consequently, filtering is partly performed by the cost of producing the candidate itself. This is not a desirable filter in any normative sense, but it has historically limited the volume of material demanding expert attention.

Agentic research systems weaken that implicit filter. The AI Scientist-v2 can autonomously formulate hypotheses, design and execute machine-learning experiments and produce manuscripts; Co-Scientist uses multi-agent generation, critique and tournament-style evolution for hypothesis development; Robin connects literature search and data analysis in experimental biology; Kosmos runs long multi-cycle investigations with a structured world model; AutoScientists coordinates self-organising teams around a shared experimental state. These systems differ considerably in domain, autonomy and evidential strength, but together they make one point difficult to dismiss: generating plausible research trajectories can be automated to a degree that was exceptional only a short time ago.

It does not follow that science will be “fully automated”. It does follow that the number of machine-produced hypotheses, experimental branches and local findings can rise rapidly in domains where evaluation is computational or where laboratory automation exists. Under that condition, ranking and adjudication are no longer secondary conveniences. They are part of the research loop itself: which anomaly deserves replication, which apparent improvement is already known under another name, which negative result closes an important branch, and which small effect changes a heavily used assumption?

An automatic evaluator can easily be built if the target is defined badly. Textual dissimilarity from the nearest papers is measurable. So is unusual co-occurrence of concepts, embedding distance, citation rarity or an LLM score. None of these quantities is equivalent to a scientific advance. A new parameter setting can be textually distinct yet scientifically irrelevant. A replication may contain little conceptual

novelty but materially alter confidence in an important claim. A negative result may be valuable because it rules out a widely assumed mechanism. An engineering result can matter because it changes a Pareto frontier even when its conceptual vocabulary is familiar.

The distinction can be stated more rigorously. Novelty is a relation between a candidate and a time-bounded knowledge state. Evidence is a relation between observations and claims. Significance is a relation between a claim and the consequences it would have for other claims, predictions, constraints or decisions. Potential impact is a still broader and less observable relation between a scientific change and future downstream use. Collapsing these relations into a single “interestingness” score obscures the very structure an evaluation system should expose.

This is why NOVA does not aim to predict which result will become famous or highly cited. Citation counts are field- and time-dependent, socially mediated and confounded by visibility. Even carefully designed novelty metrics face the problem that ground-truth novelty is not directly observable; a 2026 axiomatic benchmark explicitly avoids requiring novelty labels because citation and peer-review signals are confounded. The safer goal is narrower: detect and explain candidate changes to the scientific state that have measurable epistemic or operational consequences, then recommend verification proportional to those consequences.

The case for reuse does not rest on the claim that all sciences share one scoring function. They plainly do not. The weaker and more defensible claim is that many research domains manipulate recurring epistemic objects: claims, observations, methods, assumptions, comparisons, uncertainty, provenance, dependencies and tests. The admissible relations and evidence standards vary, but the need to state them explicitly does not.

This suggests an architecture analogous to a database engine with domain-specific schemas rather than a universal scientific theory. The common kernel should represent claims and evidence, maintain temporal state, resolve prior art, expose dependencies and record verification history. Domain packs

The phrase “high-impact discovery detection” is attractive and dangerous. If interpreted literally, it asks the system to forecast future scientific, industrial or societal importance. That target is too noisy for a credible general technology. Important discoveries can be ignored initially; fashionable results can accumulate citations without deep epistemic change; and downstream value depends on institutions, complementary technologies and historical contingencies that are absent from the result itself.

NOVA therefore uses a different operational object: a candidate scientific state transition. Given a knowledge state K at time t and a new result r , the system asks which parts of K would be revised, added, narrowed, strengthened, weakened or rendered obsolete if r were accepted. This can be local. A result does not need to transform an entire field; it may materially change one mechanism, one frontier or one decision-relevant dependency. The output is not “this is a breakthrough”. It is an argument describing the proposed state change and its uncertainty.

Design inference: Scientific-state change is proposed as a better computational target than future impact because it is closer to observable evidence and can be audited. Whether it correlates sufficiently with expert perceptions of important progress is a central validation question.

Scope and transfer How broad is the validated regime? Does the result generalise across datasets, populations, scales or domains, or is it a narrow local exception? A narrow boundary condition can itself be important if it invalidates an assumed universal claim.

Verification value Which next experiment or analysis would most reduce the uncertainty that matters for the scientific decision? A result with weak evidence but large potential consequence may be highly valuable to test even when it should not yet be accepted.

These dimensions form an Advance Profile rather than a universal scalar. A scheduler may derive a task-specific priority score—for example expected information gain per unit experimental cost—but that score should be understood as a decision heuristic. It is not a timeless property of the discovery.

A useful evaluator must avoid overfitting to the archetype “new positive effect with $p < 0.05$ ”. Different classes of scientific progress have different signatures. Empirical discoveries add or revise

observations. Mechanistic advances change a causal or explanatory structure. Methodological advances improve what can be measured, predicted or controlled. Negative findings eliminate plausible branches. Boundary-condition discoveries show where an accepted rule fails. Integrative advances unify previously separate observations. Engineering advances move a feasible frontier. Transfer advances show that a structure from one domain works in another. A platform that encodes only document novelty will systematically miss several of these forms.

The user-facing language matters. NOVA should say “high potential consequence because this claim is upstream of several accepted models and would reverse a current decision rule” rather than “high-impact discovery”. It should say “possible cross-domain leverage; structural match found, not yet experimentally validated” rather than “transferable breakthrough”. This wording is not cosmetic. It forces the architecture to expose the evidence supporting each judgement and prevents a probabilistic research signal from becoming an institutional label.

By mid-2026, it is no longer accurate to describe AI for scientific discovery as merely summarising papers or answering questions. The stronger systems execute meaningful portions of the research cycle. The AI Scientist family demonstrates automated machine-learning ideation, experimentation and writing. Co-Scientist demonstrates multi-agent generation, critique and hypothesis evolution around scientist-provided objectives. Robin integrates literature work and data analysis in experimental biology. Kosmos uses a structured world model to sustain long data-driven investigations. AutoScientists distributes work among teams that share experimental state and failed attempts. These are not equivalent achievements and should not be placed on a single autonomy ladder, but they collectively establish a richer technical baseline than a generic RAG assistant.

The AI Scientist-v2 Demonstrates end-to-end autonomous ML research including hypothesis formulation, experiment execution and manuscript production, with agentic tree search and an experiment manager. It is strong evidence for workflow automation in a verifier-rich computational domain. It does not establish reliable domain-general scientific judgement or external epistemic adjudication.

Co-Scientist Uses multiple agents that generate, critique and evolve hypotheses under scientist-specified objectives. It shows that test-time compute, debate and iterative ranking can improve ideation. Its internal scoring and hypothesis evolution are useful precedents, but they are not a proof that model-mediated ranking provides calibrated novelty or significance across domains.

Robin Published in Nature in 2026, Robin automates hypothesis generation and data analysis for experimental biology and connects those intellectual stages in one multi-agent workflow. It is important because the evidence goes beyond toy benchmarks. NOVA should complement such systems by evaluating the resulting claims against an explicit external memory and evidence model.

Kosmos Runs long cycles of literature search, data analysis and hypothesis generation while sharing information through a structured world model. This is directly relevant to NOVA: persistent state and traceable evidence are already emerging in advanced research agents. NOVA therefore cannot claim that “world models for science do not exist”; its proposed contribution is a specialised, reusable epistemic evaluator rather than a run-local research workspace.

AutoScientists Uses decentralised teams around a shared experimental state, including criticism before compute is spent and sharing successes and failures. It provides evidence that research orchestration benefits from explicit state and resource-aware selection. Its objective is still to improve research performance within tasks rather than to build a domain-independent adjudication layer for the scientific record.

A second 2026 literature is explicitly sceptical of interpreting successful workflows as scientific reasoning. Ríos-García and colleagues evaluated LLM-based scientific agents over more than 25,000 runs and report that evidence was ignored in a large fraction of traces and refutation-driven belief revision was uncommon. A separate position paper argues that autonomous science remains limited by problem selection, missing tacit and failure knowledge, hypothesis convergence and weak physical feedback, and specifically calls for persistent world models with uncertainty and belief revision. These

works do not prove that current agents are scientifically invalid in all uses, but they show why outcome-only evaluation is insufficient.

This distinction strengthens rather than weakens the NOVA motivation. If a system can produce a good result while following an epistemically weak path, downstream consumers need a record that is more explicit than the generator's narrative. Conversely, a carefully reasoned hypothesis may still be wrong. Scientific reliability ultimately depends on evidence and correction mechanisms, not on the eloquence

The most important architectural development for NOVA is not the increasing number of agent roles. It is the move toward persistent shared state. Kosmos uses a structured world model. AutoScientists shares experimental state across teams. The 2026 critique literature independently argues that an autonomous scientist requires a mutable epistemic state rather than a sequence of context windows. This convergence supports a conservative design decision: NOVA should be built around a durable scientific memory that exists independently of any particular LLM session.

What remains uncertain is the correct representation. A single knowledge graph may be too rigid; a document store is too weak; a learned latent state is difficult to audit. NOVA therefore proposes an event-sourced symbolic layer with multiple retrieval views. This is a design response to the evidence, not a result already demonstrated by the cited systems.

Novelty checking has moved from ad-hoc prompts toward dedicated retrieval and benchmark work. Shahid and colleagues' Literature-Grounded Novelty Assessment system retrieves a broad candidate set, filters it and applies facet-based LLM re-ranking; their experiments report higher agreement than previous approaches. The result is important because it demonstrates that novelty judgements improve when the model is forced to compare an idea against retrieved literature rather than judge from its parametric memory alone.

However, 2026 benchmarks reveal that the problem remains far from solved. RINoBench contains 1,381 expert-derived research ideas and finds that LLM rationales can resemble expert reasoning while the resulting novelty scores still diverge from human judgements. RQ-Bench reports an even sharper "novelty mirage": LLM judges prefer model-generated research questions as more novel while domain experts prefer author-anchored questions. An axiomatic novelty benchmark further argues that citation and review labels are confounded and tests novelty metrics using controlled changes to the comparison pool instead of assuming a clean ground-truth novelty label.

TruthHypo and its KnowHD framework provide another useful precedent. The benchmark partitions a biomedical knowledge graph by publication time so that "seen" relations precede a cutoff and "unseen" relations represent later discoveries; the authors use structured multi-hop knowledge to improve grounding and detect hallucinated hypotheses. The exact task is narrower than NOVA, but the methodology is highly relevant: temporal slicing, explicit relations and future-held-out knowledge make it possible to test whether a system could have identified something before it became known.

This suggests that the strongest retrospective benchmark for NOVA is not "ask experts whether this old discovery feels important". It is a time-machine experiment. Freeze the accessible scientific memory at date t , present candidate results available shortly after t , and measure whether the system finds the correct prior-art delta, distinguishes genuine from false novelty and identifies consequences that were supportable at the time. Later evidence can be used for validation but must never leak into the input state.

Classical scientific knowledge graphs often represent short entity-relation triples. That abstraction is useful for retrieval and link prediction but can be too coarse for understanding how one scientific contribution depends on another. The 2026 Scientific Contribution Graph explicitly targets an intermediate granularity: detailed contribution nodes and prerequisite relations extracted from full papers. SciAtlas, a 2026 technical report, similarly argues for large-scale multidisciplinary graph structure and combines lexical, vector and graph propagation retrieval in a neuro-symbolic pipeline.

These developments matter because they narrow NOVA's legitimate novelty claim. A graph-based scientific memory is not itself new. Large-scale graph construction, contribution extraction and

neuro-symbolic retrieval are already active research areas. The proposed research gap is downstream: can such a memory support explicit adjudication of state change, evidence and consequence, and can that adjudication remain calibrated across domains? NOVA should be compatible with external resources such as contribution graphs or scientific atlases rather than duplicate them by default.

The term “neuro-symbolic” is used broadly enough that it can become uninformative. For NOVA it has a narrow operational meaning. Neural models solve approximate interpretation problems: mapping scientific language, figures or raw metadata into candidate structured objects; retrieving paraphrases; proposing entity links; suggesting possible analogies; and generating human-readable explanations. Symbolic components maintain identity, time, provenance, scope, constraints, relation semantics and executable comparisons. A neural model may propose that two claims are equivalent, but the accepted equivalence is stored as an auditable relation with confidence and source evidence.

This division is justified by the failure modes of the task. Scientific identity must persist when the underlying LLM changes. Provenance should not disappear when context is truncated. Unit conversion, constraint checking and dependency traversal should be reproducible. At the same time, expecting a hand-written ontology and parser to understand arbitrary scientific prose is unrealistic. The architecture therefore accepts uncertainty at the boundary and tries to prevent that uncertainty from being silently converted into symbolic certainty.

Vector Symbolic Architectures (VSA), closely related to Hyperdimensional Computing (HDC), represent symbols and compositional structures in high-dimensional vectors using operations such as binding, bundling and permutation. Recent work continues to study VSA/HDC as a framework for graph reasoning and efficient associative computation. These properties make VSA an interesting candidate for structural fingerprints: a claim subgraph, mechanism pattern or role-filler structure could be encoded into a hypervector and searched approximately before exact symbolic verification.

The argument should not be overstated. There is not yet sufficient evidence that VSA will outperform modern dense retrieval, graph indexes or graph kernels for the specific problem of scientific contribution matching. NOVA therefore treats VSA/HDC as an optional retrieval experiment. It may improve speed, memory use or robustness to variable surface representation. If it does not, it should be removed without changing the scientific memory or the semantics of the evaluator.

A credible research programme should not create novelty by drawing boundaries around prior work. Existing systems already generate and rank hypotheses, perform literature-grounded novelty checks, maintain structured run state, execute experiments, use knowledge graphs, perform critique and select subsequent actions. Any proposal that describes those components generically as the innovation is too weak in 2026.

The narrower opportunity appears at the interface between generated research and the evolving scientific record. Current systems are typically optimised for the success of a particular research loop: improve a benchmark, produce a useful hypothesis, complete an investigation, or rank candidate ideas. Novelty benchmarks focus mainly on whether an idea differs from prior work. Knowledge graphs focus on organising and retrieving knowledge. None of these objectives is identical to maintaining a reusable, evidence-aware representation of what a new result would change and what should be tested next.

A contract for scientific identity The system needs a canonical, time-stamped object representing “the same scientific claim” across papers, agents, paraphrases and later revisions. Existing retrieval can propose equivalence, but an evaluator needs the accepted mapping and its uncertainty to persist.

NOVA can now be stated without inflated claims: build a persistent neuro-symbolic layer that receives candidate results from arbitrary human or automated research processes, resolves them against a time-bounded scientific memory, expresses the minimal proposed change to that memory, estimates evidence and consequence using domain-appropriate methods, and produces an Advance Case together with a verification recommendation. The system is reusable if the core objects and reasoning interfaces remain stable while domain-specific ontologies, evidence policies and tools are exchanged.

This thesis is ambitious because scientific semantics are difficult and because “consequence” is only partially formalizable. It is nevertheless concrete. Each stage has an observable input and output. Each neural transformation can be traced to source text or data. Each symbolic relation can be inspected. Each research claim has a baseline. The programme can discover that some dimensions are too domain-specific, that graph extraction is too noisy, that expert agreement is too low or that simple RAG baselines perform just as well. Those are informative failure modes, not reasons to avoid the experiment.

The simplest design asks a strong reasoning model to read the candidate result, retrieve a few papers and score novelty, significance and confidence. This is attractive because it is cheap to prototype, domain-flexible and often produces convincing explanations. It should be included as a baseline, not adopted as the final architecture. RINoBench and RQ-Bench provide direct reasons for caution: plausible rationales do not reliably imply correct novelty judgements, and model judges can systematically prefer model-generated questions.

The deeper issue is state. A prompt-based judge has no durable identity for claims, no guaranteed temporal boundary, no native representation of evidence dependence, and no stable semantics for changes in scope. All of these can be simulated in text, but the system becomes difficult to test once the same LLM both interprets the evidence and defines the meaning of its own judgement.

A retrieval-grounded judge is substantially stronger. The Literature-Grounded Novelty Assessment work shows that staged retrieval and facet-based comparison can improve agreement. For many practical use cases this may be the right solution. A research programme claiming more must therefore beat a high-quality RAG evaluator, not an ungrounded prompt.

RAG nevertheless remains document-centric by default. It retrieves candidate antecedents but does not necessarily preserve a stable representation of how the candidate changes a claim graph or how evidence sources depend on one another. It is best understood as a necessary component of NOVA and a strong baseline against which the symbolic state machinery must justify its cost.

A graph-first system provides identity, provenance, temporal relations and exact traversal. It is appealing for auditability. The difficulty is extraction and ontology evolution. Scientific papers contain nuanced conditions, implicit assumptions, equations, figures and domain-specific language that cannot be robustly converted into a fixed schema with hand-written rules. A purely symbolic system is likely to have high precision only on a narrow slice of science or to require prohibitive manual curation.

A biomedical evaluator trained on clinical or molecular evidence, or an ML evaluator trained on benchmark tables and ablation conventions, can exploit domain regularities much more aggressively than a generic platform. Such systems may outperform NOVA within a single domain. The cost is fragmentation and weak transfer. The research question is therefore not whether generic always beats specialised—it probably will not—but whether a common kernel can carry enough of the expensive logic that new domain evaluators become adapters rather than new products.

NOVA chooses a layered architecture because it can incorporate the strengths of the other options and make its added value testable. Retrieval and LLMs handle open language. A temporal scientific memory stores stable objects. Specialised statistical, causal or optimisation tools operate only when their assumptions are satisfied. The output is a structured case rather than a single model judgement.

NOVA can be understood as a transaction processor for scientific knowledge. A research agent or scientist submits a candidate result. The system first converts it into a canonical Result Capsule. It then retrieves the strongest antecedents from the scientific memory and expresses the minimal delta. Next it asks whether the evidence is good enough to take that delta seriously, and what accepting it would change. Finally it creates an Advance Case and, when uncertainty remains material, recommends the next discriminating test. Only after confirmation does the scientific memory receive a new accepted or revised state.

This analogy has limits—scientific truth is not a database transaction—but it is useful because it emphasises identity, provenance, atomic changes and reversibility. NOVA should be able to show

exactly which symbolic assertions were added or revised, which source spans justified them, and which later evidence caused their status to change.

Scientific Memory is the source of truth for NOVA's symbolic decisions. It should not be a monolithic graph loaded into RAM. A practical implementation is an append-only event store plus materialised graph and index views. Every new claim, correction, contradiction, replication or retraction becomes an event. Current and historical views can then be reconstructed for a knowledge cutoff, which is essential for retrospective evaluation.

The graph should distinguish claims from evidence and documents. Documents are sources; they are not the epistemic unit. Core relations include supports, contradicts, refines, generalises, narrows, depends-on, derived-from, tested-by, uses-data-from, dominates-under-constraints and equivalent-to. Relations carry scope, time, provenance and confidence. This is closer to the emerging contribution-aware graph literature than to a conventional entity-triple knowledge graph, but NOVA adds evidence state and evaluation semantics.

The memory must tolerate disagreement. It should not force a single Boolean truth when a field contains competing models. A claim can be strongly supported in one scope and weakly contradicted in another. A later result can reduce confidence without erasing the historical state. This is one reason event sourcing is attractive: epistemic changes become inspectable history rather than destructive updates.

What remains a research claim There is no evidence yet that exactly five lenses are the optimal decomposition, or that consequence can be robustly formalised across domains. The architecture should permit lenses to be removed, merged or specialised after ablation.

The Prior-Art Lens should optimise recall before judgement. Exact lexical search is indispensable for named methods, equations, identifiers, chemical compositions and unusual terminology. Dense retrieval handles paraphrase and terminology drift. Graph neighbourhood and prerequisite search finds contributions that are structurally related even when surface language differs. A final re-ranker compares candidate facets—problem, mechanism, method, scope, evidence and claimed contribution—against the retrieved set. This design follows the empirical lesson of literature-grounded novelty work while adding structured comparison.

The crucial output is not the ranked paper list. It is a delta object. For example: “method family already known; candidate differs by replacing optimisation step X with Y under resource constraint C; no antecedent found for this combination before cutoff t.” Such a delta can be wrong if retrieval is incomplete, so it carries coverage and confidence. Absolute novelty should never be asserted when the corpus is not known to be complete.

Bayesian updating, likelihood ratios, Bayes factors and information gain provide principled answers when a domain can specify hypotheses and predictive distributions. They are particularly useful for the Evidence and Verification Lenses: how much should an observation shift belief, and which next experiment best discriminates among alternatives? Bayesian experimental design gives a mature conceptual basis for selecting observations by expected utility or information.

The limitation is often ignored. Scientific agents rarely possess a defensible prior or complete hypothesis space. Using arbitrary LLM probabilities as Bayesian priors does not make the result Bayesian in a meaningful scientific sense. NOVA should use probabilistic updating only when the prior, likelihood or surrogate model is explicit enough to audit. Elsewhere it can maintain ordinal evidence states and uncertainty without pretending to numerical precision.

Large residuals, failed predictions and contradictions are valuable because science often advances where the current model fails. Statistical anomaly detection, posterior predictive checks and rule contradiction can therefore trigger deeper analysis. But mismatch has many mundane causes: corrupted data, distribution shift, measurement error, implementation bugs or an inappropriate baseline. The architecture should treat anomaly as a routing signal to the Evidence and Verification Lenses rather than a “novelty score”.

Mechanistic claims can have high scientific leverage because they change how observations are explained and what interventions are predicted to work. Causal graphs, structural causal models, intervention logic and domain simulators can support the Consequence Lens. The symbolic memory is naturally suited to storing candidate causal relations and their evidential status.

Causal language must be conservative. Many automated research systems operate mainly on observational datasets. A neural model can invent a plausible mechanism in prose without satisfying any identification conditions. NOVA should encode the type of support—randomised intervention, natural experiment, longitudinal evidence, mechanistic assay, simulation, observational association—and refuse to upgrade an association to a causal edge merely because the explanation sounds coherent. Pearl’s causal framework remains useful here precisely because it makes assumptions explicit.

Some advances matter because they compress. A new equation can explain multiple empirical regularities; a reusable programme fragment can replace a collection of ad-hoc rules; a mechanism can unify phenomena previously treated separately. Minimum Description Length (MDL) provides one principled way to compare explanatory complexity against fit. Symbolic regression and programme synthesis provide concrete search mechanisms in domains where explanatory forms can be represented.

Compression is representation-dependent. A short expression can hide enormous complexity in an opaque learned function. NOVA should therefore report the representation language and primitives used when claiming explanatory gain. “Shorter” is scientifically meaningful only relative to a fixed or justified coding scheme. This technique is promising for specific domains but should not be a mandatory lens for every result.

In engineering-oriented science, advances often appear as movement of a feasible frontier: better accuracy for the same compute, higher conductivity without losing stability, greater sensitivity at the same assay cost, or a new operating regime. Multi-objective optimisation and Pareto analysis make this operational. The challenge is comparability. NOVA must encode resource and regime assumptions so that a result using ten times more compute or a different dataset is not declared dominant by construction.

Constraint reasoning can also detect a different kind of advance: a result that makes a previously infeasible branch feasible by relaxing one bottleneck. Constraint solvers, type systems or SMT backends may be appropriate for sharply formalised subdomains. Their role is exact verification under declared assumptions, not universal scientific reasoning.

Literature-based discovery has long sought implicit connections across disjoint literatures; Swanson’s classic work is a foundational example. Modern LLMs and graphs greatly expand the search space. For NOVA, analogy is most useful when expressed structurally rather than as semantic similarity. The Scope Lens can search for role-preserving patterns—an inhibition relation, feedback loop, bottleneck or optimisation structure—that appears under different vocabulary.

The standard of success must be prospective. A clever-looking analogy is not itself a scientific advance. The system should be rewarded only if the analogy yields a testable prediction, method or mechanism that survives target-domain validation. This is an attractive research direction precisely because it is high-risk and easy to falsify.

VSA/HDC could support the analogy and prior-art paths by encoding compact compositional fingerprints. Binding can represent role-filler pairs, permutation can represent order or edge direction, and bundling can aggregate neighbourhood structure. Recent VSA research includes graph reasoning and emphasises efficient associative computation. A plausible NOVA experiment is to encode local claim or mechanism graphs into hypervectors and use nearest-neighbour search to shortlist structural analogues before exact graph matching.

The Verification Lens is where the techniques become a scientific control system rather than a ranking service. Given competing interpretations, it should select the test with the highest expected reduction in decision-relevant uncertainty subject to cost and safety. Candidate tests can be generated

by LLMs or domain planners, but feasibility and utility should be checked against symbolic constraints and explicit uncertainty models.

This design changes the optimisation target. If a candidate result has enormous potential consequence but weak evidence, NOVA should not promote it as true; it should prioritise a discriminating test. If a result is already strongly supported but has little consequence, the correct action may be archival rather than more experimentation. This asymmetry is one of the clearest ways to separate epistemic value from generic “interestingness”.

A common failure mode in cross-domain platforms is to standardise too much. NOVA should not define one ontology for all science. The core should define only objects whose semantics are stable across domains: claim, evidence item, observation, method, scope, comparator, uncertainty, provenance, dependency, contradiction, equivalence, verification action and status change. Domain packs then extend types and validators.

This keeps the architecture falsifiable. If a biomedical pilot needs a new evidence class, that is expected. If it requires changing what “provenance” or “scope” means, the core abstraction may be wrong. Cross-domain reuse can therefore be measured as schema stability, shared code and unchanged reasoning contracts rather than by superficial reuse of the same user interface.

LLMs can be standard foundation models, domain-adapted models or small specialised models. NOVA should not depend on a single provider or model family. Every neural component receives a narrow contract: extract candidate claims with source spans, propose entity links, rank retrieved antecedents, interpret a figure, propose alternative mechanisms, or generate a human-readable explanation. Outputs that affect scientific state are validated before they become canonical objects.

This boundary provides two practical advantages. Model upgrades can improve interpretation without invalidating historical claim identity. And errors can be localised: if a wrong Advance Case originates in an incorrect entity link, the system can correct and replay the affected state transition rather than

A scientific memory spanning millions of contributions cannot be loaded as one in-memory graph. The implementation should partition by domain and concept neighbourhood while maintaining compact global indexes. An append-only event log preserves provenance; materialised graph shards support local reasoning; lexical and dense indexes support high-recall retrieval; structural fingerprints can be cached; and only the active subgraph for the current evaluation is loaded into working memory. Frequently used neighbourhoods can be cached with version identifiers.

The scheduler should operate in cost tiers. Tier 0 validates schema, provenance and exact duplicates. Tier 1 runs lexical, dense and graph retrieval plus lightweight comparison. Tier 2 performs deeper evidence and dependency analysis. Tier 3 invokes expensive causal inference, symbolic search, structural analogy or domain simulators. Tier 4 proposes or triggers new experiments under explicit authority. This avoids spending frontier-model or laboratory budget on candidates that are already known or poorly formed.

The external API can remain simple even when the internals are sophisticated. A research agent submits a Result Capsule or raw artefact plus domain context. NOVA returns a case identifier immediately and a progressively refined Advance Case as deeper analyses complete. Agents can query prior art, request the evidence path behind a judgement, ask for unresolved alternatives or request a verification plan. Human reviewers can correct entity mappings, scope, evidence dependence or case status; corrections become versioned events.

This makes NOVA compatible with systems that already have their own planning and memory. It need not replace Kosmos-like world models or AutoScientists-like shared experimental states. It can operate as an independent adjudication service that those systems call when a candidate claim reaches a threshold of potential significance.

Automated science will often operate on unpublished or proprietary data. Scientific Memory therefore needs tenant-aware access control and provenance-aware visibility. A private result may be used to prevent redundant experiments within one organisation without being exposed to other tenants.

Cross-tenant novelty checking is harder: the system may need privacy-preserving summaries, secure enclaves or organisation-level policy rather than a global shared graph. NOVA should not promise a universal cryptographic solution in its core; it should define the visibility semantics so privacy technologies can be attached.

Suppose an automated research agent reports that a new training method improves a benchmark. A weak evaluator retrieves papers with similar terms and asks an LLM whether the method is novel. NOVA instead normalises the claim into method, dataset, metric, compute budget, baseline family, evaluation protocol and uncertainty. The Prior-Art Lens may discover that the core mechanism appeared earlier under different terminology. If so, the conceptual novelty is low even if the implementation is new.

If the mechanism is distinct, the Evidence Lens checks whether the comparison uses matched data, model scale, hyperparameter budget and test protocol. The Consequence Lens asks whether the result moves a Pareto frontier rather than simply spending more compute, and whether it changes a widely used assumption about optimisation or generalisation. The Scope Lens searches other datasets or model scales. The Verification Lens may recommend an ablation that separates the proposed mechanism from a simpler explanation. The resulting Advance Case can therefore classify a paper as “novel implementation, no demonstrated frontier movement” or “credible new mechanism with frontier improvement; replication across scale required”.

Consider an agent that finds an association between a compound, a pathway and a disease phenotype. The Result Capsule must separate observational association from mechanistic claim. The Prior-Art Lens searches literature and structured biomedical resources for equivalent relations and nearby mechanisms. The Evidence Lens records whether support comes from retrospective cohorts, perturbation experiments, animal models, cell assays or independent datasets, and checks whether several papers reuse the same cohort.

The Consequence Lens can identify which current mechanistic models or therapeutic hypotheses depend on the relation, but it cannot infer causality merely from graph connectivity. If the available evidence is observational, the Advance Case may say “potentially high mechanistic leverage; causal status unresolved”. The Verification Lens can propose a perturbation or targeted assay if the domain pack knows feasible experimental actions. Human approval remains mandatory for biological experimentation. This scenario demonstrates why evidence class and scope must be symbolic objects rather than prose adjectives.

A candidate material may appear impressive because one property improves substantially. Scientific value depends on the joint constraints. Conductivity, stability, synthesizability, cost, toxicity, temperature range and manufacturability can trade off. The Result Capsule therefore carries the full experimental regime and resource assumptions. The Prior-Art Lens finds related compositions and processes; the Evidence Lens checks measurement protocol and replication; the Consequence Lens evaluates whether the candidate is genuinely non-dominated under the relevant objectives.

The system may discover that the material is not a frontier advance because the reported improvement occurs only at an impractical temperature, or it may identify a small but important frontier shift because the candidate removes a manufacturing constraint. Here “high potential” is not inferred from semantic novelty. It is derived from a concrete change to the feasible set. The 2026 critique literature on autonomous materials discovery emphasises exactly why tacit synthesizability and multi-objective trade-offs are difficult for current agents. NOVA should therefore represent unknown practical constraints explicitly rather than assume published data are complete.

Suppose repeated high-quality experiments falsify a mechanism that has motivated a costly line of work. Textually, the result may contain little novelty; it may even resemble earlier null findings. A citation or novelty model can under-rank it. In NOVA, the Evidence Lens can recognise strong independent refutation, while the Consequence Lens sees that many downstream hypotheses depend on the rejected mechanism. The Advance Case can therefore assign high consequence without claiming high novelty. The Verification Lens may recommend one final discriminating test or, if the evidence is already strong, advise stopping further experiments on that branch.

This example captures the central motivation better than a glamorous “breakthrough” scenario. A scientific prioritisation system should help stop bad directions as well as celebrate new ones. An automated research ecosystem that optimises only for positive novelty will waste compute and amplify publication bias.

Imagine that a feedback structure in a biological regulatory network resembles a stabilisation mechanism in distributed control. A structural retrieval engine may surface the analogy even though the vocabularies are unrelated. NOVA does not award value for the match itself. It asks whether the mapping preserves the relevant roles and constraints and whether it suggests a target-domain prediction or intervention that was not already known. The verification step is decisive. If the analogy generates no testable consequence, it remains an interesting association, not an advance.

The architecture is a hypothesis about how to solve the problem, not the research objective itself. The programme should therefore organise evaluation around questions that can invalidate major design choices. Building all proposed modules and demonstrating a polished interface would be a weak scientific outcome if a simpler RAG baseline performs equally well.

Retrospective evaluation is attractive because later science provides evidence about whether a candidate result was duplicated, replicated, refuted or consequential. It is also dangerous because hindsight leaks easily. The benchmark must build immutable knowledge snapshots from sources available before a cutoff date. The candidate result is evaluated only against that snapshot. Later publications are stored in a separate outcome store used exclusively for scoring.

Cases should include well-known advances, ordinary incremental results, false starts, retractions, negative findings, rediscoveries and results that looked promising but did not generalise. This balanced design prevents the benchmark from becoming a collection of famous success stories where importance is obvious from hindsight. The 2026 temporal methodology used in TruthHypo is a useful precedent for separating “seen” and later “unseen” scientific relations.

Retrospective benchmarks cannot validate the central operational claim: that NOVA helps choose what to do next. Prospective pilots should therefore feed live candidate results from research agents or human teams into NOVA before the outcome is known. Experts review the Advance Case, choose or reject recommended verification actions, and later record whether the test changed the decision state. Prospective data are slower and more expensive but far less vulnerable to hindsight.

The ideal unit is not “did NOVA predict a future famous paper?” It is “did the system correctly identify a material uncertainty, consequence or prior-art relation and propose a verification action that improved the research decision under the available budget?” This metric is closer to the function the component actually performs.

Table 8. Novelty, correctness, significance, utility and potential should remain separate dimensions.

Term	Meaning in the research environment
Novelty	What is not supported as already known within the declared corpus and temporal boundary.
Correctness	Whether the claim or result is supported by adequate evidence and valid reasoning.
Significance	How much the result changes explanation, capability, frontier position or future research possibilities.
Utility	Whether the result is useful for a declared scientific, industrial or societal objective.
Potential	A forward-looking hypothesis about possible future value; it should not be disguised as an observed fact.

The most valuable output of novelty intelligence may be a better next question. If the candidate is probably a rediscovery, the system can redirect work. If it appears novel only because the corpus is weak, it can request broader retrieval. If the mechanism is known but the operating regime is new, it can identify the experiments that would establish whether the transfer matters. If a negative result closes a

large region of the search space, the system can record that scientific value even when no positive discovery was made.

This turns novelty assessment into part of experimental design. A mature research environment would not ask NOVA for a ceremonial score at publication time. It would use novelty, contradiction and frontier information throughout the project to decide where attention, compute, laboratory time and human expertise should be allocated. Whether this can be done reliably enough to improve real research programmes is one of the central falsifiable questions of the book.

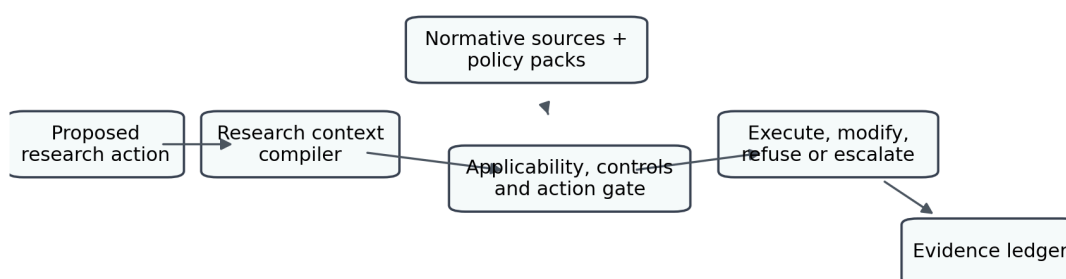
CHAPTER 8

Compliance as Continuous Scientific Context

Compliance is often represented as a document produced before or after technical work. Agentic research breaks that model because compliance-relevant choices occur inside execution. An agent chooses a data source, a model provider, a processing location, a licence, a recipient, a purpose and an output channel. The legality or institutional acceptability of the action may depend on facts that changed since the project began. Governance must therefore be able to meet the agent at the action boundary.

The Compliance Co-Pilot proposed here is not an automated lawyer and should not issue universal legal verdicts. It is a configurable governance infrastructure. It compiles research context, retrieves applicable normative sources, interprets policy packs, maps obligations to technical controls, records evidence and escalates unresolved questions. Its strongest contribution is not legal reasoning in the abstract but the ability to make compliance state machine-readable enough to constrain a concrete research action before the action occurs.

Figure 6. Compliance is most useful when it operates at the action boundary.



The proposed research gap is narrower. Enterprise governance platforms usually take an AI system, use case, model, data estate or agent as the governed asset. Automated science requires the compliance object to include the evolving scientific action itself: why a dataset is being used, whether the purpose changed, what an agent is about to transmit, which protocol covers the experiment, whether a licence permits the intended text-and-data mining, or whether publication changes a research-security risk. A mature GRC platform could ultimately host or integrate this functionality. The research question is whether scientific context modelling and executable applicability reasoning add enough value to justify a specialised layer.

The EU AI Act is an obvious starting point, but it should not dominate the architecture. The Act contains research and development exclusions, and its applicability depends on how an AI system or model is developed, placed on the market, put into service or used. The current 2026 framework also has staged enforcement dates; after the AI Omnibus, high-risk rules apply later than some other provisions. A research project can therefore be outside a particular AI Act requirement while remaining fully subject to data protection, copyright, contractual, ethical, cybersecurity or sector-specific rules.

This is a useful architectural lesson: applicability cannot be inferred from the presence of AI alone. The Co-Pilot must first classify the role of the AI component—research object, general tool, model provider, internal system, external service, future product component—and preserve the purpose and deployment state. A scientific R&D exclusion should be represented as a scoped rule with conditions, not as a global 'research exemption' flag.

The AI Act is also valuable because it normalises governance concepts that the Co-Pilot can reuse even where the Act does not directly apply: documentation, logging, human oversight, risk management, accuracy, robustness and cybersecurity. The architecture should therefore separate normative applicability from reusable control patterns.

For data-intensive research, privacy law is often more operationally important than AI-specific regulation. The GDPR permits and facilitates scientific research in several ways, but research is not a blanket exemption: lawful basis, purpose, data minimisation, rights, security, accountability and Article 89 safeguards remain context-sensitive. The Co-Pilot should therefore model data-processing purpose, categories of data, actors and roles, source, transformations, transfer destinations, retention and technical safeguards as first-class facts rather than relying on a generic 'personal data: yes/no' label.

The EHDS makes the need for executable context even clearer in health research. The Commission describes secure processing environments for secondary use, restrictions on downloading personal data, preference for anonymised data where sufficient, and a prohibition on re-identification. Important provisions phase in over time. A future health-data domain pack could convert permit conditions and secure-processing constraints into runtime controls—for example, disabling export functions, restricting accessible tools, enforcing output checking or preventing joins that would violate an approved purpose. This is a plausible use case because many controls are technical; legal-basis and permit interpretation should still be human-supervised.

Automated literature research raises a different normative problem: the right to access and computationally analyse content is not identical to the right to redistribute it, train a model on it or publish derived corpora. The EU Copyright in the Digital Single Market Directive includes text-and-data-mining exceptions, including a research-specific exception for research organisations and cultural heritage institutions with lawful access, and a broader exception that can be limited by rights reservation. The Co-Pilot therefore needs licence and source metadata, not just citation metadata.

For scientific agents, a useful policy primitive is not 'can read paper' but a set of rights over an asset and action: retrieve, cache, mine, quote, derive, train, redistribute, publish metadata, or expose content to an external service. ODRL's permissions, prohibitions and duties provide a useful conceptual vocabulary for such cases. A domain-specific implementation may be simpler than full ODRL, but interoperability with rights-expression standards is worth preserving.

The 2026 ERA Living Guidelines are unusually relevant because they address the research workflow itself rather than only AI products. They require researchers to remain responsible for scientific outputs, use generative AI transparently, protect privacy/confidentiality/IP, verify citations and generated content, seek reproducibility, and avoid substantial GenAI use in sensitive assessment activities. They ask research organisations to provide support for legal and ethical compliance, track AI use, integrate guidelines into good research practice, and where appropriate provide governed AI environments. They also introduce explicit warnings about third-party AI use and hidden prompts.

These recommendations are not all directly executable, but many can generate evidence requirements. If an agent substantially contributes to hypothesis generation or data interpretation, the environment can record tool name, model version, prompts or relevant traces and the role of AI in the method. If confidential material is about to leave an approved environment, the runtime gate can evaluate the external provider and data policy. If a document contains invisible instructions that may

Horizon Europe already treats ethics as a continuing part of funded research rather than a one-time declaration: projects with ethics issues can be subject to review and specific ethics requirements that become part of the grant agreement. In institutional settings, the Co-Pilot should ingest ethics approvals and conditions as project-specific normative objects. An automated system can check that an intended action stays within an approved protocol; it should not autonomously grant an ethics waiver or reinterpret a material protocol change.

Research security and dual-use controls create harder cases because scientific openness and security can conflict. EU export-control rules apply to dual-use items, including forms of technology, and the Commission has issued research-oriented compliance guidance. These cases are a strong argument for escalation-aware design. The Co-Pilot may detect triggers, classify assets and jurisdictions, check institutional screening results, and prevent an agent from publishing or transferring restricted material before review. It should not attempt to resolve novel export-control classifications solely with an LLM.

Domain regulation adds further layers. Clinical research can engage the Clinical Trials Regulation and ICH Good Clinical Practice; research involving animals is subject to dedicated welfare law; medical devices and in-vitro diagnostics have separate regulatory frameworks. A universal Co-Pilot must therefore be modular. A generic kernel can represent actions, actors, evidence and rules, but each domain pack must be owned and validated by experts in that domain.

Copyright is unusually important for automated research because agents can copy at scale without making the copying salient to a human operator. A research workflow may retrieve a paper, cache it, extract figures, construct an embedding index, perform text-and-data mining, use excerpts in a prompt sent to a third-party model, fine-tune a model, generate a derivative dataset and later publish parts of the resulting corpus. These are technically adjacent operations but they are not normatively identical. A useful Co-Pilot therefore cannot attach a single boolean 'copyright cleared' flag to an asset. It needs a rights profile tied to the asset, the actor, the intended use and the jurisdiction.

The EU Copyright in the Digital Single Market Directive provides a concrete example of why this distinction matters. Article 3 requires Member States to provide a text-and-data-mining exception for qualifying research organisations and cultural heritage institutions conducting scientific research on works to which they have lawful access, and allows resulting copies to be retained for research and verification subject to appropriate security. Article 4 provides a broader TDM exception for lawfully accessible material, but permits rightsholders to reserve those rights, including through machine-readable means for publicly available online content. A system therefore needs facts such as research-organisation status, scientific purpose, lawful access, rights reservation and retention/security conditions before it can even form the right question.

Research action Rights-relevant evidence the Co-Pilot should seek Read or retrieve a paper Source identity, access route, user/institution entitlement, licence or terms, publication status, applicable jurisdiction. Build a local TDM corpus Scientific purpose, research-organisation status where relevant, lawful access, rights reservation, security and retention conditions. Send an article or figure to an external model Whether the service receives a copy; provider retention/training terms; confidentiality; licence restrictions; contractual permission; project policy. Fine-tune or train on a corpus Rights basis for large-scale reproduction/extraction; opt-outs or reservations where applicable; dataset licence; provenance; model-provider obligations if the resulting model is placed on the market. Publish excerpts, figures or derived data Licence/exception for publication, attribution, quotation context, transformation, database rights, confidentiality and output-release policy. Redistribute a research corpus Licence and contract terms governing redistribution; whether TDM copies may be shared; third-party rights; access-control requirements.

The standards landscape provides useful building blocks. ODRL represents permissions, prohibitions, duties and constraints over assets; Creative Commons licences expose a machine-readable layer; SPDX provides standard licence identifiers and expressions and, in its current specification family, is no longer limited conceptually to a single software file. These mechanisms make certain rights facts much easier to ingest than free-text contracts. They do not establish that the metadata is

The proposed system should therefore distinguish observed rights metadata from a reviewed rights conclusion. An asset may contain an SPDX identifier, a repository may expose a CC licence, a website may expose machine-readable TDM reservations, and a contract may impose additional restrictions. The Co-Pilot should preserve all of these as source facts, detect contradictions and missing terms, and escalate rather than silently choosing whichever machine-readable signal is easiest to parse.

For research teams that build or substantially adapt general-purpose models, the governance question extends beyond permission to read scientific literature. Article 53 of the EU AI Act requires providers of general-purpose AI models to maintain a policy to comply with Union copyright law and, in particular, to identify and comply with rights reservations under Article 4(3) of the DSM Directive; it also requires a sufficiently detailed public summary of training content. The 2025 General-Purpose AI Code of Practice includes a dedicated Copyright chapter intended as a voluntary route for demonstrating

compliance with those obligations. The Co-Pilot should not assume every research model is a regulated GPAI model, but it should preserve the facts required to determine role and scope.

This suggests a reusable Rights Resolver rather than a collection of one-off copyright prompts. Its input is an asset plus intended action; its output is a structured set of candidate permissions, prohibitions, duties, exceptions, missing facts and evidence requirements. The result then enters the same applicability and control pipeline as privacy, contracts and institutional policy. A licence may create an attribution duty; a TDM rule may require secure retention; a contract may prohibit external processing; a publication licence may allow redistribution but a confidentiality clause may still block it.

Copyright is a poor domain for false precision. Questions such as whether a generated output reproduces protected expression, whether a particular transformation creates a derivative work, how national implementations of exceptions interact with a concrete use, or whether a contractual term is enforceable in a given setting can require fact-sensitive legal judgement. The research goal should therefore be high-quality issue spotting, rights provenance and enforcement of reviewed bright-line rules—not automated adjudication of all copyright doctrine. Similar caution applies to patents, trade secrets, database rights and personality/publicity rights: they can be represented as additional rights families, but should not be collapsed into 'copyright'.

A project being 'research' does not make its data flows inherently privacy-compliant. Under the GDPR, scientific research is recognised in several provisions, but core principles such as lawfulness, purpose limitation, data minimisation, storage limitation, security and accountability remain relevant. Article 89 requires appropriate safeguards for scientific or historical research and explicitly highlights data minimisation and possible pseudonymisation. In practice, the decisive object for an automated system is therefore not the project description but the current processing operation and the lineage of the data entering it.

The distinction matters because autonomous agents create new processing operations cheaply. Joining two datasets may increase identifiability; extracting embeddings may preserve sensitive information; sending a prompt to an external provider may disclose personal or confidential data; a model created from personal data may itself create privacy risks; and publishing apparently aggregated output can create re-identification risk when combined with other sources. The Co-Pilot

Privacy state / question Machine-actionable representation or control Data identity and sensitivity Personal/non-personal/uncertain; special-category flags; direct and quasi-identifiers; source; subject population; confidence and classifier provenance. Purpose and legal basis Declared research purpose; compatible secondary purposes; consent or other legal-basis evidence; protocol/permit identifiers; conditions and expiry. Minimisation Fields strictly required for the current action; alternative aggregate, synthetic or local computation; automatic redaction or feature projection before release. Data subject / participant constraints Consent limitations, withdrawal or exclusion signals where applicable, disclosure constraints, vulnerable-population rules, protocol-specific promises. Environment and recipient Local/approved cloud/external API; controller/processor role; region; sub-processors; retention/training settings; contractual safeguards. Transfer Destination jurisdiction, adequacy/safeguard status and institutional transfer policy; onward-transfer constraints. Retention and deletion Permitted retention period, research exception/safeguard basis, derived-artifact retention, deletion or review events. Risk / DPIA High-risk triggers, large-scale special-category processing, novel technology, systematic monitoring or institution-specific DPIA triggers; route to DPO/privacy review. Derived artefacts and models Whether a result remains personal data or presents memorisation/re-identification risk; testing evidence; release restrictions; uncertainty preserved rather than overwritten.

GDPR Article 25 requires data protection by design and by default, including measures that implement minimisation and ensure that only data necessary for each specific purpose are processed by default. Article 35 requires a data protection impact assessment before processing likely to result in high risk, including several enumerated cases. Cross-border transfers are governed separately in Chapter V. These provisions do not tell an agent exactly what code to execute, but they justify an architecture in which the default action can be transformed: run locally instead of externally, remove

columns, pseudonymise identifiers, aggregate results, use a privacy-preserving query, require an approved environment, or escalate to a DPO.

The system should therefore maintain a Privacy Processing Record at action level. It need not duplicate the organisation's formal record of processing activities; rather, it links an agent event to the data categories, purpose, parties, location, legal/approval evidence, safeguards and generated artefacts. This makes runtime enforcement auditable and allows recurring safe patterns to be cached as reviewed policy decisions.

The European Data Protection Board's Opinion 28/2024 specifically addresses personal data in the development and deployment of AI models and rejects the idea that models trained on personal data can automatically be treated as anonymous. For a research Co-Pilot this implies a conservative design rule: pseudonymisation, embedding, aggregation or model training should be represented as transformations with evidence about residual risk, not as irreversible transitions to a universal 'safe' state. Where anonymity is material to permission, the system should require the organisation's accepted test or expert conclusion and retain the evidence behind it. An uncertain or undocumented data relationship should imply escalation, while statistical privacy tests or privacy-attack tools provide evidence about re-identification or memorisation risk. The LLM can explain and map context, but it should not be the sole classifier of whether data have ceased to be personal.

Many of the constraints that determine whether an automated research action should execute are not statutes. A hospital may prohibit patient-level data from leaving a secure environment. A project may forbid external generative-AI services before publication. A funder may require open outputs after an embargo. A laboratory may require two-person approval before an instrument performs a hazardous operation. A consortium agreement may prohibit release of a jointly developed benchmark. A principal investigator may impose a cost ceiling or require that every claim entering a manuscript retain source provenance. These rules can be entirely legitimate even when they are stricter than the legal minimum.

The Co-Pilot should therefore accept arbitrary governance rules at organisation, laboratory, project, workflow or task configuration time. 'Arbitrary' should not mean unstructured or untrusted. Each rule needs an authority class, owner, scope, effective period, version, priority/precedence policy, enforcement mode, source or rationale, tests and review status. The system must also distinguish a configured preference from a mandatory rule; otherwise a user-supplied convenience setting could accidentally be presented as a legal requirement.

Scope: training action. Decision: REQUIRE rights profile in permitted set; unknown licence => ESCALATE/BLOCK according to project mode. Use the internal model for confidential manuscripts. Scope: document confidentiality. Control: route tool call to approved endpoint; external providers excluded; log chosen endpoint. Do not spend more than CHF 500 equivalent per experiment without approval.

The practical abstraction is a Policy Pack: a signed, versioned bundle containing source documents or references, machine-readable rules, terminology mappings, test cases, authority metadata and deployment settings. Packs can be layered: a generic organisational pack, an EU privacy pack, a biomedical pack, a project contract pack and a task-specific safety pack. The effective policy for an action is the composition of applicable packs, not a single global file.

Policy-pack composition creates a research problem of its own. Rules can overlap, contradict or refer to facts at different abstraction levels. A project policy may be stricter than a licence; a law may override a local preference; an ethics approval may authorise only a subset of what a data licence permits. The Co-Pilot needs an explicit conflict model rather than 'last configuration wins'. In the

What the evidence does not establish is a universal machine-readable model of research compliance. It does not show that LLMs can translate arbitrary regulation into reliable executable rules, that a single ontology can cover life sciences and software research equally well, or that runtime gates can avoid both dangerous permissions and excessive blocking. These are the core uncertainties of the research programme.

Accordingly, the proposal should be evaluated against three alternatives rather than against a straw man: a strong legal/policy RAG agent; a mature enterprise governance platform with custom workflow logic; and conventional human compliance supported by static controls. The Compliance Co-Pilot is justified only if scientific context modelling and neuro-symbolic execution produce measurable gains in traceability, critical-error detection, response time or operational friction.

The proposed primary abstraction is the Compliance Case: a versioned, machine-readable record created whenever a research action needs normative evaluation. It is intentionally closer to an audit dossier than to a chatbot answer. A case states what the agent proposes to do, why, with which resources, under which project and approvals, which requirements were considered, what controls were applied, what uncertainty remains, who approved exceptions and what actually happened at runtime.

This object solves several architectural problems at once. It decouples compliance reasoning from any particular LLM; it allows decisions to be reproduced after model versions change; it creates training and evaluation data without treating historical human decisions as unquestionable ground truth; and it gives institutions an inspectable unit for audit, appeal and policy improvement. Most importantly, it prevents the final natural-language explanation from becoming the only record of the decision.

The Compliance Case should be append-only at the evidence level. A decision can be superseded when facts or policy change, but the system should preserve what source version and assumptions produced the earlier result. This is essential for scientific reproducibility and for governance accountability.

The Research Context Compiler translates workflow state into facts that policy engines can reason over. Inputs include project metadata, data catalogues, model registries, ethics approvals, consent or data-use conditions, tool manifests, execution environment, collaborator roles and the proposed agent action. LLMs are useful here because scientific plans and approval documents contain natural language. But extracted facts should carry source spans and confidence, and high-impact facts should be confirmed or obtained from authoritative metadata wherever possible.

A central design goal is minimality. The compiler should not create a complete ontology of science. It needs only the facts that affect normative decisions. A gene-expression dataset may require categories such as human/personal/health/genomic, source institution, consent/permit conditions and identifiability; it does not need every biological concept in the study. A cybersecurity artefact may require exploit capability, target class, release destination and access controls; it does not need a full ontology of network protocols unless a policy rule refers to them.

Each tool call can be represented as a proposed transition from one research state to another. 'Upload dataset D to endpoint E' changes data location and third-party access. 'Publish repository R' changes audience and export status. 'Join dataset A with B' may change identifiability. 'Use model output to select trial participants' changes the consequential role of an AI system. The compliance engine evaluates the transition, not only the static assets.

This state-transition model is intuitive for researchers because it supports counterfactual alternatives. If an external upload is blocked, the system can ask whether the same computation would be allowed on an approved local model, after de-identification, or within a secure processing environment. Those alternatives should be derived from controls and rules, not improvised as generic advice.

The Research Context should include a typed Asset–Rights–Data graph rather than treating files as opaque resources. Nodes represent papers, datasets, records, code, models, prompts, derived features, experiment outputs and manuscript fragments. Edges record derivation, copying, transformation, aggregation, training use, disclosure, publication and deletion. Rights and privacy attributes attach to nodes and can propagate or change through edges according to reviewed transformation rules.

This representation makes several important questions computable. If a derived dataset contains rows originating in two licences, the system can recover both provenance chains. If a manuscript figure is generated from embargoed partner data, dissemination control can trace the dependency. If a model

was fine-tuned on a restricted corpus, release review can inspect the training lineage. If a dataset is pseudonymised, the graph records the transformation and its evidence rather than erasing the prior sensitive classification.

Graph object Policy-relevant attributes Asset Type; source; hash/version; creator/provider; confidentiality; publication state; licence/rights metadata; sensitivity; jurisdiction/source location. Derivation edge Transformation; code/tool; timestamp; input/output relation; reversibility; aggregation; privacy test; provenance confidence. Use edge Action performed on asset: mine, train, query, execute, transform, publish, redistribute; actor; purpose; environment; policy case ID. Rights assertion Permission/prohibition/duty; source; scope; assignee; action; constraints; exception; review status; conflicting assertions. Privacy assertion Personal-data/special-category status; legal/approval basis; purpose; subject group; retention; transfer constraints; safeguards; confidence. Configured rule binding Policy-pack ID/version; matching selector; enforcement mode; authority class; owner; test suite; activation/revocation status.

The Source Registry is the system's normative memory. It stores authoritative text, source authority, jurisdiction, effective interval, amendment relationships, cross-references, official language version, retrieval timestamp, hash and review status. It also stores project-specific sources such as ethics approvals, consent templates, licences and institutional policy. Free-form web retrieval may discover a source but should not automatically make it authoritative.

A key design principle is separation between text and compiled rule. The source text remains immutable evidence. A compiled rule is a maintained interpretation linked to exact source passages and reviewer decisions. When a source changes, the rule is marked for revalidation; the system can run impact analysis over active Compliance Cases before the new rule becomes an enforcement policy. This is safer than silently regenerating policies from the latest text.

The Normative Compiler converts selected provisions into a compact intermediate representation rather than directly into executable code. A rule can express authority, subject/role, action, object, modality (permission, prohibition, duty), conditions, exceptions, temporal validity, required evidence, consequence/control and source provenance. This is deliberately close to concepts found in ODRL and LegalRuleML while remaining tailored to runtime research decisions.

LLMs can propose candidate structures and identify referenced definitions. A validation workflow then tests the candidate against curated examples and counterexamples. Only reviewed rules are promoted to enforcement status. Rules that depend on open-textured interpretation can remain 'advisory': they trigger retrieval and escalation rather than deterministic blocking. This tiered compilation model is more realistic than demanding full formalisation of legislation.

The Normative Source Registry should accept not only legislation and standards but also configuration-supplied Policy Packs. A pack may be authored by an institution, research programme, consortium, laboratory or project. The pack manifest declares authority class, owner, scope selectors, effective dates, compatible context-schema version, dependencies on other packs, conflict policy, default enforcement mode and required approvals. Each compiled rule retains a pointer to the exact source clause or configuration statement from which it was derived.

A Policy Pack should be deployable like software but governed more conservatively. Activation requires validation against the current schema, regression tests over positive and negative scenarios, signature/integrity checks and—where the rule can block or permit high-consequence actions—review by an authorised human role. Deactivation and replacement must be logged. This makes arbitrary configuration practical without turning every user message into a policy mutation.

A shared intermediate representation does not require pretending that all normative sources are equivalent. It requires only a common operational skeleton: authority; subject; action; resource; purpose; conditions; temporal and jurisdiction scope; permission/prohibition/duty; exceptions; required evidence; control; enforcement state; source; confidence in extracted facts; and interpretation status. Source-specific modules can retain richer semantics. For example, a copyright module can preserve rights reservations and attribution duties; a privacy module can preserve legal basis and safeguards; a project pack can define a simple threshold rule.

Rule field Illustrative values across different sources Authority class EU law; national law; ethics approval; contract; licence; institutional mandatory policy; project mandatory rule; recommendation/preference. Subject / actor Research organisation; controller; processor; agent role; named collaborator; model provider; reviewer; instrument operator. Action Read, copy, mine, transform, train, transfer, query external model, publish, redistribute, operate instrument, disclose, delete.

Rule field Illustrative values across different sources Resource Dataset, personal record, paper, figure, code, model, biological material, instrument, result, manuscript, credential. Purpose Scientific research; verification; clinical study; internal analysis; model development; publication; quality control. Condition / trigger Data category; licence; publication state; amount/cost; destination; jurisdiction; date; model/provider class; approval present. Normative effect PERMIT; PROHIBIT; REQUIRE duty/control; REQUIRE APPROVAL; RESTRICT route/environment; ADVISE. Evidence Source passage, licence metadata, consent/permit, DPA, endpoint configuration, hash, approval token, privacy test, audit event. Interpretation state Reviewed executable; reviewed advisory; unresolved ambiguity; machine-proposed and inactive.

Applicability is the intellectual centre of the system. Retrieval answers 'which sources look relevant?' Applicability answers 'which requirements are triggered by these facts?' The resolver combines symbolic matching over context facts with semantic assistance for terms that do not align exactly. For example, an institutional policy may refer to 'unpublished confidential research material' while a workflow asset is tagged as a draft manuscript inherited from a private repository. The LLM can propose the mapping; a typed policy fact makes it stable for subsequent decisions.

Exceptions must be first-class. Many normative errors arise not from missing the general rule but from missing a condition, definition, derogation or approval. The resolver should therefore produce both supporting and defeating paths. A rule is not considered executable if required definitions or referenced conditions are unresolved. In complex domains this can be modelled using defeasible logic or priority rules, but the implementation should start with transparent precedence mechanisms rather than maximal formal expressivity.

Conflicts occur across normative layers as well as within law. A licence may permit a use that institutional policy forbids; an ethics approval may impose a narrower purpose than data-protection law would allow; a funder may require openness while a security review temporarily restricts release. The Co-Pilot should not resolve these by averaging. It should apply an explicit authority and policy hierarchy, surface unresolved contradictions and indicate which accountable role can authorise an exception.

The system should distinguish uncertainty about a contextual fact from uncertainty about a rule interpretation. 'This dataset probably contains personal data' is a data-classification uncertainty. 'This activity falls within a statutory research exemption' is an applicability interpretation. Combining both into a single confidence score obscures what a human needs to inspect. Compliance Cases should retain typed uncertainty and the evidence that could resolve it.

A practical triage strategy is to identify 'critical unknowns': facts whose alternative values change the decision from allow to block/escalate. The system asks for those facts first. This converts compliance from an exhaustive questionnaire into active information gathering. The same mechanism can be evaluated quantitatively by measuring how many questions are needed to reach a stable decision.

A stricter organisational rule may intentionally block what law permits. A licence may grant a permission while a contract narrows it; an ethics approval may authorise a protocol while privacy safeguards still constrain transfer; two project packs may simply be misconfigured. The system therefore needs a reviewed precedence relation between authority classes plus explicit conflict states.

When no safe precedence is known, the correct output is not a guessed winner but a conflict case containing the competing sources, the triggering facts and the action that is paused. This is especially important for generated policy packs: compilation can detect syntactic contradiction, but determining legal priority can itself be a normative judgement. The architecture should make such unresolved conflicts visible and measurable.

A compliance assistant becomes operationally useful only when it can translate requirements into controls. Controls may be technical—deny network egress, require an approved model endpoint, pseudonymise a dataset, log an action, restrict repository visibility—or procedural—obtain ethics approval, request a DPO decision, attach a disclosure statement, perform export screening. The Control Planner maps normative requirements to a library of institutionally approved control patterns.

Control selection should prefer minimal intervention. If the risk arises solely because an external LLM would receive confidential material, the system can propose a local governed model rather than block the research task itself. If a literature corpus cannot be redistributed, the system can keep full text inside an approved environment while exporting only allowed derived features. This 'minimal compliant transformation' is technically attractive because it turns compliance into a planning constraint rather than a terminal obstacle. It is also speculative: automatically generated alternatives must be validated to ensure they do not create new obligations or change the scientific question.

The most useful outcome of rights/privacy reasoning is often not BLOCK but transformation. A copyright duty can become attribution metadata attached to an output. A redistribution restriction can become a private storage route. A privacy rule can become local execution, field minimisation, pseudonymisation, aggregation, deletion after a fixed interval or an approved secure-processing environment. A project policy can become tool routing, a budget check or a two-person approval. This is why the Control Planner should operate over a vocabulary richer than allow/deny.

Normative finding Possible executable control External provider not approved for sensitive data Route to internal model; redact/minimise; or ESCALATE if external service is scientifically necessary. Licence permits reuse with attribution Attach provenance/attribution object to derived asset and publication workflow; verify it survives export. TDM allowed but secure retention required Store corpus in approved encrypted workspace; restrict access; set retention/review event; prohibit public redistribution. Rights reservation applies to intended mining route Exclude affected source; obtain licence; use alternative corpus; or escalate for legal review. DPIA threshold triggered Pause new high-risk processing branch; create DPIA evidence package; request DPO workflow approval. Project embargo active Permit internal analysis but block public repository/publication actions until date or approved waiver. Cost threshold reached Require budget-owner approval token; throttle or switch to lower-cost approved method.

The runtime gate should sit between research orchestration and side-effecting tools. Read-only reasoning can be largely unconstrained; actions that move data, invoke external services, modify experimental state, contact people, purchase materials or publish artefacts pass through the gate. The gate receives the proposed action plus current context, requests a decision from the compliance layer, applies required controls and emits evidence back to the Compliance Case.

This is compatible with existing agent runtimes because the gate can be exposed as middleware or a policy-decision API. It is also compatible with OPA/Cedar-style enforcement: high-level normative reasoning can compile a stable subset into low-latency policies, while uncertain cases call the richer reasoning service or human workflow. The system therefore has a fast path and a deliberative path rather than forcing an LLM round trip for every tool call.

The fast path is essential for usability. A research environment that adds seconds of latency to every local file read will be bypassed. The architecture should precompute project-level policy bundles and cache context, reserving expensive retrieval and interpretation for boundary transitions.

A literature agent wants to download and locally mine 20,000 papers, then publish a derived dataset. The Co-Pilot distinguishes lawful access to the source documents, permission to perform text-and-data mining, local retention/cache conditions and permission to redistribute full text or derived output. Instead of a binary copyright answer, it can create separate policy edges for each action and asset class. This makes the system useful even when legal interpretation is partly manual: once rights are recorded, the technical workflow can enforce them consistently.

An agent proposes to retrieve 40,000 papers, build a local full-text corpus, embed it, use it for hypothesis generation and later publish a benchmark subset. The Context Compiler identifies source repositories, access mechanisms, institution/research-organisation role, licences, rights reservations

and third-party provider use. The Rights Resolver separates operations: retrieval, TDM, local retention, external embedding API, benchmark redistribution. The same source can be acceptable for local scientific TDM but not for redistribution. The gate can therefore allow secure local mining, route embeddings to a local model, exclude sources with incompatible rights from the redistribution subset and retain an evidence report explaining each branch.

A project policy states: 'No unpublished partner data may be sent to external generative-AI services.' The rule is compiled at project setup into a selector over asset provenance/publication state and outbound model calls. Months later an agent tries to upload a derived table that contains unpublished partner measurements but no personal data. A purely privacy-focused system might allow the call. The configured pack blocks it because the restriction arises from consortium governance, not data protection. The Compliance Case records the project rule as the authority and does not falsely attribute the decision to GDPR or copyright law.

The Evidence Ledger is not a blockchain requirement. It is a tamper-evident, versioned provenance store that answers: what did the system know, which policy version did it apply, what was the decision, which controls actually ran and who approved deviations? Append-only event logs, content hashes and signed approval objects are sufficient for many deployments. Cryptographic transparency structures may be added where cross-organisation trust requires them.

Evidence should be generated automatically whenever possible. A control that requires local inference should attach the endpoint identity and execution environment; a redaction control should attach a transformation report; a human approval should include scope, expiry and the questions considered. The goal is not to produce paperwork after the fact, but to make compliance evidence a by-product of execution.

This creates a secondary scientific benefit. Automated research is already under pressure to improve provenance and reproducibility. Compliance evidence overlaps with scientific traceability: which model version was used, which data left the environment, which transformations were applied, and which human decisions shaped the workflow. The Co-Pilot can therefore share infrastructure with provenance rather than building a parallel audit universe.

A Change Monitor watches authoritative sources and institution-controlled policy repositories for new versions. LLMs can summarise candidate changes and identify affected compiled rules, but activation follows a controlled workflow. Once a new policy is approved, the system performs impact analysis over active projects and open Compliance Cases: which existing permissions become conditional, which planned actions need new evidence, and which old decisions remain valid under grandfathering or effective-date rules?

For rights-sensitive actions the Evidence Ledger should retain the licence or rights metadata observed at decision time, source URL/repository, relevant terms snapshot or hash where lawful, TDM reservation signal, identity of the rule pack, and any human rights conclusion. For privacy-sensitive actions it should retain data classification, purpose, recipient/environment, safeguards, approval/DPIA references, transformations and the resulting action. This does not mean storing unnecessary personal or copyrighted content in the audit log; evidence can be represented by hashes, identifiers and controlled references where appropriate.

The ledger should also support policy replay. If a licence changes, a project embargo expires, a regulator changes guidance or a dataset is reclassified, the system can identify prior and pending actions whose decisions depended on the affected rule or fact. This turns change management from a full manual search into a bounded re-evaluation problem. Different uncertainties should route to different authorities: protocol amendments to an ethics committee; export-control classification to a security/export officer; publication-security questions to a PI or institutional security function; licence ambiguity to legal or library specialists. The Compliance Case should contain the smallest question that changes the decision plus the evidence already gathered.

This is one area where a symbolic representation can reduce administrative load even if no autonomous compliance decision is made. A human expert should not receive a 200-page policy dump. The system can present: proposed action; decisive facts; candidate applicable provisions; conflicting

interpretations; missing evidence; and the exact decision boundary. Human answers can then be encoded as scoped policy facts rather than disappearing into email.

Overrides must be explicit and bounded. An authorised human can approve an exception for a specific project, action class and time period, but the system should not infer that the same exception applies globally. Overrides become versioned normative objects with provenance. This makes institutional governance learnable without silently training an LLM on opaque historical decisions.

A single universal rule base is neither realistic nor desirable. The proposed generality lies in the execution model, not in pretending that all disciplines share the same law. A core pack defines actors, actions, assets, purposes, approvals, rights, evidence, decisions and escalation. Jurisdiction packs add authoritative legal sources and institutional mappings. Domain packs add concepts and rules that are meaningful for a class of research.

A useful first programme would validate three deliberately different packs. A biomedical/health pack tests personal and sensitive data, ethics, secure processing, clinical/research distinctions and potentially regulated outputs. A software/cybersecurity pack tests licences, secrets, security research, dual-use or misuse, external services and open-source release. A general computational-science pack tests literature/data licences, cloud/AI use, reproducibility and collaboration without the special regimes of medicine. If the core representation needs to be rewritten for each pack, the generality claim should be abandoned or narrowed.

Table 9. The Compliance Co-Pilot is a governance compiler and action gate, not an autonomous legal authority.

Term	Meaning in the research environment
Context	Actor, role, purpose, data, jurisdiction, location, model, tool, recipient and current project state.
Norm	Versioned source of law, licence, ethics rule, institutional policy or project-specific restriction.
Applicability	Reasoned determination of which norms are relevant to the concrete action and which exceptions or conflicts matter.
Control	A technical or procedural measure that changes, blocks, routes or documents the proposed action.
Evidence	The source, interpretation, decision, action and receipt preserved for later audit and change management.
Escalation	A normal path to qualified human authority when the system lacks a justified machine decision.

The architecture is useful only if uncertainty is preserved. A policy engine can deterministically apply a rule that has already been formalised, but it cannot establish that the rule is the correct interpretation of an ambiguous law. A language model can help map project facts into a rule schema, but it should not acquire legal authority by doing so. A provenance system can show which rights statement was used, but not whether that statement was valid. Human escalation is therefore an ordinary execution path, not a failure mode hidden at the edge of the product.

This becomes increasingly important as research environments support different institutions and jurisdictions. The universal layer should remain small: context, source identity, applicability, rule versioning, control generation, evidence and escalation. Domain, jurisdiction, organisation and project packs can then specialise the system. The same machinery can support copyright, privacy, laboratory safety, research security, ethics and arbitrary local rules supplied by the institution, provided the source and authority of each rule remain visible.

CHAPTER 9

Why Future Research Infrastructure Needs Neuro-Symbolic Foundations

The problem is not symbolic versus neural

The research infrastructure of the future should not be built around a contest between neural and symbolic AI. That framing is too coarse for science. Researchers will need systems that can interpret ambiguous language, search broadly, generate unfamiliar hypotheses, manipulate formal objects, calculate precisely, enforce constraints, remember provenance, and expose reasons for consequential actions. No single computational style currently provides all of these properties well. The architectural problem is therefore one of division of labour: which kind of mechanism should be trusted with which kind of scientific responsibility, and how can the transitions between them remain inspectable?

Generative models are particularly useful at the edges of scientific problems, where vocabulary is unstable, the relevant analogy is not known in advance, or the researcher wants several plausible formulations rather than one fixed derivation. Their weakness is not that they are incapable of reasoning. It is that the reasoning they perform is difficult to bind to stable intermediate objects, difficult to replay exactly, and difficult to separate from the language in which the result is narrated. In many research tasks this is acceptable. In others, especially when the result controls downstream experiments or institutional decisions, the missing structure becomes a serious limitation.

Explicit computational structures are valuable for the opposite reason. They make some dependencies, assumptions, alternatives, and checks visible. They can be versioned, executed again, compared, or invalidated locally. Their weakness is that they normally require the problem to have been represented with enough precision before the mechanism can operate. Science repeatedly violates that condition. The object under investigation may not yet have a stable ontology, the relevant boundary may be contested, and the important insight may arise from a reformulation that no formal schema anticipated.

The most realistic future is therefore hybrid but not hybrid in the superficial sense of attaching a language model to a rules engine. The infrastructure must preserve different epistemic statuses across the boundary. A generated hypothesis remains a proposal. A retrieved passage remains evidence of what a source says. A statistical estimate remains conditional on a model and dataset. A formally checked relation remains valid only for the formalisation that was checked. The system becomes trustworthy when it can keep these distinctions operational rather than blending them into one synthetic confidence score.

What explicit structure should buy us

A useful neuro-symbolic substrate should be judged by what it makes easier for the research project. The first benefit is stable identity. Claims, assumptions, variables, sources, procedures, and validations should be addressable across time even when their human-readable wording changes. The second benefit is visible dependency. A researcher should be able to discover which conclusions depend on a corrected dataset or on a particular interpretation of a method. The third benefit is bounded authority. A component that can perform one check should not silently acquire authority over the entire scientific conclusion.

The fourth benefit is selective recomputation. Research projects change locally. A new source, revised parameter, or corrected experiment should not force the whole intellectual history to be regenerated by a model. Where dependencies are explicit, affected portions can be reconsidered while unrelated evidence remains stable. The fifth benefit is reproducible challenge. A reviewer or collaborator should be able to rerun the same check, substitute a different assumption, or branch the reasoning without asking an AI to reconstruct the original process from prose.

These properties do not require one universal formal language. They require an intermediate layer in which enough of the research process becomes machine-addressable. Different domains can choose different representations underneath. A mathematical proof, a simulation, a qualitative coding scheme,

and a laboratory protocol may each need different semantics. The common infrastructure can still preserve identities, versions, provenance, authority boundaries, and links among results. This is why the role is more important than the concrete syntax used to realise it.

The design should also preserve ambiguity when ambiguity is real. An infrastructure that forces every uncertain interpretation into a single formal branch can become less scientific than ordinary prose. Several candidate interpretations may need to coexist until evidence discriminates among them. The system should be able to say that a sentence has two operational readings, that both remain consistent with the current evidence, and that a proposed experiment would separate them. Explicitness should increase epistemic discipline, not fabricate certainty.

Meta-rational execution as a research direction

One of the deeper requirements is the ability to reason about how to reason. Future research systems will have many possible methods available: direct model inference, retrieval, simulation, statistical analysis, formal checking, specialised scientific software, external experts, or physical experiments. The infrastructure must decide which combination is appropriate under the current goal, budget, uncertainty, and risk. That decision is itself a scientific and engineering problem. It cannot be reduced to choosing the largest available model.

A meta-rational execution layer would treat reasoning procedures as selectable capabilities with known preconditions, costs, effects, and limitations. A fast approximate method may be sufficient for exploration, while a more expensive validation route is required before publication or before an automated laboratory acts. A result that remains uncertain may justify another search branch; a result that is already robust may not deserve more computation. The goal is not maximal computation but appropriate computation.

This direction also supports customisation. Different institutions can have different evidence thresholds, data restrictions, preferred tools, review practices, and acceptable degrees of automation. The infrastructure should therefore allow policies and scientific methods to be configured without rewriting the whole system. A pharmaceutical organisation, a theoretical physics group, and a small software-research company may share the same conceptual infrastructure while implementing very different approval and validation paths.

Axiologic's work on explicit process representations and meta-rational execution should be read in this context. The current experiments are useful because they test whether complex natural-language-derived intentions can be decomposed into inspectable operations, dependencies, alternatives, and validations. The important research question is not whether one notation becomes standard. It is whether such decomposition provides enough practical advantage in real research to justify the additional structure. If other representations prove more effective, the conceptual requirement remains the same.

Explainability must move from answers to projects

Most explainability work is framed around one model output: why did the system classify this image, recommend this action, or generate this answer? Scientific projects need a wider form of explanation. They need to know why a hypothesis became central, why one source was trusted more than another, why a method was selected, which failed attempts changed the plan, and why an automated agent was allowed to act. This is explanation across people, tools, models, and time.

Project-level explainability depends on the same substrate as Project Consciousness. The system must retain enough history to reconstruct decisions, but it must also distinguish recorded facts from retrospective interpretation. A decision may have several causes: evidence, cost, institutional pressure, available expertise, or simple timing. The AI maintaining the project history can propose a coherent account, but the underlying events and alternative explanations should remain inspectable. Explainability becomes a property of the research environment rather than a narrative generated after the result.

This shift matters for accountability as well. When an error appears in a large human-agent project, the useful question is rarely 'which model hallucinated?'. The more important questions are whether the system provided the wrong context, whether an agent had too much authority, whether a review gate was skipped, whether an earlier correction failed to propagate, or whether a human decision overrode available evidence. A structured project history makes these failure classes distinguishable and therefore improvable.

The same principle applies to successful research. Institutions should be able to learn from how a result was produced, not only from the result itself. Successful reasoning paths, useful review practices, productive collaboration patterns, and effective experiment sequences may become reusable organisational knowledge. In that sense, explainability is also a learning mechanism for research institutions.

Customisation is not optional

Scientific infrastructure cannot assume that all disciplines share one definition of evidence. A controlled trial, a field observation, a mathematical derivation, an ethnographic interpretation, and an industrial stress test acquire credibility in different ways. Even within one field, laboratories disagree about acceptable controls, statistical conventions, or how much prior evidence is sufficient to justify a new experiment. The future environment therefore needs domain-specific epistemic configuration rather than one global notion of correctness.

Customisation should operate at several levels. Institutions may define which agents can access which resources, which kinds of results require human approval, and which external services are permitted. Research groups may define preferred methods and validation standards. Individual projects may introduce their own ontologies, assumptions, and stopping rules. The infrastructure should make these differences explicit enough to govern without forcing every scientist to become a systems engineer.

This is another reason specialised agents are preferable to one omnipotent assistant. A specialised agent can be evaluated against a narrow responsibility. Its context can be smaller, its permissions tighter, and its failures easier to characterise. Several such agents can cooperate through the project state and shared artefacts. The underlying models may still be general-purpose, but the scientific role becomes constrained by the environment.

The long-term research challenge is to discover how much of this customisation can be learned safely from practice. A project may observe that one validation procedure repeatedly catches important errors and should become a standard gate. Another procedure may add cost without changing decisions and should be removed. The environment can learn from its own execution history, but promotion of new procedures must remain a governed event rather than an invisible self-modification.

What would falsify this direction

The neuro-symbolic and trustworthy-AI direction should remain falsifiable. It may turn out that explicit intermediate structures add too much overhead for most scientific work, that researchers rarely inspect the traces provided, or that modern language models become sufficiently reliable that some planned symbolic layers provide little additional value. It may also turn out that the main benefits come from conventional versioning, provenance, and workflow discipline rather than from new reasoning substrates. These are legitimate outcomes.

Conversely, the direction gains credibility if explicit structure produces measurable benefits: errors can be localised more quickly, corrections propagate more reliably, model replacement becomes easier, review can target the actual basis of a claim, agent actions remain bounded, and projects retain continuity across years. The relevant comparison is not philosophical elegance. It is whether scientific teams can move faster without becoming less able to explain and govern what they are doing.

The strongest experiments will therefore compare complete research tasks rather than isolated reasoning benchmarks. A team can perform the same analysis with an ordinary AI assistant and with a structured environment. Evaluators can measure correction effort, reproducibility, the ability to recover abandoned alternatives, resistance to misleading evidence, and the time required for a new team

member to understand why the project believes what it believes. Such tests connect architecture to scientific practice.

The desired outcome is a substrate that becomes less visible as it matures. Researchers should not have to think constantly about graphs, symbolic representations, or execution engines. They should experience the practical consequences: clearer histories, better-controlled agents, more meaningful review, easier branching, safer automation, and confidence that changing the underlying AI model does not erase the intellectual identity of the project. That is the standard by which this research direction should be judged.

CHAPTER 10

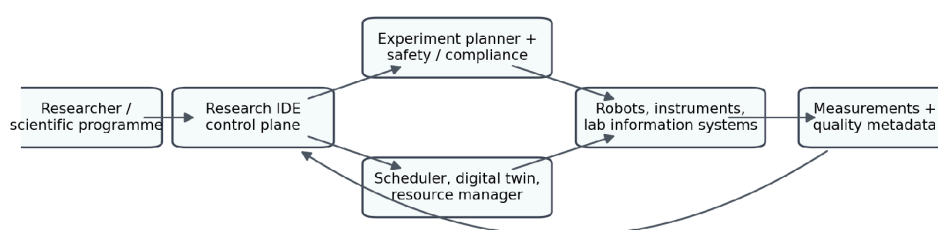
When Research Infrastructure Reaches the Laboratory

The research environment becomes more consequential when its actions leave the computer. A literature agent can be wrong without breaking a robot. An automated laboratory can waste scarce material, damage equipment, create unsafe states or generate a false empirical record if its control architecture confuses a plausible plan with an authorised protocol. The extension from digital research to automated laboratories therefore clarifies the whole thesis: a research environment must separate scientific intent, operational authority, physical execution and evidence.

The external research landscape is moving in exactly this direction. The European Commission's RAISE programme now includes Automated Scientific Discovery calls that explicitly target closed-loop systems in existing laboratories. The work programme describes AI systems that plan, execute and analyse experiments while integrating instruments, robotics, data infrastructure and human oversight. Earlier autonomous materials laboratories have already shown that machine learning, robotics and active learning can close meaningful experimental loops. These achievements are important because they demonstrate feasibility, but they also reveal how bespoke current systems remain. A general research environment cannot assume that every laboratory shares one robot API, one safety model or one ontology of experiments.

The right abstraction is a control plane above heterogeneous laboratory systems. The research environment should hold the scientific project state, the experiment specification, the current evidence and the decision rights. A laboratory adapter should expose capabilities such as prepare sample, set temperature, acquire spectrum or move plate through typed contracts. A scheduler should reason about resource availability and dependencies. A safety layer should enforce hard interlocks that no research agent can override. A digital twin can provide a simulation surface for planning and failure analysis where it is trustworthy enough. The resulting architecture resembles an operating system more than a single autonomous scientist.

Figure 8. The same Research IDE principles extend naturally into automated laboratories.



Experiment specifications become particularly important here. A protocol written for people often relies on tacit skill, local conventions and safety knowledge. An executable laboratory protocol needs explicit preconditions, materials, calibration state, instrument identities, tolerances, stopping conditions,

quality checks and recovery procedures. It also needs to distinguish what the agent is allowed to optimise from what is fixed by scientific design. If the system may vary temperature but not reagent identity, that distinction must be machine-enforceable. If a parameter is exploratory, the resulting branch should be represented as a new experimental version rather than silently altering the canonical protocol.

The laboratory therefore exposes the need for several kinds of authority. Scientific authority concerns why an experiment is worth doing. Operational authority concerns whether the experiment may run on the available equipment. Safety authority concerns whether the action is physically permissible. Compliance authority concerns whether the data, materials, participants or dual-use implications permit the work. Financial authority concerns scarce resources. A research environment can integrate these decisions without pretending they are equivalent. Different gates can be owned by different people, institutions and deterministic systems.

Closed-loop discovery adds a further complication: the result of one experiment changes what should happen next. Active learning, Bayesian optimisation and other sequential design methods already choose experiments according to expected information or objective improvement. NOVA-like scientific-state evaluation adds another layer. The system can ask not only which experiment is likely to improve a numerical objective, but which uncertainty matters for novelty, mechanism, reproducibility or strategic direction. This combination is promising precisely because it can also fail. The benefit must be measured against simpler optimisation policies on real laboratory campaigns.

A useful automated laboratory should generate richer evidence than a human-written notebook entry. Every measurement can be linked to instrument state, calibration, sample identity, software version, environmental metadata, agent plan, protocol version and quality checks. Where sensors or instruments produce large streams, the research environment should preserve enough raw or signed reference material to support later audit without requiring every byte to remain in the active project state. Research objects and provenance standards become the bridge between high-volume laboratory data and claim-level scientific reasoning.

The version graph becomes physical as well. Suppose an agent proposes a new synthesis route after a failed batch. The new protocol should fork from the old one, preserve the failure evidence, record the reason for the change and identify which parts of the process are comparable. If the new route succeeds, the accepted project state may merge the new evidence while retaining the failed branch. This history is scientifically useful because automation tends to generate many more micro-decisions than a human laboratory notebook would record. Without structured compression, the trace becomes unusable. Without trace, autonomous experimentation becomes difficult to trust.

This is another reason why agent traces are not enough. A raw sequence of model messages and tool calls contains too much implementation detail and too little scientific meaning. The research environment should transform laboratory execution into state transitions: sample created, calibration accepted, experiment deviated from protocol, measurement failed quality control, replicate confirmed result, safety gate blocked action, hypothesis weakened. The full operational trace can remain available for forensic analysis, but scientific memory should store the semantic events that alter what the project believes or permits.

Compliance in a laboratory is also different from compliance in a document workflow. A biological experiment may involve biosafety constraints. A clinical or human-subject workflow may carry consent and data-protection conditions. A materials laboratory may use controlled substances. A dual-use project may restrict who can receive methods or results. A research environment should not hardcode all of these regimes into a universal kernel. It should allow domain and institutional policy packs to bind to instrument and data actions, with hard safety mechanisms remaining below the AI layer.

The same architecture is relevant to industrial research even when no robot is involved. A pharmaceutical team may coordinate simulation, literature, proprietary assays and external contract laboratories. A semiconductor team may combine design software, expensive fabrication steps and metrology. An AI company may run large-scale training experiments whose resource cost resembles physical laboratory work. The research environment's task is to make heterogeneous research

resources addressable through common scientific state and governance, while preserving the domain-specific execution systems that already work.

Human oversight should also become more specific than a generic button labelled approve. The researcher may approve the scientific objective while a laboratory manager approves instrument use. A safety officer may define non-negotiable constraints. A data steward may approve export. A system can pre-authorise routine actions inside a bounded envelope and escalate deviations. The amount of human involvement can therefore change with risk and maturity without changing the core architecture. Autonomy becomes a property of a capability under a policy, not a global property of the laboratory.

The most ambitious future research companies may operate many such loops in parallel. Their scarce resource will not be only laboratory time. It will be the ability to determine which branches deserve to exist. Research direction, review and compliance become scheduling signals. A hypothesis with weak novelty may lose priority. A contradictory result may receive more replication budget. A compliance uncertainty may route the work to a local model or a different laboratory. A failed review check may block publication while allowing internal exploration. This is where the four early research programmes converge operationally.

Interoperability becomes unavoidable. Automated laboratories will be assembled from instruments with long lifetimes, vendor-specific APIs, scientific software, electronic laboratory notebooks, laboratory information management systems, robotics and newer agent interfaces. research environments should therefore prefer adapters and stable capability contracts over ownership of every device. Existing standards and domain protocols should be reused where possible. The new research problem is the semantic layer that relates those operations to questions, hypotheses, evidence and authority.

The architecture must also survive network and model failure. Safety-critical operations cannot depend on a remote language model remaining available. A laboratory should be able to enter a safe state if the research environment loses connectivity. Deterministic local controllers should own hard real-time and safety behaviour. High-level agents can propose and adapt plans within envelopes. This layered authority is not a compromise forced by weak AI; it is good systems engineering even if future models become dramatically more capable.

An automated laboratory also creates a powerful falsification environment for the research environment programme itself. Digital tasks can sometimes hide weak reasoning behind plausible prose. Physical experiments return stubborn evidence. If NOVA steers the laboratory toward unproductive novelty claims, the cost becomes measurable. If specification-driven protocols fail to transfer between instruments, the abstraction is wrong. If provenance is too expensive to capture, throughput drops. If compliance gates generate excessive false positives, researchers route around them. The laboratory turns architectural claims into operational hypotheses.

The long-term vision is therefore not a robotic facility run by a single omniscient agent. It is a layered scientific system in which human intent, AI planning, formal constraints, deterministic control, physical execution and empirical evidence remain separable. The research environment coordinates those layers and maintains the state that lets them learn from one another. A future laboratory may be highly autonomous, but its autonomy should be reconstructable as a sequence of authorised scientific state transitions.

Digital twins deserve careful placement in this architecture. A useful twin can predict instrument behaviour, resource conflicts or process states and allow an agent to test a protocol before physical execution. But a twin is itself a model with a validity envelope. The research environment should record which decisions relied on simulation, what calibration evidence supports the twin and where reality diverged from prediction. A high-fidelity twin can reduce cost and risk; an unqualified twin can turn simulated confidence into physical error. The same proposal-versus-authority distinction therefore applies to models of the laboratory itself.

Multi-laboratory research introduces another layer of versioning. Two sites may execute nominally identical protocols with different instruments, calibration procedures, operators and local policies. A research environment should preserve a shared protocol identity while representing site-specific adaptations explicitly. This allows cross-site reproducibility to be studied as a structured difference

rather than a binary outcome. When a result replicates only under one instrument family or preparation method, the dependency becomes part of scientific knowledge instead of being buried in supplementary prose.

Automated experiment selection also needs negative-control discipline. An agent that optimises only for confirming progress can create self-reinforcing loops in which surprising failures are treated as noise. The scheduler should reserve capacity for controls, replication and deliberately discriminating tests when the scientific state warrants them. NOVA-like direction intelligence can propose such tests, but the policy for spending real laboratory resources must remain explicit. A system should be rewarded for eliminating an attractive but false branch when that saves months of future work.

The laboratory is therefore an unusually strong falsification environment for the research environment thesis. Project-state errors become visible because physical outputs do not conform. Weak provenance becomes visible when results cannot be compared across instruments. Poor compliance models encounter real material and data constraints. Bad novelty steering wastes expensive runs. Brittle agent plans meet equipment failures and asynchronous delays. If the architecture can remain understandable and useful under these pressures, its value for less physically constrained research becomes much more credible.

The near-term objective should not be a universal autonomous laboratory. It should be a portable supervisory layer that can integrate one or two real automated facilities, preserve their native safety systems and show measurable improvement in experiment traceability, recovery, resource allocation or scientific decision quality. That is a sufficiently ambitious research target. It also creates a path from today's digital ACHILLES IDE toward future closed-loop research without requiring the project to pretend that laboratory autonomy is already a solved software problem.

The infrastructure for evaluation is arguably more important than the infrastructure for generation. AI makes it easy to produce fluent explanations, plausible hypotheses and large numbers of experimental variants. Science advances only when some of those candidates survive contact with evidence and when the project can explain why. The research environment should therefore treat evaluation as a persistent object with methods, inputs, outputs, uncertainty and provenance. An evaluation is not a hidden score attached by the orchestrator; it is part of the scientific record.

The first level is source support. A citation evaluator checks that a source exists, that the cited passage is accessible, that it actually supports the claim, and that scope or qualification has not been lost. For numerical or tabular claims, the evaluator should locate the relevant data when possible. The output can distinguish direct support, partial support, contextual relevance, contradiction and unverifiable citation. This is a bounded task that current retrieval and language systems can assist with effectively, but source licensing, inaccessible supplementary material and ambiguous terminology still require uncertainty to be visible.

The second level is reproducibility. For computational research, the platform should capture code revision, data identifiers, dependency versions, parameters, random seeds, hardware-relevant metadata, execution logs and expected outputs. A Reproduction Agent then attempts to rebuild the environment and execute the procedure independently. Benchmarks such as CORE-Bench and PaperBench show both the value and the current difficulty of this task. A failed reproduction is not automatically a falsification of the scientific claim; the cause may be an incomplete environment or missing data. The result should therefore decompose operational failure, numerical mismatch and scientific disagreement.

The third level is methodological and statistical evaluation. A research result can be perfectly reproducible and still be poorly designed. Evaluators should be able to check obvious leakage, inappropriate train-test separation, missing controls, unstable estimates, underpowered comparisons, unsupported causal interpretation and inconsistencies between stated protocol and executed analysis. Domain-specific validators are preferable where available. In software verification, formal tools may validate invariants. In mathematics, a proof assistant or solver may verify part of an argument. In materials or chemistry, physical constraints and simulation checks can reject impossible candidates.

The architecture should combine exact tools with model-based reviewers rather than pretending one evaluator covers all disciplines.

Contradiction is another first-class evaluation. The environment should distinguish evidence that fails to support a claim from evidence that actively supports an incompatible claim. A Contradiction Agent can search literature, project results and structured state for conflicts. When conflicts appear, the correct response is often not to choose a winner immediately but to create a research question: different populations, assumptions, measurement regimes or versions may explain the disagreement. This is one reason the Scientific Project State must support competing claims rather than a single truth value.

Novelty is more difficult because it is partly a property of the search process. A candidate can appear novel only because the relevant prior art was not retrieved. Language similarity is useful for retrieval but insufficient for judgement. A method can use different vocabulary and still be identical in substance; two papers can be textually similar but make different scientific contributions. Novelty also has granularity. The overall project may be known while one mechanism, dataset, proof, experimental regime or combination is new. The novelty service therefore needs to decompose the proposed contribution before it compares it.

A pragmatic novelty pipeline begins with candidate contribution decomposition. The system extracts claims, mechanisms, method choices, assumptions, datasets, target phenomena and reported advantages. It then constructs several retrieval queries: terminology and synonyms, citation neighbourhoods, ontology concepts, method structures and semantic representations. Search should include relevant paper and, where appropriate, patent corpora, with an explicit time boundary. The strongest antecedents are then compared element by element. The final output is not “novel: 0.83”. It is an evidence-backed assessment of what appears known, what appears different, which sources are closest and where search coverage is weak.

Temporal integrity is essential for evaluating the evaluator. Historical discovery tasks should freeze the literature available at the target date. Otherwise a model can retrieve the paper that later published the result and appear to “discover” it. Memory scopes must also be controlled: an agent that previously saw the target result should not reuse it in a held-out rediscovery benchmark. The same principle applies to procedural memories. Benchmark contamination is not only a foundation-model problem; it can enter through the research environment’s own long-term memory.

Novelty, correctness and significance should be separate dimensions. A false claim may be novel. A correct result may be entirely known. A technically novel modification may have no scientific significance. Conversely, reproducing a known result under a previously untested condition can be important without constituting a new theory. The IDE should avoid a single “impact score” that collapses these concepts. An evaluation portfolio is more informative: correctness evidence, reproducibility status, novelty assessment, uncertainty, expected utility and domain-expert judgement can coexist.

Current evidence justifies caution about autonomous novelty judgement. SciAgentArena reports difficulty with genuinely novel open-ended research, the shadow evaluations of open-ended AI research show poor judgement about what reaches a publishable bar, and a 2026 study of research-agent ideation found concentration around seed literature. These results do not prove that novelty evaluation is impossible. They imply that the research environment should expose novelty analysis as a documented, revisable decision-support process and should benchmark it separately from retrieval quality. The system should be rewarded for abstaining when coverage is poor.

A useful output vocabulary can be conservative. “Likely known” means strong prior art matches the substantive contribution. “Incremental or compositional novelty” means the components are known but a specific combination, regime or application differs. “Potentially novel” means no close antecedent was found within the documented search coverage. “Unresolved” means the corpus or comparison is insufficient. None of these labels replaces expert or legal novelty assessment. The value is that a researcher receives the comparison evidence early enough to redirect work before expensive experimentation or publication.

Novelty evaluation should also separate absence of evidence from evidence of absence. A search that finds no close paper in one database is weak evidence. A search that covers several relevant

corpora, follows citation neighbourhoods, tests alternative terminology and still finds no substantive antecedent is stronger, but remains conditional on coverage. The system should record the search protocol and corpus snapshot so that the assessment can be repeated. This is particularly important for fast-moving fields, non-English literature and patent-heavy domains.

Significance assessment is even more context-dependent. A result can be scientifically important because it changes a theory, enables a new method, resolves a long-standing contradiction, makes an expensive process practical or opens access to a previously unavailable regime. These forms of significance cannot be reduced reliably to predicted citations. A Significance Reviewer can help articulate possible consequences and compare them with project goals, but human experts should retain authority. The useful automation is the structured argument: what changes if the result is accepted, which downstream questions become possible, and which assumptions are affected.

Evaluation diversity should be measured experimentally. Using three LLM reviewers with nearly identical prompts may create the appearance of consensus without independent evidence. Greater independence can come from different modalities: execution, formal verification, statistical tests, literature retrieval and human review. The IDE should record evaluator dependencies so that a committee of correlated models is not mistaken for five independent confirmations. Research hypothesis: heterogeneous evaluation portfolios will be more robust than homogeneous self-consistency at a comparable cost.

Evaluation should feed orchestration but not be fully controlled by it. A branch with repeated reproduction failures may be deprioritised. A hypothesis with converging independent evidence may receive more resources. A novelty finding that the core method is already known may redirect exploration toward a different contribution. Research hypothesis: such evaluation-aware orchestration improves scientific yield per unit of compute and human attention. The claim must be tested against simpler strategies such as fixed budgets and human scheduling because complex prioritisation can introduce hidden bias.

The platform should support adversarial review as a normal operation. A researcher can request the strongest case against a favoured hypothesis, an independent reproduction, a second novelty search using different retrieval strategies, or a statistical reanalysis under alternative assumptions. These actions should be cheap to request and explicit in state. They create a culture in which challenge is an operational feature rather than a social afterthought.

Evaluators also need evaluation. Novelty agents can be calibrated on historical cut-offs and expert-labelled comparisons; citation evaluators on supported and unsupported claim-source pairs; reproduction agents on reproducible and broken packages; statistical reviewers on injected flaws. Evaluator performance should be versioned in the agent manifest. Evaluation infrastructure is trustworthy only if it exposes the reliability of its evaluators.

Trustworthy AI in a research environment should be defined operationally. The system needs to know who or what acted, what it was allowed to access, what it changed, which evidence supports the change, whether the action was reversible, and who has authority to accept it. These questions map directly to identity, capability, provenance, policy and human control. General statements about “alignment” are insufficient because research environments combine sensitive data, unpublished intellectual property, executable code, external services and potentially expensive or irreversible operations.

Identity is the first primitive. Humans, agents, model configurations, services and tools should have distinguishable identities and versions. A result should record more than “AI generated”. It should identify the agent role, model or model class, toolchain, prompt or procedure version where relevant, project state snapshot and input artefacts. This is necessary for debugging and for scientific interpretation. If a model update changes the outcome of an evaluator, the project should be able to identify which past judgements used the previous version.

Capability-scoped authority is the second primitive. An agent receives only the actions and data access needed for a task. The general copilot may be able to read broad project context but not execute arbitrary external code. A reproduction agent may execute within a sandbox but have no right

to publish. A novelty agent may search external literature but not expose confidential claims to an unapproved service. A resource scheduler can terminate jobs but not alter scientific conclusions. These scopes should be machine-enforced. Natural-language prompts can describe intent, but they are not a security boundary.

Data classification and privacy-aware routing are the third primitive. Research projects may mix public literature, unpublished drafts, commercial secrets, personal data, clinical data and partner-specific information. The environment needs policy metadata on artefacts and derived objects. When an agent requests a model or tool, the runtime checks whether the data class permits that route. Some tasks can use public cloud models; others may require a private endpoint, local model, federated computation or a specialised privacy-enhancing mechanism. The architecture should not require one privacy technology everywhere. It should make the decision explicit and auditable. Collaboration and meeting memory require consent, retention policy and clear separation between raw transcript, generated summary and accepted state changes. A meeting agent should be able to forget or restrict material according to project policy. “Always-on memory” is a long-term speculation with substantial privacy cost, not a default requirement.

Security must treat retrieved content and tools as potentially hostile. Scientific agents open papers, repositories, notebooks, web pages and datasets that can contain prompt-injection text, malicious code or poisoned metadata. Tool invocation should happen through typed interfaces, sandboxing, network restrictions and secrets isolation. Retrieved text should not be able to grant itself permissions. Agents that analyse untrusted code should operate in disposable environments. Security testing should include data exfiltration attempts, prompt injection, malicious dependencies and contaminated retrieval corpora.

Provenance is both a scientific and a trust mechanism. When an agent produces a figure, the platform should know the code, data and run that created it. When a claim is based on a paper, the platform should retain the source and relevant span where licensing permits. When an evaluator rejects a result, its method and inputs should be visible. W3C PROV and research-object practices such as RO-Crate provide useful external models for interoperable provenance and packaging, although the research environment may need a smaller operational core. The objective is not metadata completeness for its own sake; it is the ability to trace consequential results.

Human control should be risk-sensitive rather than absolute. Requiring explicit approval for every agent action would destroy the productivity benefit. Allowing unrestricted autonomy would be unsafe and scientifically opaque. A practical policy distinguishes reversible, low-risk operations from consequential ones. Reading public literature, running a sandboxed local analysis or creating a draft can often proceed automatically. Committing a high-impact project conclusion, transferring restricted data, spending a large compute budget, submitting a paper, contacting an external party or controlling equipment may require explicit authority. The thresholds can vary by project and role.

The threat model should include accidental and adversarial failure. Accidental failure includes hallucinated tool parameters, wrong file versions, stale credentials, mistaken data classification and over-broad retrieval. Adversarial failure includes prompt injection, malicious code, poisoned datasets, compromised external tools, social engineering through collaboration channels and attempts to exfiltrate sensitive context. The response differs by threat: deterministic validation can prevent malformed parameters; sandboxing limits code; provenance helps detect poisoning; policy scopes limit exfiltration. Treating all failure as “model hallucination” would miss most of the system-level risk.

Trustworthy operation also requires observability that does not become surveillance. Researchers need enough logs to reconstruct consequential actions, but project environments may contain sensitive personal behaviour and preliminary ideas. Audit should focus on actions, data flows and state changes rather than storing every internal token or speculative conversation indefinitely. Retention policies can differ by data class. This is particularly relevant for WebMeet and chat, where a permanent record of all informal discussion can discourage collaboration and create legal or privacy burdens.

Governance must remain configurable. A public computational project may allow broad autonomous execution. A clinical consortium may require local processing and explicit approval for many data operations. A laboratory may require safety interlocks before instrument control. The core platform

should express policies and capability scopes without hard-coding one institution’s rules. Profiles can instantiate stricter defaults for regulated or sensitive domains.

Incident handling should operate on scientific dependencies. Suppose a dataset is later found to contain a processing error, a tool version is compromised, or an agent leaked information to an unauthorised service. The system should identify which experiments and claims depended on the affected element, mark them for review, revoke relevant capabilities and schedule targeted re-evaluation. This is more useful than treating the incident only as an infrastructure ticket because the scientific record has been affected. Explicit project state makes dependency-aware remediation possible.

Trust also depends on uncertainty and the ability to abstain. An agent that cannot access key prior art should not declare novelty. A privacy agent that cannot classify a dataset should escalate rather than guess. A reproduction agent should distinguish “failed because environment is incomplete” from “scientific result did not reproduce”. The interface should make these states visible without turning every uncertainty into a warning. Calibrated abstention is a useful capability to benchmark.

Privacy and reproducibility sometimes conflict. Reproducibility favours complete data and environment access; privacy may prohibit sharing them. The research environment should support graduated evidence. A partner can publish code and metadata while keeping sensitive data local, expose a remote reproduction capability, provide aggregate validation, or use synthetic fixtures for part of the workflow. The important requirement is that the limitation be explicit. A “reproducible” badge should not hide the fact that external parties cannot inspect key inputs.

The broader principle is that autonomy should be earned by evidence. An agent begins with narrow scope. Reliable performance and acceptable failure modes justify wider scope. Regressions reduce authority. This mirrors how scientific instruments are qualified: trust is attached to tested behaviour under conditions, not to a personality label. The research environment can make this operational through agent lifecycle tests, policy scopes and audit. The result is more robust than attempting to decide once whether an AI system is “aligned”.

Table 11. Automated laboratories require several control planes that should not be collapsed into one autonomous agent.

Term	Meaning in the research environment
Scientific control	Chooses questions, hypotheses and the information value expected from the next experiment.
Operational control	Schedules resources, binds instrument capabilities and manages retries or recovery.
Safety control	Imposes hard physical and procedural constraints below the authority of open-ended agents.
Evidence capture	Links measurements to sample, instrument, calibration, protocol, software and project state.
Closed-loop update	Transforms validated observations into a new scientific state that can justify or block further work.

CHAPTER 11

A Research Programme for the Infrastructure of AI-Augmented Science

The preceding chapters describe a future that cannot be reached by implementing a fixed feature list. The necessary infrastructure has to be discovered experimentally because the character of AI-assisted science is itself changing. The most useful research programme is therefore organised around unresolved roles and transitions: how projects preserve memory, how agents become specialised and governable, how evidence is linked to claims, how collaborative interfaces expose parallel work, how scientific direction is evaluated, how constraints accompany execution, and how digital reasoning can safely reach physical laboratories. Technologies should be selected because they solve these problems, not because a fashionable stack has already named them.

Current Axiologic work as experimental continuity, not as the final architecture

Axiologic Research is not approaching this agenda from a blank page. The ACHILLES work has already created a first-generation researcher-oriented environment based on AssistOS, together with a copilot and specification-driven workflows. AchillesAgentLib and related runtime work explore how specialised AI capabilities can be packaged and deployed. MRP-VM investigates explicit, inspectable execution of natural-language-derived processes. SOP Lang and dynamic-circuit experiments investigate human-readable representations of dependencies and reasoning. OpenDSU contributes prior experience with provenance-oriented, sovereignty-conscious data sharing. WebMeet explores real-time human-agent collaboration. These lines have different maturity and should not be flattened into one claimed platform.

Their role in this book is evidential. They show that several of the virtual components described earlier can be built in partial form and that the research questions arise from implementation pressure rather than from architecture diagrams alone. They also expose limits. A project workspace is not yet Project Consciousness. An agent library is not yet a scientific organisation of autonomous researchers. An execution representation is not yet executable science. A provenance technology does not decide whether evidence is scientifically credible. The distance between these prototypes and the larger vision is precisely the space in which the research programme lives.

The programme should remain technologically plural. Some problems may be solved by better language models. Others may need symbolic methods, formal solvers, statistical systems, secure computation, version-control ideas, distributed data infrastructures, new interaction paradigms, or technologies that have not yet been invented. The stable object of research is the role that must be fulfilled and the evidence that a proposed mechanism fulfils it. This keeps the programme falsifiable and prevents today's implementation vocabulary from becoming tomorrow's conceptual prison.

The first category is the persistent scientific state substrate. We need representations for questions, hypotheses, claims, evidence, experiments, specifications, decisions, policies, unresolved contradictions and authority without forcing every discipline into one ontology. The strongest candidate is a typed event and object model with domain extensions, stable identities and multiple projections: relational indexes for retrieval, graphs for dependencies and provenance, vector indexes for language access, and executable views for selected procedures. The research question is which semantics must be universal and which must remain in domain packs.

The second category is the scientific compiler. Research intent arrives as natural language, documents, protocols, regulations and conversations. Before it can govern machine action it has to be transformed into explicit objects. Scientific compilation includes claim extraction, source binding, requirement extraction, method identification, normative interpretation, controlled-language normalisation and construction of executable procedures. Neural models are likely to remain important

front ends. The challenge is to make their outputs inspectable candidates and to preserve ambiguity when the source does not justify one formal interpretation.

The third category is specification-driven execution. A specification should be able to generate or constrain plans, agents, workflows, tests and laboratory actions. This requires interfaces between the specification and conventional workflow engines, the experimental meta-rational execution approach-like harnesses, SOP-style circuits, notebooks and external schedulers. It also requires a theory of acceptance: how a system knows that the work fulfilled the scientific contract rather than merely reached the end of a script. The acceptance object may combine deterministic checks, statistical criteria, evidence thresholds and human decisions.

The fourth category is scientific evidence infrastructure. Provenance today is often captured at the level of files, jobs and workflows. research environments need claim-level and decision-level provenance that can answer why a conclusion is currently believed, what would invalidate it, and which computation or observation supports it. W3C PROV and RO-Crate provide interoperability foundations. a provenance-oriented data-sharing approach and related technologies can strengthen object continuity and cross-organisational rights. The open research lies in semantic dependency, evidence sufficiency, uncertainty propagation and efficient maintenance as projects change.

The fifth category is versioned scientific memory. Git demonstrates the value of immutable histories and branches but knows nothing about scientific status. Research memory must distinguish imported fact, observation, assumption, hypothesis, failed hypothesis, derived result, reviewer issue, accepted claim and superseded claim. It must preserve temporal meaning: what was known at a given time and which sources were available. It must compress routine operational history without erasing the events necessary for reproduction and learning. This memory will be one of the defining assets of research-native organisations.

The sixth category is governed agent execution. the deployment and isolation infrastructure, the agent capability infrastructure, the experimental meta-rational execution approach and the wider agent-runtime ecosystem make many engineering mechanisms available: isolation, capability registries, model routing, permissions, budgets, traces and package lifecycles. Research adds stricter requirements. An agent role must be associated with a scientific responsibility, an evidence contract, a scope of authority and a qualification history. The system should be able to replace the model behind a role without silently changing the role itself.

The seventh category is evidence-centred review. Review Cases, scientific object compilers, executable review protocols, specialised verifiers and issue graphs need to be tested on controlled flaws and real prospective review. The field should resist the temptation to optimise for reviewer imitation or acceptance prediction. The relevant metrics are defect detection, evidence grounding, reproducibility of checks, calibration, useful abstention and human correction effort. Different domains will require different verifier portfolios, but the Review Case contract may be portable.

The eighth category is novelty and research-direction intelligence. A system needs temporal prior-art search, contribution decomposition, contradiction analysis, mechanism comparison, frontier reasoning, uncertainty and explicit corpus coverage. The Advance Case should become testable against time-machine benchmarks and prospective research decisions. The hard question is not whether an AI can call an idea novel. It is whether the structured evaluation changes resource allocation in a way that improves scientific progress without suppressing unconventional but valuable paths.

The ninth category is runtime compliance and rights. The Compliance Case, context compiler, normative source registry, policy packs, action gates and evidence ledger need robust semantics and tooling. Formal policy engines can execute well-specified rules; language models can help interpret messy facts and documents; humans remain responsible for unresolved authority. The research challenge is keeping these layers synchronised as laws, licences, institutional policy, data lineage and research purpose change during a project.

Table 12. The same research environment contracts serve different organisational scales with different dominant pressures.

Term	Meaning in the research environment
Individual	Continuity across literature, hypotheses, code, data and decisions while models and tools change.
Institution	Shared capabilities, policy, provenance, review, secure deployment and durable organisational memory.
Research company	Parallel research programmes, proprietary knowledge, automated laboratories, expensive resources and continuous prioritisation.

The tenth category is neuro-symbolic executable reasoning. a human-inspectable executable representation dynamic circuits, the experimental meta-rational execution approach and neighbouring technologies provide candidate approaches, but the category should remain broader than any one language. research environments need human-inspectable intermediate representations that can compose neural models, databases, code, statistical systems, logic engines and formal solvers while preserving provenance and budget. They also need constructors capable of selecting a small relevant computation from a large library. This is an open algorithmic problem, not a solved property of graph notation.

The eleventh category is evaluation, calibration and falsification infrastructure for the research environment itself. Every component described in this book can create new forms of automation bias if it is evaluated only by end-to-end user satisfaction. The environment should make component-level failures observable. Controlled defects, counterfactual tests, time-machine corpora, negative controls, adversarial documents, red-team data flows and prospective human studies are not auxiliary work. They are the mechanism by which an ambitious architecture avoids becoming an accumulation of impressive demonstrations.

The twelfth category is laboratory and resource control. research environments need capability abstractions for instruments, experiment specifications, scheduling, safety envelopes, digital twins, quality metadata, active experimental design and closed-loop scientific state updates. The technical foundations will vary drastically across domains. The common problem is how to relate a physical action to the scientific reason for performing it, the authority that permitted it and the evidence it produced.

A useful architecture should support very different users without becoming three different products. For an individual researcher, the dominant value is continuity: literature, hypotheses, code, data and decisions remain connected while models and tools change. For a research institution, the dominant value is governance and reuse: shared capabilities, provenance, policy, review and institutional memory. For a research-native company, the dominant value is orchestration at scale: many concurrent projects, expensive resources, automated laboratories, proprietary knowledge and high consequences for poor prioritisation. The same project-state contracts can serve all three if the permission, deployment and policy layers are sufficiently modular.

The architecture should also distinguish the workspace from the scientific services. Users should not be forced to understand every internal component. They should see a coherent project in which a claim can be inspected, an experiment can be reproduced, a branch can be compared and a decision can be challenged. Behind that interface, specialised systems can be replaced independently. This separation is strategically important because the AI stack is changing faster than scientific institutions can replace their workflows. The research environment must be model-neutral and backend-plural by design.

Higher-maturity components should therefore be treated as replaceable infrastructure, not as proof of the low-maturity research claims. the current Axiologic workspace can host a useful research workspace even if dynamic circuits fail. the deployment and isolation infrastructure can isolate agents even if the novelty evaluator remains immature. a provenance-oriented data-sharing approach can provide data-sharing and provenance mechanisms without establishing scientific correctness. the experimental meta-rational execution approach can improve operational control even if no universal executable language emerges. This modularity is a risk-control strategy for the research programme

and a product strategy for organisations that need useful capability before every research question is solved.

The converse is equally important: the new research components should not be hidden inside the mature infrastructure before they earn their place. A novelty evaluator should remain a separately testable service with explicit evidence and uncertainty. A compliance compiler should expose its source and rule versions. A review verifier should declare what class of defect it can detect. A learned circuit should remain a candidate until it passes qualification. The research environment becomes scientifically credible when its own architecture mirrors the epistemic discipline it asks of researchers.

Interoperability is another research field rather than an implementation detail. Scientific data systems, Git repositories, cloud platforms, workflow standards, laboratory software, identity systems, provenance standards and AI-agent protocols already exist. The research environment should map to them instead of replacing them, but semantic mismatches will appear. A workflow completion event is not the same as a scientific claim. A provenance record is not a validity judgment. An agent tool schema is not a scientific capability description. A Git commit is not an experiment. The integration layer must preserve these distinctions while still allowing the systems to cooperate.

Economics will decide which parts of the vision become infrastructure. Explicit provenance and formal checks impose cost. Human escalation imposes latency. Maintaining symbolic representations imposes engineering effort. Automated laboratories impose capital cost. The correct comparison is therefore not against an imaginary perfect system but against current failure costs: irreproducible results, duplicated work, weak novelty decisions, compliance incidents, lost institutional memory, insecure agents and expensive human review of machine-generated noise. A research environment earns its place when structured control reduces total research cost or increases the value of the same scientific resources.

The research programme should prefer experiments that stress the interfaces between components. A review system should consume the same project state used during research rather than reconstructing the project from a PDF. NOVA should see version history and negative results rather than only final claims. Compliance should gate the actual tool calls issued by agents. Provenance should survive model and backend changes. Laboratory runs should update the same evidence graph that informs publication. These cross-component experiments are harder than isolated benchmarks, but they test the central thesis: that integration around scientific state creates value beyond a collection of specialised copilots.

The strongest long-term outcome would be a new class of research organisation in which human scientists and AI systems share an explicit, evolving model of the work. Humans would not need to supervise every action. They would supervise the architecture of authority, the meaning of evidence and the decisions whose consequences require judgement. AI systems would not be reduced to deterministic scripts. They would retain their advantage in search, synthesis, interpretation and adaptation while being progressively compiled into more stable procedures where repeated work justifies it.

A weaker outcome would still be valuable. research environments may become the equivalent of integrated development environments for computational science without achieving broad automated discovery. Evidence-centred review may work only for specific methodological defects. Novelty intelligence may remain a decision-support tool. Compliance compilation may be viable only for well-bounded institutional policies. Neuro-symbolic circuits may become an auditable orchestration layer rather than a symbolic language model. Each of these outcomes would define a useful technology category and clarify where stronger claims failed.

The book therefore ends where the programme should begin experimentally. Build a persistent project substrate. Connect specifications to bounded execution. Capture provenance by default. Treat versions and forks as scientific objects. Deploy agents through explicit capabilities and permissions. Add evaluators whose claims can be tested independently. Introduce novelty and compliance as structured services rather than rhetorical judges. Extend the same control plane toward laboratories

only when the digital contracts are stable enough to survive physical consequences. Measure where the architecture helps and where it merely moves complexity.

Executable science, in this view, is not the replacement of science by software. It is the gradual conversion of hidden research structure into objects that humans and machines can inspect together. A future research environment should make a project easier to challenge because it is more automated, not harder. It should make institutional memory stronger because more agents are acting, not weaker. It should make the boundary of authority clearer as autonomy grows. If the programme can achieve those properties, AI can become a more powerful research partner without requiring researchers to surrender the scientific process to an opaque artificial colleague.

A credible research map should separate representation problems from decision problems. Representation research asks how scientific objects, versions, claims, evidence and policies can be encoded so different tools can operate on them without silently changing meaning. Decision research asks when a project should accept a result, spend more resources, trigger a verifier, ask a person or stop. The first can often be evaluated through interoperability, trace completeness and correction locality. The second requires prospective studies because a decision system can be internally coherent and still make research worse. Mixing the two too early makes failures difficult to diagnose.

A second separation is between local correctness and system-level usefulness. A citation verifier may be excellent at determining whether a source span supports a claim and still have little effect on overall review quality if the system selects the wrong claims to check. A novelty retriever may achieve high recall and still mislead a team if the downstream contribution model confuses terminology with mechanism. A compliance rule engine may execute its rules perfectly while the context compiler supplies the wrong actor or purpose. research environment evaluation therefore needs causal decomposition: which layer produced the error, which evidence would have prevented it, and whether the architecture allowed the correction to remain local.

Scaling research is equally important. The persistent project and institutional memory may eventually contain millions of claims, artefacts, runs and capability versions. The active state for one decision must remain much smaller. Database indexing, graph partitioning, caches, content-addressed objects and lazy materialisation are mature engineering tools, but semantic routing over research objects is not solved by storage alone. The environment must learn how to activate a relevant slice of scientific memory under a time budget while exposing what was not searched. This is one of the places where symbolic indexes, learned retrieval and dynamic-circuit construction can be compared directly.

Cross-project learning is another frontier. A research institution should benefit from prior projects without converting every historical artefact into universal truth. A failed data-cleaning procedure may become a negative example. A review protocol may transfer only within one experimental family. A specification pattern may generalise across domains. A compliance control may be organisation-wide. The system therefore needs scopes of reuse and evidence for transfer. This is more conservative than conventional model fine-tuning but potentially more governable: the institution can identify which capability changed, why it was promoted and which projects are affected.

Version graphs offer a particularly interesting learning surface. A final paper shows what survived. A version history shows what was tried, rejected, corrected and merged. For AI systems this can provide examples of counterfactual scientific work: an earlier interpretation followed by the evidence that defeated it, a method before and after a review finding, a specification whose ambiguity caused an agent failure, or a branch that was abandoned after a negative experiment. These histories may support training, retrieval or symbolic rule discovery. They also contain sensitive and copyrighted material, so any use as training data must be governed by rights, privacy, consent and institutional policy rather than assumed from technical availability.

The research map must also include human-computer interaction. A perfect internal evidence graph is useless if researchers cannot understand what the system is asking them to decide. Interfaces should reveal uncertainty, alternatives and authority without overwhelming users with execution traces. Progressive disclosure is a strong hypothesis: ordinary work remains familiar, while the system can open the evidence, version or policy structure when a decision requires it. Experiments should test

whether these interfaces improve correction and calibration rather than merely user satisfaction. An interface that makes automation feel smooth can increase risk if it hides important disagreement.

Institutional governance deserves technical treatment as well. Research organisations will need registries of approved models, tools, evaluators and data routes; qualification policies for reusable capabilities; retention and audit rules; and mechanisms for contesting machine-produced decisions. These should not be separate administrative systems glued onto the IDE. They are part of the execution semantics because they determine what an agent is permitted to do and what evidence changes project state. At the same time, the software must allow institutions to disagree. A universal research environment should support plural policy packs and authority structures rather than encode one research bureaucracy.

Economic evaluation should accompany technical evaluation from the beginning. The strongest architecture may be too expensive for routine work. Formal checks, redundant agents, cryptographic provenance and human review all consume resources. The programme should therefore measure the marginal value of stronger controls. A low-risk literature clustering task may need little more than provenance and reproducible retrieval. A clinical analysis or expensive automated experiment may justify multiple independent checks and stronger execution isolation. The goal is not maximal verification but proportional verification whose cost is visible and whose benefit can be compared with simpler baselines.

Security research becomes more difficult as scientific objects become executable. Papers, datasets, repositories and external tools are untrusted inputs that may contain prompt injections, malicious code, poisoned metadata or misleading provenance. A research environment needs strict boundaries between data and instruction, sandboxing for execution, capability-scoped credentials, policy checks on egress, signed or content-addressed capability packages and auditable model routing. Neuro-symbolic representations can help make dependencies visible, but they do not remove software vulnerabilities. Security has to be tested with adversarial workflows rather than inferred from architecture diagrams.

Privacy research intersects the entire system. Scientific memory is valuable precisely because it preserves history, yet some information must expire, remain local or be inaccessible to particular collaborators. Provenance can itself reveal sensitive relationships. Agent traces can contain secrets. Training on version histories can reproduce confidential material. Privacy-enhancing technologies such as differential privacy, federated learning, secure computation and trusted execution environments may be useful in specific subproblems, but the research environment first needs a policy and data-lineage model that can decide when those mechanisms are necessary. Privacy is therefore both a cryptographic problem and a scientific-state problem.

A mature programme should also study federation between institutions. Research increasingly crosses universities, companies, hospitals, laboratories and public infrastructures whose data and policies cannot simply be merged. A provenance-oriented data-sharing approach-style bounded objects, verifiable identity, signed provenance and rights-aware sharing provide one family of mechanisms; conventional repositories, data spaces and standards provide others. The open research question is how a project can compute over distributed evidence while retaining source authority, local policy and contestability. A global knowledge graph that erases institutional boundaries would be easier to query and harder to govern.

The automated laboratory provides the most demanding integration benchmark. It combines uncertain scientific planning with hard real-time and safety constraints, physical resources, calibration, provenance, data quality and institutional responsibility. Success should not be defined as the number of experiments a robot can run autonomously. A stronger benchmark asks whether the closed loop chooses informative experiments, records enough evidence to reconstruct the result, respects safety and policy, detects anomalies, recovers from failure and knows when to stop. The same benchmark can compare a generic agent, a fixed optimisation pipeline and a Research-IDE-controlled system.

The programme should remain deliberately plural in its technical foundations. Some domains will prefer probabilistic programming, some Datalog or knowledge graphs, some theorem provers, some workflow languages, some learned world models and some direct LLM orchestration. a

human-inspectable executable representation and the experimental meta-rational execution approach are useful because they force us to ask what a common executable layer would need, not because every future system must use their syntax. A successful research environment can expose a small shared contract for identity, inputs, outputs, effects, provenance, budgets and authority while allowing domain engines to retain their native semantics.

Standardisation should therefore follow evidence rather than precede it. Stable interfaces are valuable once repeated implementations reveal the abstractions that actually transfer. Premature standards can freeze accidental details of one prototype. The near-term objective should be conformance around a few essential contracts: project-state identity, capability description, evidence receipts, version binding, provenance export, policy decisions and human authority events. Competing internal representations can then be evaluated behind those contracts. This gives the ecosystem interoperability without requiring premature agreement about a universal scientific language.

The final research question is organisational rather than computational: what kind of institution becomes possible when scientific work is continuously machine-readable without ceasing to be human-governed? A research company could operate hundreds of parallel branches, preserve failed ideas, reuse qualified capabilities and allocate expensive experiments according to evidence rather than presentation quality. A university could make reproducibility and provenance a property of the working environment rather than a publication-stage burden. An individual researcher could retain long-term continuity across model generations. These possibilities are worth testing because they change not only how AI performs research tasks, but how research organisations remember and decide.

None of these outcomes requires accepting a single grand prediction about artificial intelligence. The programme can advance through bounded experiments whose results improve the infrastructure or narrow its claims. This is the strongest reason to treat research environments as a research field. The object is not a finished product waiting to be engineered; it is a set of technical categories at the boundary between AI systems, scientific method, software infrastructure and institutional governance. The map in this book is an attempt to make that boundary concrete enough to build, measure, falsify and revise.

Benchmark infrastructure itself should become a shared component of the programme. Every major research environment service needs test corpora with known ground truth where possible, prospective studies where ground truth emerges later, and replayable receipts that show why a result changed. A common harness can compare direct LLM prompting, tool-using agents, fixed workflows, domain-specific systems and neuro-symbolic variants under the same budgets. This avoids the recurrent mistake of comparing a carefully engineered new architecture against an intentionally weak baseline. The benchmark should expose not only final accuracy but correction effort, provenance completeness, active artefact count, latency, cost and abstention quality.

The programme also needs a theory of graceful degradation. A component can fail because a source is unavailable, a model is offline, a formalisation is ambiguous, a solver times out, a laboratory instrument is unavailable or a policy cannot be interpreted confidently. The research environment should preserve useful partial state and choose a weaker but legitimate mode where possible. A literature search may continue with a smaller corpus while marking coverage. A formal verifier may fall back to an advisory check without relabelling the result verified. A laboratory plan may stop at simulation. Graceful degradation is what turns heterogeneous research infrastructure from a brittle demonstration into an operational system.

Finally, adoption should be treated as an experimental variable. Scientists will not reorganise their work merely because an architecture is theoretically cleaner. The environment has to reduce friction in ordinary tasks before asking users to accept richer formalisation. The strongest deployment path is therefore to begin with capabilities that are immediately legible—project continuity, Git integration, reproducible agent runs, evidence capture, secure deployment and useful review checks—and allow deeper scientific objects to appear when they solve a visible problem. The programme succeeds institutionally when researchers experience the extra structure as recovered control rather than administrative overhead.

A further research area is scientific state consistency across distributed execution. Future research environments will not run on one machine or even inside one organisation. Agent sandboxes, institutional repositories, cloud analyses and laboratory controllers may all update related objects under different latency and trust conditions. Ordinary distributed-systems techniques can preserve data integrity, but scientific meaning introduces additional constraints. A late-arriving measurement may invalidate an interpretation without invalidating the historical fact that a decision was made earlier. A corrected dataset may require selective re-review rather than global rollback. Research is therefore a promising domain for semantically typed event sourcing, conflict detection and replay.

Evaluation infrastructure should be designed for counterfactual questions as well as scoring. When a system recommends continuing a research branch, the useful question is not only whether the recommendation was eventually correct. We should ask what would have happened if a weaker model had been used, if one source had been removed, if a verifier had been absent, if the compliance policy had changed, or if the researcher had received the evidence in a different form. Versioned project state and replayable capability calls make these counterfactual experiments possible. They can reveal which components actually improve scientific decisions and which merely correlate with good outcomes.

Another open category is institutional memory compression. Research organisations cannot keep every execution trace permanently in hot storage, yet aggressive summarisation can destroy exactly the negative results and decision context that later become valuable. The programme needs principled methods for deciding what can be compacted, what must remain addressable and what should be promoted into reusable knowledge. One possible direction is hierarchical memory: immutable raw receipts remain in cold storage, project-level scientific objects retain the important dependencies, and institution-level capabilities capture patterns that have survived transfer tests. The quality of such compression can be measured by whether future investigations can still reconstruct critical decisions and discover relevant prior failures.

The research map should also include economics of verification. Not every result deserves the strongest available check. Formal proof, independent replication, exhaustive prior-art search and human expert review can each be expensive. A research environment therefore needs policies that allocate verification effort according to consequence, reversibility, novelty, uncertainty and cost. This is closely related to value-of-information methods, but the object is broader than experiment selection: the system chooses how much epistemic assurance to buy before allowing a state transition. A lightweight exploratory branch may tolerate provisional evidence; a safety-critical laboratory action or a public high-impact claim may require several independent gates. The allocation policy itself must be inspectable and testable.

Finally, the programme needs longitudinal institutional experiments. Short benchmarks can establish local capabilities, but the central claim of a research environment concerns accumulated advantage: whether months or years of structured versions, qualified agents, review cases, compliance receipts and negative results make later research faster, safer or more insightful. The strongest evidence will therefore come from organisations that use the environment continuously and compare correction cost, duplicated work, time-to-decision, auditability and scientific reuse against their prior practice. Such studies are slower than model benchmarks, but they test the proposition that the infrastructure changes the productivity and epistemic resilience of research rather than merely producing impressive demonstrations.

research environments also need an explicit ontology of intervention. Different actions change science at different depths: retrieving a paper changes available context; editing a specification changes the intended work; promoting a capability changes future system behaviour; accepting a result changes project belief; publishing a claim changes an external scientific record; sending a command to a robot changes the physical world. The infrastructure should not treat these as equivalent tool calls. Research should determine which intervention classes require stronger identity, review, rollback, simulation or multi-party approval. This becomes increasingly important as one project state connects both computational and physical actions.

A related problem is policy composition. Research institutions already contain overlapping rules from data governance, cybersecurity, ethics, intellectual property, funders, journals, laboratories and contractual collaborations. AI agents make those rules operationally relevant at higher frequency because actions happen faster and cross boundaries more easily. Future research environments need ways to compose policy without creating an unreadable monolith. One promising direction is scoped policy packs that declare their authority, applicability and conflict behaviour while producing machine-readable decisions and evidence. The research question is how to keep such composition predictable when rules are incomplete, jurisdiction-dependent or mutually inconsistent.

These questions reinforce the programme's central methodological commitment: every ambitious component should expose a narrower hypothesis that can be falsified independently. Scientific-state models can be tested for correction locality. Novelty systems can be tested prospectively against later expert and experimental outcomes. Review protocols can be mutation-tested. Compliance gates can be tested against known cases and policy changes. Agent runtimes can be red-teamed for authority bypass. Version-derived learning can be evaluated for transfer. Laboratory control can begin in simulation and digital twins. A coherent architecture is valuable because it lets negative results refine one layer without invalidating the entire vision.

Table 13. The programme should distinguish engineering, research, exploration and deliberately retained human authority.

Term	Meaning in the future infrastructure
Build now	Persistent project workspaces, version history, isolated agent execution, provenance capture, collaborative interfaces and configurable policy.
Research now	Project consciousness, scientific-state models, specialised evaluators, robust novelty and direction assessment, interpretable orchestration and cross-domain evidence structures.
Longer horizon	Collective research memory, trustworthy multi-agent science, machine-actionable scientific objects, increasingly autonomous experiment systems and institutional AI research capability.
Keep bounded	Autonomous authority over publication, legal interpretation, safety-critical control and scientific truth.

CONCLUSION

The Instrument We Do Not Yet Have

Science has repeatedly become more powerful by building instruments that change what can be observed, measured or calculated. AI is likely to become such an instrument, but language models alone are not the infrastructure. They are unusually flexible components inside a larger scientific machine. The missing layer is the environment that gives those components continuity, boundaries, memory, evidence and a relationship to human authority.

The research environment described here is one possible form of that environment. It is intentionally plural. It can use different models, different formal systems, different data stores and different laboratory platforms. Its identity comes from the contracts around scientific state: specifications are versioned; actions are bounded; evidence is captured; alternatives survive; review is structured; novelty is qualified; compliance reaches the action boundary; provenance remains recoverable; and no component gains authority merely because it is fluent.

The programme may produce several outcomes rather than one universal platform. Some components will become ordinary engineering. Some will remain domain-specific research tools. Some neuro-symbolic ideas may fail. Some forms of human judgement may resist useful formalisation. That is compatible with the thesis. A research environment should be a place where such boundaries become measurable, not an architecture that assumes them away.

The decisive transition will occur when AI-generated work ceases to be exceptional and becomes the normal background rate of research. At that point the scarce resource is coherent scientific attention. The organisation that can preserve evidence, select promising branches, stop bad automation, reuse validated procedures and reconstruct why it believes what it believes will have an advantage over the organisation that merely owns the strongest model. The same is true for individual scientists.

Executable science is therefore not a promise that machines will know the truth. It is a discipline for making more of the path toward a claim inspectable and repeatable. The aspiration is an environment in which a future researcher can ask not only “what did the AI conclude?” but “which scientific object changed, what evidence caused the change, which alternatives were tested, which rules constrained the work, which version was used, what remains uncertain, and who or what had authority to accept the result?” When those questions become ordinary features of the research environment, AI will have become easier to trust precisely because it has become easier to challenge.

Selected External Sources and Axiologic Research Context

European Commission CORDIS. ACHILLES - Human-Centred Machine Learning: Lighter, Clearer, Safer, Grant Agreement 101189689. The official project description identifies the ACHILLES IDE as a specification-oriented toolkit for efficient, compliant and trustworthy AI. <https://cordis.europa.eu/project/id/101189689>

Axiologic Research. Official research and technology overview, including ACHILLES, AssistOS and a provenance-oriented data-sharing approach. <https://www.axiologic.net/>

Axiologic Research Editions. Free technology and executable-science books documenting the broader the current Axiologic workspace, Agentic AI, the experimental meta-rational execution approach, a provenance-oriented data-sharing approach and related research programmes. <https://www.axiologic.net/books.html>

the deployment and isolation infrastructure. Secure AI Agent Deployment & Development Platform. <https://www.ploinky.com/>

Lu, C. et al. Towards end-to-end automation of AI research. Nature, 2026. <https://www.nature.com/articles/s41586-026-10265-5>

Gottweis, J. et al. Accelerating scientific discovery with Co-Scientist. Nature, 2026. <https://www.nature.com/articles/s41586-026-10644-y>

Ghareeb, A. E. et al. A multi-agent system for automating scientific discovery. Nature, 2026. <https://www.nature.com/articles/s41586-026-10652-y>

European Commission. European AI in Science Strategy and RAISE: Resource for AI Science in Europe. https://research-and-innovation.ec.europa.eu/strategy/strategy-research-and-innovation/our-digital-future/european-ai-science-strategy_en and https://research-and-innovation.ec.europa.eu/strategy/strategy-research-and-innovation/our-digital-future/european-ai-science-strategy/raise-resource-ai-science-europe_en

European Commission. Horizon Europe Work Programme 2026-2027, Horizontal Activities, including HORIZON-RAISE-2027-01-01 Automated Scientific Discovery. https://ec.europa.eu/info/funding-tenders/opportunities/docs/2021-2027/horizon/wp-call/2026-2027/wp-14-horizontal-activities_horizon-2026-2027_en.pdf

NIST. Artificial Intelligence Risk Management Framework and AI Resource Center. The AI RMF 1.0 is under revision in 2026; the framework remains a useful source for context-dependent trustworthiness and TEVV. <https://www.nist.gov/itl/ai-risk-management-framework>

European Commission. AI Act transparency guidance and enforcement information, 2026. <https://digital-strategy.ec.europa.eu/en/policies/guidelines-ai-transparency-obligations> and <https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai>

W3C. PROV family of provenance specifications. <https://www.w3.org/TR/prov-overview/>

RO-Crate Community. RO-Crate Metadata Specification 1.3, Recommendation published 22 June 2026. <https://www.researchobject.org/ro-crate/specification/1.3/index.html>

GitHub. Spec Kit - an open-source specification-driven development toolkit for AI coding agents. <https://github.com/github/spec-kit> and <https://github.com/github/spec-kit/blob/main/spec-driven.md>

Git Project. Pro Git documentation on version control, commits and branching. <https://git-scm.com/book/en/v2>